



Universidad Loyola

Grado en Ingeniería Informática y Tecnologías Virtuales

TRABAJO FIN DE GRADO

Aprendizaje por refuerzo para la ciberseguridad: un defensor basado en flujos para decisiones binarias de permitir/bloquear

Autor: Javier Rivero Iglesias

Tutor: Alfonso Carlos Martínez Estudillo

Julio de 2026

Agradecimientos

Estas líneas cierran una etapa que no habría sido posible recorrer solo, y es de justicia dedicárselas a quienes la han hecho más llevadera.

A mis padres, Juan y Esmeralda, por su apoyo incondicional, por sostenerme en los momentos de duda y por aguantarme —que no es poco— durante todos estos años de estudio. Todo lo bueno que haya en este trabajo empezó, sin duda, en casa.

A mi hermano, Carlos, por soportarme en el día a día, por las conversaciones a deshoras y por recordarme, cuando más falta hacía, que siempre hay vida más allá de una pantalla.

A mi tutor, Alfonso Carlos, por guiarme durante todo este tiempo con paciencia y criterio, por sus consejos en cada reunión y por la confianza depositada en este proyecto desde el primer día.

A todos ellos, gracias de corazón: este trabajo también es vuestro.

Resumen

Este Trabajo Fin de Grado diseña y evalúa un agente autónomo de ciberseguridad que decide, flujo a flujo, si el tráfico de una red debe permitirse o bloquearse. El problema se formula como una tarea de decisión sensible al coste mediante aprendizaje por refuerzo: un agente QRDQN (*Quantile Regression Deep Q-Network*) observa un vector canónico de 152 dimensiones (76 características de flujo y 76 indicadores de presencia) y recibe recompensas asimétricas que penalizan más dejar pasar un ataque que bloquear tráfico legítimo. Con factor de descuento nulo, la formulación equivale a un bandido contextual, decisión que se declara y justifica explícitamente.

El agente se entrena sobre el conjunto CICIDS2017 con una partición aleatoria estratificada 80/20 y se somete a una escalera de validación de exigencia creciente: evaluación directa, reentrenamiento con etiquetas barajadas, partición temporal por días, cuantificación de duplicados entre particiones, comparación con una línea base de Random Forest bajo el mismo protocolo y una segunda fase de inferencia *offline* sobre tráfico capturado en un laboratorio doméstico cerrado.

En la partición aleatoria el agente alcanza un F1 de ataque de 0.984, con un recall de 0.995 y una tasa de falsos positivos del 0.66 %; no obstante, se cuantifica que el 24.86 % de las filas de prueba está duplicado en el entrenamiento, por lo que estas cifras se interpretan como una cota optimista. Bajo la partición por

días, el recall de ataque desciende a 0.530 —medido con una red reducida de comprobación—, frente al 0.081 del Random Forest, que domina en distribución pero se desploma ante el cambio temporal. La Fase 2 alcanza un F1 de 0.984 con una tasa de bloqueo del 25.2 %, con validez externa limitada. Se concluye que la generalización dura, y no el rendimiento sobre el benchmark cómodo, es la señal creíble de utilidad defensiva.

Palabras clave: aprendizaje por refuerzo; detección de intrusiones en red; CICIDS2017; generalización; QRDQN.

Abstract

This Bachelor’s Thesis designs and evaluates an autonomous cybersecurity agent that decides, flow by flow, whether network traffic should be permitted or blocked. The problem is formulated as a cost-sensitive decision task using reinforcement learning: a QRDQN (Quantile Regression Deep Q-Network) agent observes a canonical 152-dimensional vector (76 flow features and 76 presence indicators) and receives asymmetric rewards that penalise letting an attack through more heavily than blocking legitimate traffic. With a zero discount factor, the formulation is equivalent to a contextual bandit, a decision that is explicitly disclosed and justified.

The agent is trained on the CICIDS2017 dataset with a stratified 80/20 random partition and subjected to a validation ladder of increasing rigour: direct evaluation, retraining with shuffled labels, a temporal day-based split, quantification of duplicates across partitions, comparison against a Random Forest baseline under the same protocol, and a second phase of offline inference on traffic captured in a closed home laboratory.

On the random partition the agent reaches an attack F1 of 0.984, with a recall of 0.995 and a false-positive rate of 0.66 %; however, 24.86 % of the test rows are found to be duplicated in the training set, so these figures are interpreted as an optimistic upper bound. Under the day-based split, attack recall drops to 0.530 —measured with a reduced verification network—, against

0.081 for the Random Forest, which dominates in-distribution but collapses under temporal shift. Phase 2 reaches an F1 of 0.984 with a block rate of 25.2 %, with limited external validity. The thesis concludes that hard generalisation, rather than performance on a comfortable benchmark, is the credible signal of defensive usefulness.

Keywords: reinforcement learning; network intrusion detection; CICIDS2017; generalization; QRDQN.

Índice general

Agradecimientos	i
Resumen	ii
Abstract	iv
Índice de figuras	xii
Índice de tablas	xiii
Índice de algoritmos	xiv
Glosario de acrónimos y términos	xv
1. Introducción	1
1.1. Motivación	1
1.2. Definición del problema	3
1.3. Enfoque propuesto	4
1.4. Contribución de la tesis	6
1.5. Alcance y limitaciones	7
1.6. Estructura del documento	8
2. Objetivos y alcance	9

2.1. Objetivo general	9
2.2. Objetivos específicos	10
2.2.1. Representación del tráfico basada en flujos	10
2.2.2. Preprocesamiento canónico y esquema de características	11
2.2.3. Entorno de aprendizaje por refuerzo	12
2.2.4. Agente de decisión binaria basado en QRDQN	12
2.2.5. Comparación con línea base supervisada	13
2.2.6. Evaluación con control de fugas	14
2.2.7. Análisis de escala de entrenamiento y eficiencia de datos	15
2.2.8. Validación complementaria con tráfico de laboratorio	16
2.3. Alcance del trabajo	17
2.4. No objetivos	18
2.5. Resultados esperados	19
3. Estado del arte	20
3.1. Introducción y alcance del capítulo	20
3.2. Detección de intrusiones y monitorización de seguridad en red	21
3.3. Representación del tráfico de red	24
3.4. Conjuntos de datos públicos para la detección de intrusiones en red	27
3.5. CICIDS2017 como benchmark interno principal	29
3.6. Aprendizaje supervisado y aprendizaje profundo para NIDS	31
3.7. Fundamentos del aprendizaje por refuerzo	34
3.8. Q-Learning profundo y aprendizaje por refuerzo distribucional	36
3.9. Aprendizaje por refuerzo para la detección de intrusiones y la defensa cibernética	38
3.10. Clasificación como RL y decisiones de seguridad binarias	40
3.11. Riesgos metodológicos en NIDS basado en ML/DL/RL	41
3.12. Protocolos de evaluación, métricas y reproducibilidad	44
3.13. Eficiencia de datos y evaluación a escala de entrenamiento	46

3.14. Brecha de investigación y posicionamiento de esta tesis	47
4. Diseño del sistema	50
4.1. Visión general metodológica	50
4.2. Fuente de datos y representación de la entrada	52
4.2.1. CSVs de flujos de CICIDS2017	52
4.2.2. Mapeo de etiquetas binarias	53
4.2.3. Representación tabular a nivel de flujo	53
4.3. Limpieza de datos y preprocesamiento	54
4.3.1. Normalización de columnas y coerción numérica	54
4.3.2. Valores inválidos, valores ausentes e imputación	55
4.3.3. Exclusión de campos susceptibles de fuga	55
4.4. Esquema canónico de características	56
4.4.1. Características canónicas de flujo	56
4.4.2. Máscara de presencia/ausencia	57
4.4.3. Vector de observación final	59
4.5. Formulación del aprendizaje por refuerzo	59
4.5.1. Espacio de observación	61
4.5.2. Espacio de acciones	61
4.5.3. Función de recompensa	62
4.5.4. Estructura del episodio	64
4.5.5. Estado del entorno y protocolo de transición	64
4.5.6. Limitaciones de la formulación conjunto-de-datos-como-entorno	67
4.6. Agente QRDQN	67
4.6.1. Papel algorítmico	67
4.6.2. Función de pérdida de regresión cuantílica	68
4.6.3. Factor de descuento $\gamma = 0$: formulación de un solo paso	69
4.6.4. Estrategia de exploración	70
4.6.5. Configuración del entrenamiento	71

4.6.6.	Procedencia de los hiperparámetros	73
4.6.7.	Persistencia del modelo y el preprocesamiento	73
4.7.	Marco de implementación	75
4.7.1.	Pila de procesamiento de datos y aprendizaje supervisado	75
4.7.2.	Gestión de artefactos	76
4.7.3.	Controles de reproducibilidad	76
5.	Protocolo experimental	77
5.1.	Fases experimentales	77
5.1.1.	Fase 1: benchmark interno de CICIDS2017	77
5.1.2.	Fase 2: inferencia offline sobre tráfico de laboratorio	79
5.2.	Separación entrenamiento-prueba y escalado	80
5.2.1.	Partición aleatoria estratificada	80
5.2.2.	Partición por CSV/día	81
5.2.3.	Partición leave-one-CSV-out	81
5.2.4.	Estandarización ajustada sobre entrenamiento	82
5.3.	Línea base supervisada	83
5.3.1.	Línea base con Random Forest	83
5.3.2.	Comparación bajo el mismo protocolo	83
5.3.3.	Desbalanceo de clases y ponderación sensible al coste	84
5.4.	Protocolo de escala de entrenamiento y eficiencia de datos	84
5.4.1.	Protocolo de curva de aprendizaje	85
5.4.2.	Niveles de presupuesto y comparación	85
5.5.	Salidas de evaluación y reproducibilidad	86
5.5.1.	Métricas	87
5.5.2.	Escalera de validación	87
5.5.3.	Artefactos de ejecución	90
5.6.	Pipeline de inferencia offline en la Fase 2	92
5.6.1.	Mapeo canónico para flujos de laboratorio	94
5.6.2.	Diagnósticos de cambio de distribución	94

5.6.3.	Métricas de verdad cuando están disponibles	95
5.7.	Limitaciones metodológicas	95
5.7.1.	Entorno estático y toma de decisiones secuencial	95
5.7.2.	Maapeo de etiquetas binarias y detalle por familia de ataque	96
5.7.3.	Fragilidad del benchmark y generalización	96
5.7.4.	Algoritmo único y alcance de la reproducibilidad	96
6.	Resultados	98
6.1.	Configuración del modelo principal	98
6.2.	Rendimiento en distribución (partición aleatoria fija)	99
6.3.	Intervalos de confianza bootstrap	100
6.4.	Escalera de validación: comprobaciones A, B y C	102
6.5.	Análisis de duplicados y fuga en la partición aleatoria	103
6.6.	Línea base de Random Forest bajo el mismo protocolo	104
6.7.	Inferencia de la Fase 2 sobre tráfico de laboratorio	106
7.	Discusión	108
7.1.	In-distribution frente a generalización dura	108
7.2.	QRDQN frente a la línea base supervisada	109
7.3.	Comportamiento en la Fase 2 y cambio de dominio	110
8.	Limitaciones	111
8.1.	Modelo y semilla únicos	111
8.2.	Fuga en la partición aleatoria	112
8.3.	Comparación con la línea base	112
8.4.	Validez externa de la Fase 2	113
8.5.	Experimentos planificados pero no ejecutados	113
8.6.	Trabajo futuro	114
9.	Consideraciones éticas y legales	115
9.1.	Uso dual y alcance estrictamente defensivo	115

9.2. Privacidad y tratamiento de los datos	116
9.3. Riesgo operativo y asimetría de errores	116
9.4. Reproducibilidad como práctica responsable	117
Bibliografía	118

Índice de figuras

4.1. Pipeline general del sistema	52
4.2. Construcción del vector de observación	61
4.3. Interacción agente-entorno	63
4.4. Arquitectura QRDQN de la ejecución MAIN	68
5.1. Composición visual por día de CICIDS2017	79
5.2. Balance de clases en la partición aleatoria fija	80
5.3. Escalera de validación del proyecto	89
5.4. Pipeline de inferencia offline de la Fase 2	92
6.1. Curvas de entrenamiento de MAIN	99
6.2. Matriz de confusión de MAIN	100
6.3. Intervalos bootstrap de MAIN	101
6.4. Matriz de confusión de la Comprobación C	103
6.5. Duplicados y fuga entre entrenamiento y prueba	104
6.6. QRDQN frente a Random Forest	105
6.7. Matriz de confusión de Random Forest por día	106
6.8. Resultados de la Fase 2	107

Índice de tablas

3.1. Comparación de conjuntos de datos públicos de NIDS	28
4.1. Agrupación lógica de las 76 características canónicas	57
4.2. Matriz de recompensa del entorno del defensor binario	62
4.3. Lectura multiobjetivo de la función de recompensa	64
4.4. Hiperparámetros de la ejecución oficial MAIN	74
5.1. Composición por día de CICIDS2017	78
5.2. Inventario de ejecuciones oficiales	91
6.1. Métricas del modelo MAIN (partición aleatoria fija)	100
6.2. Intervalos de confianza bootstrap del modelo MAIN	101
6.3. QRDQN frente a Random Forest bajo el mismo protocolo	105

Índice de algoritmos

1.	Mapeo al esquema canónico y construcción de la observación . .	60
2.	Paso de decisión del entorno	66
3.	Entrenamiento del defensor QRDQN	72
4.	Inferencia offline robusta de la Fase 2	93

Glosario de acrónimos y términos

Acrónimos

ACK	<i>Acknowledgement</i> , flag TCP que confirma recepción de datos.
CIC	Canadian Institute for Cybersecurity (Universidad de Nuevo Brunswick), creador del conjunto de datos CICIDS2017.
CICIDS2017	Conjunto de datos de referencia para la detección de intrusiones en red, publicado por el CIC en 2017.
CPU	Unidad central de procesamiento (<i>Central Processing Unit</i>).
CSE	Communications Security Establishment, organismo coautor del conjunto CSE-CIC-IDS2018.
CSV	Valores separados por comas (<i>comma-separated values</i>), formato de los ficheros tabulares usados por el proyecto.
DDoS	Denegación de servicio distribuida (<i>Distributed Denial of Service</i>).
DDPG	<i>Deep Deterministic Policy Gradient</i> , algoritmo actor-crítico citado como antecedente de RL continuo.
DL	Aprendizaje profundo (<i>Deep Learning</i>).

DQN	<i>Deep Q-Network</i> , algoritmo de aprendizaje por refuerzo profundo basado en valores.
DRL	Aprendizaje por refuerzo profundo (<i>Deep Reinforcement Learning</i>).
ECE	<i>Explicit Congestion Notification Echo</i> , flag TCP incluido entre las estadísticas de flujo.
F1	Media armónica de la precisión y el recall.
FIN	Flag TCP que indica cierre ordenado de una conexión.
FN / FNR	Falso negativo (ataque no bloqueado) / tasa de falsos negativos.
FP / FPR	Falso positivo (tráfico benigno bloqueado) / tasa de falsos positivos.
GPU	Unidad de procesamiento gráfico (<i>Graphics Processing Unit</i>).
HIDS	Sistema de detección de intrusiones en host (<i>Host Intrusion Detection System</i>).
HTTP	<i>Hypertext Transfer Protocol</i> , protocolo de aplicación citado como ejemplo de tráfico de red.
IC	Intervalo de confianza.
ID	Identificador; en esta tesis se evita como característica cuando puede introducir fuga de información.
IDS	Sistema de detección de intrusiones (<i>Intrusion Detection System</i>).
IoT	Internet de las cosas (<i>Internet of Things</i>).
IP	Protocolo de Internet; las direcciones IP se excluyen de las características del modelo por riesgo de fuga.
IPFIX	<i>IP Flow Information Export</i> , estándar de exportación de registros de flujo.

IPS	Sistema de prevención de intrusiones (<i>Intrusion Prevention System</i>).
JSON	<i>JavaScript Object Notation</i> , formato usado para configuraciones, métricas y manifiestos de artefactos.
KDD	KDD Cup 1999, conjunto de datos histórico de detección de intrusiones.
MCC	Coficiente de correlación de Matthews (<i>Matthews Correlation Coefficient</i>).
ML	Aprendizaje automático (<i>Machine Learning</i>).
MLP	Perceptrón multicapa (<i>Multilayer Perceptron</i>).
NIDS	Sistema de detección de intrusiones en red (<i>Network Intrusion Detection System</i>).
NIST	National Institute of Standards and Technology, organismo de referencia en guías técnicas de seguridad.
NSL-KDD	Revisión depurada del conjunto KDD Cup 1999.
POMDP	Proceso de decisión de Markov parcialmente observable (<i>Partially Observable Markov Decision Process</i>).
PSH	<i>Push</i> , flag TCP incluido entre las estadísticas de flujo.
QRDQN	<i>Quantile Regression Deep Q-Network</i> , variante distribucional de DQN empleada en este trabajo.
RF	<i>Random Forest</i> , línea base supervisada de comparación.
RL	Aprendizaje por refuerzo (<i>Reinforcement Learning</i>).
RST	<i>Reset</i> , flag TCP que indica reinicio abrupto de una conexión.
RUN_ID	Identificador único de ejecución usado para localizar configuración, métricas y artefactos reproducibles.
SB3	Stable-Baselines3, biblioteca de algoritmos de RL; en el proyecto se usa junto con <code>sb3-contrib</code> .
SDN	Redes definidas por software (<i>Software-Defined Networking</i>).

SHA-256	Función hash criptográfica usada para verificar identidad de artefactos o particiones.
SIEM	Gestión de eventos e información de seguridad (<i>Security Information and Event Management</i>).
SOAR	Orquestación, automatización y respuesta de seguridad (<i>Security Orchestration, Automation and Response</i>).
SYN	Flag TCP usado en el establecimiento de conexión.
TCP	Protocolo de control de transmisión (<i>Transmission Control Protocol</i>).
TFG	Trabajo Fin de Grado.
TLS	Seguridad de la capa de transporte (<i>Transport Layer Security</i>).
TN / TP	Verdadero negativo / verdadero positivo.
UNSW-NB15	Conjunto de datos de detección de intrusiones de la Universidad de Nueva Gales del Sur (2015).
URG	<i>Urgent</i> , flag TCP incluido entre las estadísticas de flujo.

Términos

Bandido	contextual	Formulación de decisión de un solo paso en la que la acción no modifica los estados futuros. En esta tesis describe la consecuencia de usar $\gamma = 0$ sobre un conjunto de datos estático.
Benchmark		Conjunto de datos y protocolo usados como referencia de evaluación. CICIDS2017 funciona aquí como benchmark interno, no como prueba de rendimiento en producción.

Bootstrap	Técnica de remuestreo usada para estimar intervalos de confianza sobre el conjunto de prueba fijo. En esta memoria cuantifica precisión de muestreo, no varianza entre semillas de entrenamiento.
Buffer de repetición	Memoria de transiciones usada por DQN/QRDQN para muestrear minibatches durante el entrenamiento y reducir correlaciones inmediatas entre pasos.
Cambio de dominio	Diferencia entre la distribución usada para entrenar el modelo y la distribución sobre la que se aplica después; por ejemplo, CICIDS2017 frente a tráfico de laboratorio.
Despliegue activo	Integración operativa que modifica tráfico real, por ejemplo bloqueando paquetes o flujos. Este trabajo no implementa despliegue activo.
Escalador	Transformación de normalización ajustada sobre entrenamiento, aquí <code>StandardScaler</code> ; debe reutilizarse en inferencia sin recalcularse sobre datos de prueba o laboratorio.
F1	Métrica que combina precisión y recall mediante su media armónica; resulta útil cuando hay desbalanceo, pero no sustituye a la lectura de falsos positivos y falsos negativos.
Falso negativo	Error en el que un ataque se clasifica como benigno o se permite. En la tesis es el error más penalizado por la recompensa.
Falso positivo	Error en el que tráfico benigno se clasifica como ataque o se bloquea. Afecta a disponibilidad y carga operativa.

Flujo	Resumen tabular de una comunicación de red, normalmente bidireccional, descrita por estadísticas agregadas de paquetes, bytes, tiempos y flags.
Inferencia offline	Aplicación del modelo sobre ficheros ya capturados y almacenados, sin actuar sobre la red en vivo.
Leave-one-CSV-out	Protocolo de validación que retiene un CSV completo como prueba y entrena con los restantes, tratando cada fichero como una unidad de dominio.
Línea base	Modelo comparativo usado para contextualizar los resultados del método principal; en esta tesis, Random Forest es la línea base supervisada principal.
Máscara de presencia	Vector binario que indica si cada característica canónica procede de una columna fuente disponible o ha sido imputada por ausencia de columna.
Partición aleatoria	División entrenamiento/prueba por filas, normalmente estratificada. Es útil como comprobación de aprendizaje, pero puede sobreestimar generalización cuando hay duplicados o dependencia entre filas.
Partición por día	División que separa ficheros completos de CICIDS2017 según día o escenario de captura. Evalúa generalización de forma más estricta que la partición aleatoria.
Pipeline	Secuencia reproducible de pasos de procesamiento, entrenamiento, evaluación o inferencia. En esta tesis incluye carga, limpieza, mapeo canónico, escalado, modelo y artefactos de salida.
Precisión	Proporción de predicciones positivas que son correctas. Para la clase ataque, mide cuántos bloqueos o alertas corresponden realmente a ataques.

Recall	Proporción de ejemplos reales de una clase que el modelo detecta. Para la clase ataque, mide qué fracción de ataques se bloquea o identifica.
Recorte por percentiles	Limitación de valores crudos a rangos observados en entrenamiento, por ejemplo entre los percentiles 0.5 y 99.5. Sirve para acotar valores extremos en inferencia.
Recorte por z-score	Limitación de valores escalados a un umbral absoluto de desviaciones estándar. En Fase 2 ayuda a controlar valores fuera de distribución después del escalado.
Red objetivo	Copia estabilizadora de la red de valores usada en DQN/QRDQN para calcular objetivos temporales. Con $\gamma = 0$ no contribuye al objetivo de esta tesis.

Introducción

1.1 Motivación

Las infraestructuras de red modernas generan volúmenes de tráfico que superan con creces lo que los analistas humanos pueden inspeccionar manualmente. Una única red empresarial puede producir millones de registros de flujo al día, y los despliegues a gran escala agregan flujos procedentes de miles de endpoints, servicios en la nube y enlaces entre centros de datos (Scarfone et al. 2007). Distinguir la comunicación legítima de la actividad maliciosa dentro de este volumen es uno de los desafíos operativos centrales en la seguridad de redes. Las respuestas tradicionales se apoyan en sistemas de detección de intrusiones en red que aplican coincidencia de firmas, umbrales estadísticos o reglas específicas de protocolo para señalar actividad sospechosa. Estos enfoques son efectivos frente a patrones de amenaza conocidos, pero requieren actualizaciones manuales constantes de reglas, resultan en gran medida ineficaces frente a variantes novedosas de ataque y generan volúmenes de alertas que habitualmente desbordan a los equipos de seguridad.

El aprendizaje automático ofrece una perspectiva complementaria. En lugar de codificar el conocimiento experto como reglas estáticas, los modelos de ML

pueden aprender patrones estadísticos a partir de datos de tráfico etiquetados y generalizar hacia instancias de ataque no vistas previamente. A medida que el campo ha madurado, las representaciones a nivel de flujo se han convertido en un sustrato práctico para este aprendizaje: cada conexión de red se resume como un conjunto fijo de características estadísticas que cubren duración, conteos de paquetes, totales de bytes, tiempos entre llegadas, distribuciones de flags TCP y métricas basadas en tasas (Sperotto et al. 2010; Umer et al. 2017). Esta reducción hace abordable el aprendizaje a gran escala y evita las complicaciones legales y técnicas de la inspección profunda de paquetes, que resulta cada vez más ineficaz de todos modos a medida que crece la fracción de tráfico cifrado en internet.

La limitación clave del aprendizaje supervisado estándar en este contexto es que trata la detección de intrusiones como un problema de clasificación simétrico, minimizando el error de clasificación global sin tener en cuenta las consecuencias operativas de los distintos tipos de error (Axelsson 2000). En la práctica, esas consecuencias son profundamente asimétricas. Un falso negativo permite a un actor de amenazas avanzar sin ser detectado y puede conducir a exfiltración de datos, despliegue de ransomware o interrupción del servicio. Un falso positivo genera una alerta o un bloqueo de tráfico que consume tiempo del analista, pero rara vez causa daños duraderos. El aprendizaje por refuerzo proporciona un marco natural para codificar esta asimetría directamente en el objetivo de aprendizaje: al diseñar una función de recompensa que penaliza los ataques no detectados más severamente que las falsas alarmas, un agente de RL puede ser guiado hacia políticas que reflejen las prioridades de riesgo operativo real en lugar de la exactitud agregada (Sutton et al. 2018; Lee et al. 2002). Explorar cómo un algoritmo de RL distribucional como QRDQN navega estos compromisos sobre datos a nivel de flujo es la motivación central de esta tesis (Dabney et al. 2018).

1.2 Definición del problema

Aplicar el aprendizaje por refuerzo a la detección de intrusiones en red plantea varios desafíos metodológicos que van más allá de sustituir un algoritmo de RL por un clasificador estándar. El primer desafío es la calidad de los datos. Los benchmarks públicos de NIDS como CICIDS2017 son ampliamente utilizados, pero contienen artefactos conocidos de su pipeline de extracción de flujos, entre ellos ruido en las etiquetas, características duplicadas, fallos de implementación de CICFlowMeter y campos estadísticos que codifican inadvertidamente el contexto del día de captura (Ring et al. 2019; Engelen et al. 2021; Lanvin et al. 2023). Los modelos entrenados sin auditar estos artefactos pueden aprender a reconocer firmas de extracción o identificadores específicos de escenario en lugar de comportamiento de ataque genuino, produciendo puntuaciones infladas en el benchmark que no se transfieren a ningún entorno nuevo.

El segundo desafío es la fuga de información. Muchos estudios publicados de NIDS evalúan los modelos utilizando particiones aleatorias por filas sobre todo el tráfico disponible, lo que puede situar flujos altamente correlacionados a ambos lados del límite entrenamiento-prueba. Más allá de las filas duplicadas, la fuga puede introducirse a través de direcciones IP, marcas temporales absolutas, identificadores del flujo (*Flow IDs*) y números de puerto que son específicos de un escenario de ataque programado en lugar de indicativos del comportamiento malicioso en general (Arp et al. 2020; Kaufman et al. 2011). Los pasos de preprocesamiento como el escalado de características, aplicados globalmente antes de la partición, pueden contaminar adicionalmente la evaluación al permitir que la información distribucional de la partición de prueba influya en la normalización del entrenamiento. Un estudio riguroso no puede limitarse a elegir un algoritmo de alto rendimiento y reportar su exactitud sobre un conjunto de datos público dividido aleatoriamente.

El tercer desafío es la comparación justa contra un baseline de un método estándar. Los métodos de DRL pueden ser costosos de ajustar, sensibles a los hiperparámetros y más lentos en converger que los modelos supervisados tabulares. Random Forest y los ensamblados de árboles con *gradient boosting* son líneas base sólidas para la detección de intrusiones a nivel de flujo y pueden alcanzar puntuaciones casi perfectas en CICIDS2017 bajo condiciones de evaluación favorables (H. Liu et al. 2019; Apruzzese et al. 2022). Un defensor basado en RL no puede evaluarse de manera significativa sin una comparación bajo el mismo protocolo frente a dichas líneas base. La pregunta no es si QRDQN alcanza alta exactitud en una sola ejecución, sino si su formulación sensible al coste ofrece alguna ventaja empírica sobre un modelo supervisado bien ajustado bajo esquemas de características, protocolos de partición y métricas de evaluación equivalentes.

1.3 Enfoque propuesto

Esta tesis formula la tarea defensiva como un problema de aprendizaje por refuerzo offline con el conjunto de datos como entorno (Levine et al. 2020). La elección del entorno offline es deliberada: desplegar un agente de RL que deba interactuar con una red en vivo para aprender generaría riesgos de seguridad inaceptables, ya que un agente en exploración tomaría inevitablemente acciones subóptimas durante el entrenamiento. Al tratar un conjunto de datos etiquetado estático como el entorno, el agente puede entrenarse y evaluarse sin ninguna exposición a una red en vivo. El tráfico de red se procesa en un esquema canónico de flujo de 76 características fijo, independiente del formato nativo de cualquier conjunto de datos específico. Este contrato canónico separa la nomenclatura de columnas propia de la extracción del espacio de observación del modelo y hace posible procesar tanto flujos de CICIDS2017 como tráfico capturado en un laboratorio doméstico cerrado a través del mismo pipeline sin

modificación alguna.

Para gestionar datos ausentes o heterogéneos al transitar entre conjuntos de datos y herramientas de captura, se añade una máscara de presencia/ausencia de 76 valores al vector de características, produciendo una observación de 152 dimensiones para el agente. Cada posición en la máscara registra si la característica canónica correspondiente estaba presente y era válida (codificada como 1) o estaba ausente y fue reemplazada por un valor por defecto (codificada como 0). Esto hace que los huecos en los datos sean visibles tanto para el modelo como para el evaluador, evitando el problema de la imputación silenciosa en el que una red aprende sobre valores de sustitución sin ninguna indicación de que la medición subyacente no estaba disponible. Al pasar de CICIDS2017 al tráfico capturado en laboratorio, distintos pipelines de extracción pueden producir conjuntos de columnas diferentes, y la máscara de presencia/ausencia garantiza que cualquier discrepancia de ese tipo se codifique en la observación en lugar de quedar oculta a la inspección.

La tarea de decisión se mapea a un espacio de acciones binario: **PERMIT** para el tráfico clasificado como benigno, y **BLOCK** para el tráfico clasificado como ataque. Un agente *Quantile Regression Deep Q-Network* (Dabney et al. 2018), implementado a través de la biblioteca **sb3-contrib** (Stable-Baselines3 Contributors 2023), se entrena dentro de un entorno compatible con Gymnasium (Farama Foundation 2023). El entorno aplica una función de recompensa asimétrica que penaliza los falsos negativos más severamente que los falsos positivos, codificando directamente las prioridades de riesgo operativo descritas anteriormente (Cárdenas et al. 2006). El conjunto de datos CICIDS2017 (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017) sirve como benchmark interno primario, procesado a través de un pipeline de carga curado que excluye los campos propensos a fuga de información antes de que comience cualquier entrenamiento del modelo. El rendimiento se evalúa frente a una línea base de Random Forest bajo el mismo esquema de características y protocolo de partición (Engelen et al. 2021). Un segundo nivel de validación extiende la

evaluación ejecutando inferencia offline sobre características de flujo extraídas de tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial, ofreciendo una indicación parcial y de validez externa limitada del comportamiento del sistema ante un cambio de dominio.

1.4 Contribución de la tesis

La contribución principal de este trabajo es un prototipo experimental reproducible, con control de fugas y sensible al coste para evaluar decisiones de seguridad binarias basadas en RL sobre registros de flujo de red. El sistema se construye en torno a un contrato explícito de características canónicas que separa los artefactos de extracción del conjunto de datos del espacio de observación del modelo, haciendo posible auditar con precisión qué características se utilizan, cuáles se excluyen y por qué. Esta decisión de diseño aborda directamente una carencia en los trabajos existentes de NIDS basados en RL, donde las decisiones de selección de características suelen ser implícitas y difíciles de reproducir entre distintos formatos de conjunto de datos o herramientas de captura.

La segunda contribución es una escalera de validación multi-etapa que proporciona evidencia estructurada sobre cómo cambia el rendimiento del modelo a medida que las condiciones de evaluación se vuelven progresivamente más estrictas. La escalera abarca particiones aleatorias estratificadas, particiones temporales por patrón de día, evaluaciones *leave-one-CSV-out* e inferencia offline sobre tráfico capturado por el operador en un laboratorio doméstico cerrado (Lanvin et al. 2023). Cada peldaño pone a prueba un modo de fallo diferente: las particiones aleatorias prueban la capacidad de aprendizaje, las particiones temporales prueban la generalización basada en el tiempo, las particiones *CSV-out* prueban la sensibilidad a escenarios de captura específicos, y la inferencia sobre tráfico de laboratorio ofrece una comprobación parcial,

de validez externa limitada, del comportamiento ante un cambio de dominio. Reportar los resultados en todos los peldaños ofrece una imagen más completa del comportamiento del modelo de la que puede proporcionar cualquier configuración de evaluación individual.

La tercera contribución es una comparación empírica bajo el mismo protocolo entre el agente QRDQN y una línea base de Random Forest. Ambos modelos se entrenan sobre las mismas características canónicas, se evalúan bajo el mismo protocolo de partición y se valoran utilizando el mismo conjunto de métricas, que incluye exactitud (*accuracy*), precisión, *recall*, puntuación *F1*, tasa de falsos positivos (FP) y tasa de falsos negativos (FN). Esta comparación está diseñada para producir evidencia honesta sobre si la formulación de RL ofrece ventajas prácticas, sin reclamar en exceso la preparación para el despliegue ni generalizar más allá de las condiciones experimentales controladas del estudio.

1.5 Alcance y limitaciones

El alcance de esta investigación se limita estrictamente a la evaluación offline, tabular y a nivel de flujo. Las salidas PERMIT y BLOCK son clasificaciones binarias experimentales sobre registros de flujo estáticos; el sistema no realiza bloqueo en vivo de cortafuegos, descarte de paquetes, manipulación de `iptables` ni prevención de intrusiones en línea. El entorno no simula un adversario activo cuyas acciones posteriores dependan de las decisiones del defensor, lo que significa que el sistema no opera como un plano de control de toma de decisiones secuenciales completo. Estas restricciones son decisiones de diseño deliberadas que hacen que la evaluación sea abordable, reproducible y segura. No son omisiones que deban subsanarse en iteraciones futuras de este mismo prototipo.

CICIDS2017 se utiliza como benchmark interno primario para la experimentación controlada. Los resultados sobre este conjunto de datos deben interpretarse como evidencia sobre el comportamiento bajo un benchmark

específico y bien caracterizado, no como prueba de generalización a redes de producción arbitrarias. El tráfico de laboratorio, generado por el operador en un entorno doméstico cerrado y no adversarial, se utiliza como nivel de evaluación secundario para examinar el cambio de dominio; dada su procedencia, su validez externa es limitada y, además, también es inferencia offline y no despliegue en vivo. La tesis ocupa una posición metodológica entre la evaluación pura sobre benchmark y la validación de despliegue completo, apuntando a la reproducibilidad e interpretabilidad del marco experimental más que a la operatividad en producción.

1.6 Estructura del documento

El resto de este documento se organiza como sigue. El Capítulo 2 detalla los objetivos específicos, los límites del alcance, los no objetivos y los resultados esperados de la tesis. El Capítulo 3 revisa el estado del arte, cubriendo los NIDS basados en flujos, los conjuntos de datos públicos incluyendo CICIDS2017, las líneas base de aprendizaje supervisado, los fundamentos del aprendizaje por refuerzo, los trabajos previos de detección de intrusiones basados en RL y los errores metodológicos comunes en la investigación de seguridad con ML. El Capítulo 4 describe el diseño metodológico, cubriendo el pipeline de datos, el esquema canónico de características y la formulación del aprendizaje por refuerzo. Los capítulos posteriores cubren la configuración experimental y los resultados empíricos finales.

Objetivos y alcance

2.1 Objetivo general

El objetivo general de esta tesis es diseñar, implementar y evaluar rigurosamente un defensor de ciberseguridad basado en aprendizaje por refuerzo, de carácter offline, que toma decisiones binarias sensibles al coste (PERMIT o BLOCK) sobre observaciones de flujos de red estandarizados. Este objetivo se persigue mediante un *pipeline* experimental controlado que opera íntegramente sobre datos de flujo etiquetados y estáticos, priorizando el rigor metodológico y la reproducibilidad sobre cualquier pretensión de preparación para el despliegue.

El enfoque se fundamenta en la observación de que la detección de intrusiones implica costes inherentemente asimétricos: permitir un ataque conlleva consecuencias que son, en general, considerablemente más graves que las de generar una falsa alarma. Una formulación basada en recompensas hace explícita esta asimetría en el objetivo de aprendizaje, en lugar de dejarla implícita en un ajuste posterior de umbrales. La pregunta central que guía el trabajo es si un algoritmo de RL distribucional como QRDQN puede configurarse, entrenarse y evaluarse de forma que demuestre un comportamiento sensible al coste y medible sobre un benchmark público bien estudiado, manteniéndose a la vez

comparable con sólidas líneas base supervisadas bajo las mismas condiciones controladas.

2.2 Objetivos específicos

2.2.1 Representación del tráfico basada en flujos

El primer objetivo específico es extraer y normalizar datos de tráfico de red en registros tabulares de flujos, garantizando que el formato de entrada sea compatible tanto con benchmarks públicos como CICIDS2017 como con capturas de laboratorio offline. Los flujos de red son resúmenes bidireccionales de intercambios de paquetes que comparten una quintupla común, y su estructura tabular compacta y de anchura fija los hace muy adecuados como entradas para modelos de aprendizaje automático (Sperotto et al. 2010; Umer et al. 2017). Alcanzar este objetivo requiere auditar las características disponibles en los ficheros CSV generados por CICFlowMeter, identificar qué campos contienen señal estadística legítima y cuáles representan artefactos de extracción o posibles vectores de fuga de información.

Este objetivo implica también definir un contrato de características estable que tienda un puente entre las publicaciones académicas en formato CSV y las exportaciones brutas de CICFlowMeter. Un contrato estable significa que el conjunto de características utilizado como entrada al modelo no varía entre experimentos, que los campos excluidos por razones de fuga se excluyen de forma consistente, y que cualquier persona que lea el código fuente puede verificar exactamente qué medidas llegan al modelo. Esta estabilidad es un requisito previo para todos los objetivos posteriores, ya que todos los demás componentes del pipeline dependen de un espacio de observación bien definido y auditable.

2.2.2 Preprocesamiento canónico y esquema de características

El segundo objetivo específico es definir y hacer cumplir un esquema canónico estricto de 76 características junto con una máscara de presencia/ausencia de 76 valores, lo que da lugar a un vector de observación estable de 152 dimensiones. Las 76 características canónicas se extraen de estadísticas de estilo CICFlowMeter que cubren la duración del flujo, conteos de paquetes y bytes, distribuciones de tiempos de llegada, conteos de flags TCP, estadísticas de longitud de paquetes, resúmenes de periodos activos e inactivos, y métricas de tasa de flujo. Los campos que podrían exponer información de identidad específica del conjunto de datos, como las direcciones IP de origen y destino, las marcas temporales absolutas, los *Flow ID* y los números de puerto que funcionan como proxies directos del tipo de ataque, se excluyen explícitamente a nivel de esquema, en lugar de filtrarse en pasos de preprocesamiento ad hoc.

La máscara de presencia/ausencia de 76 valores registra si cada valor canónico fue medido directamente y está presente (codificado como 1) o ausente y sustituido por un marcador de posición por defecto (codificado como 0). Este diseño hace visibles las lagunas de datos tanto para el modelo como para el evaluador, evitando una situación en la que la red se entrena silenciosamente sobre ceros imputados sin ninguna indicación de que la medición subyacente no estaba disponible. Al pasar de CICIDS2017 a tráfico capturado en laboratorio, distintos pipelines de extracción pueden producir conjuntos de columnas diferentes; la máscara de presencia/ausencia garantiza que cualquier discrepancia de este tipo quede codificada en la observación en lugar de quedar oculta, de modo que el lector pueda distinguir un cero medido de uno imputado. Juntos, el esquema canónico y la máscara constituyen el contrato de observación del que depende cada componente posterior.

2.2.3 Entorno de aprendizaje por refuerzo

El tercer objetivo específico es construir una interfaz conjunto-de-datos-como-entorno compatible con Gymnasium (Farama Foundation 2023) que presente los registros de flujo al agente como una secuencia de observaciones. El entorno envuelve un conjunto de datos etiquetado estático de modo que cada llamada a `step()` devuelve un registro de flujo como observación, junto con la recompensa calculada a partir de la acción anterior del agente y la etiqueta verdadera del último registro. Esta formulación permite que los bucles de entrenamiento de RL estándar, desarrollados para entornos interactivos, se apliquen directamente a conjuntos de datos de flujo offline sin modificar el algoritmo.

El entorno debe implementar una función de recompensa asimétrica y sensible al coste que penalice los falsos negativos más que los falsos positivos, reflejando los perfiles de riesgo en ciberseguridad en los que los ataques no detectados son operacionalmente más perjudiciales que las falsas alarmas (Lee et al. 2002; Cárdenas et al. 2006). Los pesos de recompensa específicos se tratan como parámetros experimentales configurables en lugar de constantes universales fijas, ya que la relación de costes adecuada depende del modelo de amenaza y del contexto operativo del escenario de despliegue. El entorno debe también admitir reinicios reproducibles, gestión de límites de episodios y particiones del conjunto de datos configurables, de modo que la misma interfaz pueda utilizarse tanto para el entrenamiento como para la evaluación bajo distintos modos de partición.

2.2.4 Agente de decisión binaria basado en QRDQN

El cuarto objetivo específico es configurar y entrenar un agente Quantile Regression Deep Q-Network (Dabney et al. 2018), aprovechando su enfoque distribucional para la estimación del valor, para navegar el espacio de acciones

binario PERMIT/BLOCK a partir de las observaciones canónicas de 152 dimensiones. QRDQN extiende el DQN estándar estimando la distribución completa de los retornos esperados en lugar de solo la media, utilizando un conjunto de cuantiles aprendidos y una pérdida de regresión cuantílica. Esta representación distribucional resulta de interés en un contexto sensible al coste porque proporciona información más rica sobre la dispersión y la forma de los posibles resultados, no solo el retorno promedio, y en principio permite que la política sea más conservadora ante los peores casos.

La implementación práctica utiliza el módulo QRDQN de la biblioteca `sb3-contrib` (Stable-Baselines3 Contributors 2023), que proporciona una implementación estable y probada del algoritmo con arquitectura de red configurable, buffer de repetición, esquema de exploración e hiperparámetros de entrenamiento. El objetivo incluye seleccionar una configuración razonable por defecto, validar que el bucle de entrenamiento converge en el benchmark interno, y persistir todos los artefactos del modelo entrenado junto con la configuración y el estado de preprocesamiento utilizados para producirlos. La reproducibilidad exige que un modelo almacenado pueda cargarse y evaluarse de forma idéntica en una fecha posterior sin necesidad de reconstruir el pipeline de entrenamiento completo desde cero.

2.2.5 Comparación con línea base supervisada

El quinto objetivo específico es implementar una línea base supervisada robusta, utilizando principalmente Random Forest (H. Liu et al. 2019), para ofrecer una evaluación comparativa justa, con el mismo protocolo, del valor empírico del agente QRDQN. Random Forest se elige como línea base principal porque está bien establecido para la clasificación tabular, gestiona interacciones no lineales entre características sin un preprocesamiento extenso, es robusto ante características irrelevantes y tiende a obtener buenos resultados en benchmarks de NIDS basados en flujos (Apruzzese et al. 2022). Utilizar el

mismo esquema de características, el mismo protocolo de partición y las mismas métricas de evaluación tanto para el agente de RL como para la línea base es esencial; cualquier discrepancia en el preprocesamiento, la disponibilidad de características o las condiciones de evaluación comprometería la validez de la comparación.

Esta comparación cumple varios propósitos. En primer lugar, determina si la formulación QRDQN ofrece alguna ventaja empírica sobre un clasificador supervisado bien ajustado en condiciones controladas. En segundo lugar, proporciona una comprobación de cordura sobre la dificultad de la tarea en el benchmark, ya que un Random Forest que alcanza puntuaciones muy altas contextualiza lo que QRDQN logra. En tercer lugar, fundamenta cualquier discusión sobre el diseño de la función de recompensa sensible al coste en evidencia concreta. Si ambos modelos se evalúan utilizando las mismas métricas ponderadas por coste, la comparación revela si la función de recompensa de RL conduce realmente al agente hacia un punto de operación diferente al de un modelo supervisado entrenado con los mismos datos, o si ambos enfoques convergen a un comportamiento similar.

2.2.6 Evaluación con control de fugas

El sexto objetivo específico es diseñar pipelines de preprocesamiento y protocolos de validación que impidan estrictamente la fuga de información en todos los niveles de evaluación. Esto requiere la exclusión programática de campos identificativos, como las direcciones IP de origen y destino, las marcas temporales absolutas, los Flow ID y los proxies directos de puertos, en la fase de carga de datos antes de que comience cualquier entrenamiento de modelos (Arp et al. 2020; Kaufman et al. 2011). Estos campos se excluyen a nivel de esquema mediante el contrato canónico de características, en lugar de depender de un filtrado ad hoc que podría aplicarse de forma inconsistente en distintas ejecuciones o versiones del conjunto de datos.

Las transformaciones de escalado deben ajustarse exclusivamente sobre la partición de entrenamiento y luego aplicarse a las particiones de validación y prueba utilizando únicamente los parámetros aprendidos de los datos de entrenamiento. Ajustar un escalador sobre el conjunto de datos completo antes de la partición permite que la información distribucional del conjunto de prueba influya en la normalización de las características de entrenamiento, una forma sutil de fuga de información que puede inflar el rendimiento reportado en conjuntos de datos con una estructura temporal o de escenario marcada. El escalador ajustado se persiste como parte del artefacto de la ejecución para que pueda recargarse y aplicarse de forma idéntica durante la inferencia de la Fase 2 sobre tráfico de laboratorio, garantizando que se utilice la misma normalización para flujos fuera de distribución sin que ninguna información sobre su distribución llegue al estado del escalador.

2.2.7 Análisis de escala de entrenamiento y eficiencia de datos

El séptimo objetivo específico es evaluar el rendimiento tanto del agente de RL como de las líneas base supervisadas bajo distintas cantidades de datos de entrenamiento, identificando las características de eficiencia de datos de la formulación propuesta y evaluando con qué rapidez converge QRDQN en comparación con las alternativas supervisadas. Los métodos de RL profundo pueden ser muy demandantes en cuanto a muestras, y su dinámica de aprendizaje difiere sustancialmente de la de los métodos de árboles de decisión en conjunto. Comprender cuánto tráfico etiquetado requiere cada enfoque para alcanzar un rendimiento útil es prácticamente relevante: el tráfico de ataque etiquetado es costoso de recopilar, y los escenarios de despliegue reales pueden no tener acceso a los grandes conjuntos de datos equilibrados disponibles en los benchmarks académicos (Ring et al. 2019).

Los experimentos de escala de entrenamiento utilizan subconjuntos de en-

trenamiento progresivamente más grandes extraídos de la misma partición, midiendo cómo cambian la precisión, el recall, el F1, la tasa de falsos positivos y la tasa de falsos negativos a medida que crece el presupuesto. La comparación entre QRDQN y Random Forest en cada nivel de presupuesto revela si la formulación de RL tiene un requisito mayor de muestras, si converge más lentamente, o si alcanza una meseta de rendimiento final diferente. Estos resultados se interpretan como evidencia interna sobre esta combinación específica de formulación y benchmark, no como afirmaciones generales sobre la eficiencia de datos de RL en todos los contextos de NIDS.

2.2.8 Validación complementaria con tráfico de laboratorio

El octavo objetivo específico es construir un pipeline de inferencia offline robusto capaz de ingerir tráfico capturado por el operador en un entorno de laboratorio doméstico cerrado y no adversarial, y producir predicciones a partir de un modelo entrenado en CICIDS2017. Este nivel de validación ofrece una indicación parcial, de validez externa limitada, de si el esquema canónico de características y el modelo entrenado se comportan de forma razonable sobre flujos extraídos de un entorno de red distinto del benchmark, potencialmente utilizando una versión distinta de CICFlowMeter o una configuración de captura diferente. El cambio de dominio entre la distribución de entrenamiento y la distribución de inferencia es un reto fundamental para los modelos de NIDS y rara vez se evalúa en estudios que solo evalúan sobre un único conjunto de datos público (Apruzzese et al. 2022; Layeghy et al. 2023).

El pipeline de inferencia aplica recorte opcional por percentiles y z-score para gestionar los valores de características brutas fuera de distribución que caen fuera del rango observado durante el entrenamiento en CICIDS2017. El escalador ajustado de la Fase 1 se carga y aplica sin reajuste, preservando la integridad de la separación entre entrenamiento y prueba. La máscara de

presencia/ausencia en el vector de observación codifica explícitamente cualquier característica que no haya podido calcularse a partir de la captura de laboratorio, haciendo visible el caso de datos incompletos en la entrada de predicción en lugar de imputarlo silenciosamente. El resultado esperado de este objetivo es un conjunto de artefactos de predicción y métricas que permitan comparar directamente los resultados del tráfico de laboratorio con los de la validación de la Fase 1, proporcionando evidencia concreta sobre cómo varía el rendimiento bajo un cambio de dominio.

2.3 Alcance del trabajo

El trabajo está delimitado por los requisitos de una evaluación offline de un pipeline de aprendizaje automático tabular. Comprende la ingesta de registros de flujo en formato CSV tanto de CICIDS2017 como de capturas de laboratorio, la aplicación de escalado estándar y recorte opcional de valores atípicos, la ejecución de bucles de entrenamiento mediante el framework Stable Baselines3 (`sb3-contrib`), y la generación de artefactos de ejecución estructurados y reproducibles que persisten modelos, métricas, escaladores y ficheros de configuración junto a cada ejecución de entrenamiento. La experimentación interna se realiza íntegramente sobre el conjunto de datos CICIDS2017, utilizando particiones explícitas entre entrenamiento y prueba y una escalera de validación multietapa para evaluar el comportamiento del modelo bajo condiciones progresivamente más estrictas antes de probarlo con tráfico capturado externamente.

El alcance incluye explícitamente la escalera de validación multietapa descrita en los objetivos de evaluación. Esta escalera cubre particiones aleatorias estratificadas para comprobaciones básicas de aprendizaje, una comprobación de etiquetas barajadas para verificar que el rendimiento colapsa cuando las etiquetas no contienen señal real, una partición por día o por CSV para comprobar la generalización dentro de CICIDS2017, una evaluación *leave-one-CSV-out*

para comprobar la sensibilidad a ficheros de captura específicos, y la inferencia offline sobre flujos capturados por el operador en un laboratorio doméstico cerrado. Cada nivel aporta evidencia diferenciada y forma parte del alcance experimental acordado.

2.4 No objetivos

Esta tesis excluye explícitamente el desarrollo de monitorización de red en línea o la interceptación de paquetes en tiempo real. El sistema no se integrará con cortafuegos, reglas de `iptables`, controladores de Software-Defined Networking (SDN), ni sistemas de prevención de intrusiones (IPS) activos. Estas exclusiones existen porque el despliegue en producción requeriría un diseño de sistema sustancialmente diferente, una validación de seguridad y una metodología de pruebas operacionales que quedan fuera del alcance de un estudio académico controlado. La tesis está diseñada como un prototipo experimental, no como un producto operativo destinado a su uso en producción.

La tesis tampoco pretende resolver el problema más amplio de las operaciones cibernéticas adversariales multiagente o la interrupción de ataques con dependencia de secuencias, donde el estado del entorno cambia dinámicamente en respuesta a las acciones del defensor. En la formulación actual, cada registro de flujo se clasifica de forma independiente, y el conjunto de datos no reacciona a las decisiones del agente. Esta es una limitación conocida del enfoque *conjunto-de-datos-como-entorno*, e implica que el sistema no puede modelar la adaptación adversarial, la persistencia del atacante ni las cadenas de ataques multietapa que abarcan muchas decisiones individuales de flujo. Por último, el objetivo no es demostrar que QRDQN represente el estado del arte definitivo para todas las tareas de NIDS, sino evaluar su comportamiento bajo un marco experimental cuidadosamente controlado, libre de fugas y reproducible, y compararlo honestamente con líneas base supervisadas.

2.5 Resultados esperados

El resultado esperado de esta tesis es un pipeline de inferencia offline completamente funcional, documentado y reproducible, respaldado por un análisis empírico detallado. El proyecto producirá artefactos persistentes para todas las ejecuciones de entrenamiento y validación, incluyendo ficheros de modelo entrenado, escaladores ajustados, informes de métricas, salidas de validación y registros de configuración que especifican completamente las condiciones bajo las cuales se produjo cada resultado. Esta disciplina en la gestión de artefactos forma parte de la contribución metodológica: un resultado que no puede reproducirse a partir de su rastro de artefactos proporciona una evidencia más débil que uno que sí puede.

En cuanto al contenido empírico, el proyecto producirá mediciones de precisión, recall, F1, tasa de falsos positivos y tasa de falsos negativos tanto para el agente QRDQN como para la línea base de Random Forest en todos los niveles de validación. Los experimentos de escala de entrenamiento producirán curvas de aprendizaje que caracterizarán cómo evoluciona el rendimiento de cada modelo a medida que crece el presupuesto de entrenamiento. Los resultados de inferencia de la Fase 2 caracterizarán cómo varía el rendimiento cuando el modelo se enfrenta a tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial, en lugar de flujos de CICIDS2017 reservados para la prueba. En conjunto, estos resultados cuantificarán los límites de generalización de los modelos de RL entrenados en benchmarks cuando se someten a particiones temporales estrictas, evaluación *leave-one-CSV-out* y tráfico de laboratorio doméstico generado por el operador, y proporcionarán una evaluación empírica honesta de si la formulación de RL sensible al coste ofrece ventajas medibles sobre una línea base supervisada bien equiparable.

Estado del arte

3.1 Introducción y alcance del capítulo

Este capítulo mapea el trasfondo técnico de un defensor de ciberseguridad basado en aprendizaje por refuerzo que opera sobre observaciones de red a nivel de flujo. El alcance es deliberadamente más amplio que una comparación de clasificadores binarios. La tesis combina detección de intrusiones en red, representación de características de flujo, conjuntos de datos de referencia públicos, líneas base de aprendizaje supervisado, una formulación de conjunto de datos como entorno al estilo Gymnasium, QRDQN, preprocesamiento con control de fugas, comprobaciones de validación más estrictas, experimentos de eficiencia de datos, y validación planificada o preferida sobre tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial.

El capítulo sigue por tanto un embudo. Primero revisa la detección de intrusiones en red y la representación del tráfico, luego sitúa a CICIDS2017 entre los conjuntos de datos públicos, después discute las líneas base de aprendizaje supervisado y aprendizaje profundo, y finalmente enmarca la formulación de RL tanto dentro de la teoría del aprendizaje por refuerzo como en trabajos previos de RL/DRL para detección de intrusiones y defensa cibernética. Las últimas

secciones se centran en los riesgos metodológicos, los protocolos de evaluación, la evidencia sobre escala de entrenamiento y la brecha de investigación que aborda esta tesis.

La afirmación central es cautelosa. El aprendizaje por refuerzo para la detección de intrusiones ya existe, y los benchmarks públicos de NIDS son ampliamente utilizados (Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024). Esta tesis no pretende introducir el primer IDS basado en RL, demostrar que QRDQN es el estado del arte para NIDS, ni demostrar una prevención en línea lista para producción. Estudia un pipeline de decisión reproducible, offline, a nivel de flujo, con acciones PERMIT/BLOCK, y evalúa cómo se comporta dicha formulación bajo benchmarks internos controlados, comparación con líneas base y validación progresivamente más estricta.

La revisión también separa tres niveles que con frecuencia se confunden. El primero es la monitorización: recopilar paquetes, flujos, registros o eventos derivados y decidir si son sospechosos. El segundo es la predicción: entrenar un modelo que asigne representaciones de tráfico a etiquetas benignas o maliciosas. El tercero es el control: tomar acciones que modifiquen el estado futuro de la red. Esta tesis trabaja principalmente en los dos primeros niveles. Utiliza el lenguaje de PERMIT y BLOCK porque el entorno expone una decisión de seguridad binaria, pero la implementación actual no es un plano de control en línea. Esa distinción condiciona todas las afirmaciones posteriores.

3.2 Detección de intrusiones y monitorización de seguridad en red

Los sistemas de detección de intrusiones monitorizan la actividad de hosts o redes en busca de evidencias de ataques, usos indebidos o violaciones de políticas (Scarfone et al. 2007). Un sistema de detección de intrusiones en host

observa endpoints, eventos del sistema operativo, registros, procesos, cambios en la integridad de archivos o llamadas al sistema. Un sistema de detección de intrusiones en red observa el tráfico que atraviesa enlaces, taps, puertos de mirror de switches, brokers de paquetes, interfaces de red virtual o colectores de flujos. Ambas perspectivas son complementarias: el HIDS puede ver el contexto local de procesos y archivos que los sensores de red no pueden, mientras que el NIDS puede inspeccionar movimientos laterales, escaneos, tráfico de mando y control, y patrones de comunicación agregados sin instalar un agente en cada host.

También es necesario distinguir el IDS de los sistemas de prevención de intrusiones. Un IDS detecta y notifica principalmente actividad sospechosa. Su salida puede ser una alerta, un evento de registro, un registro de flujo enriquecido, un ticket, un evento SIEM o un artefacto forense. Un IPS se posiciona en línea y puede bloquear, descartar, modificar, limitar la tasa, reiniciar o redirigir el tráfico (Scarfone et al. 2007). En la práctica, los productos y arquitecturas frecuentemente difuminan esta frontera porque un motor de detección puede alimentar reglas de firewall, acciones EDR, playbooks SOAR o actualizaciones de política SDN. Para la evaluación, sin embargo, la distinción importa. Un detector puede tolerar un perfil de falsos positivos diferente al de un bloqueador en línea, dado que las alertas falsas principalmente afectan a la carga de trabajo del analista, mientras que los bloqueos falsos pueden interrumpir tráfico legítimo de negocio.

Las taxonomías clásicas de IDS distinguen enfoques basados en firmas, basados en anomalías, basados en especificaciones e híbridos (Scarfone et al. 2007). La detección basada en firmas coteja patrones maliciosos conocidos: secuencias de bytes, predicados de reglas, condiciones de protocolo, cadenas de comandos, rastros de exploits o combinaciones de comportamiento conocido. Su fortaleza es la precisión y la explicabilidad ante amenazas conocidas. Su debilidad es la dependencia del mantenimiento de reglas y la escasa cobertura de ataques nuevos, comportamiento polimórfico, cargas útiles cifradas y variaciones

específicas del entorno.

La detección basada en anomalías modela el comportamiento esperado y señala las desviaciones (Sommer et al. 2010; Ring et al. 2019). Esta es la familia más naturalmente asociada con el NIDS basado en ML, pero la palabra anomalía no debe interpretarse como sinónimo automático de aprendizaje no supervisado. Muchos estudios públicos de NIDS son clasificadores supervisados entrenados sobre flujos etiquetados como benignos y de ataque, incluso cuando la motivación más amplia es la detección de anomalías. El reto operativo es que el tráfico benigno es amplio y no estacionario, la prevalencia de ataques es baja, las etiquetas son incompletas, y los cambios en la actividad de negocio pueden parecer anómalos sin ser maliciosos.

La detección basada en especificaciones codifica el comportamiento permitido o esperado, por ejemplo máquinas de estado de protocolo, secuencias de comandos permitidas o flujos de trabajo de control industrial restringidos. Puede ser potente cuando el sistema monitorizado tiene un comportamiento estrecho y estable, pero requiere conocimiento del dominio y mantenimiento. Los sistemas híbridos combinan estas ideas, como reglas de firmas para exploits conocidos, puntuaciones estadísticas de anomalías para comportamiento de flujo inusual, y telemetría de host para confirmación en el endpoint. Los programas modernos de monitorización raramente dependen de un único paradigma de detección.

La generación de alertas y la respuesta activa forman un eje separado. Un sistema puede limitarse a producir alertas para su posterior clasificación, puede enriquecer las alertas con evidencia contextual, puede recomendar pasos de respuesta o puede desencadenar contención activa. Esta tesis se sitúa deliberadamente en el extremo conservador de ese eje. Estudia un modelo de decisión offline similar a un NIDS sobre filas de flujo extraídas. El modelo emite predicciones PERMIT/BLOCK durante los experimentos, pero no descarta paquetes, no reescribe políticas de firewall, no actualiza reglas SDN, no pone en cuarentena hosts ni opera en un bucle de retroalimentación en vivo. La

bibliografía de NIDS es, por tanto, relevante, pero las afirmaciones sobre IPS en producción quedan fuera del alcance soportado.

3.3 Representación del tráfico de red

El tráfico de red puede representarse como paquetes, bytes de carga útil, sesiones, conexiones o flujos. Las representaciones a nivel de paquete preservan la temporización, las cabeceras, los tamaños, las direcciones, los flags y, en ocasiones, parte de la carga útil por paquete. Son adecuadas para el análisis de protocolos a grano fino y el modelado de secuencias, pero resultan costosas de almacenar y procesar en redes de alta velocidad. Las representaciones a nivel de carga útil retienen el contenido de la aplicación y pueden identificar exploits o firmas de malware invisibles en los metadatos. Su utilidad está limitada por el cifrado, las restricciones de privacidad, las restricciones legales y el coste de CPU.

Las representaciones a nivel de sesión o conexión agrupan paquetes pertenecientes al mismo intercambio, habitualmente utilizando la quintupla de IP de origen, IP de destino, puerto de origen, puerto de destino y protocolo. Pueden incluir el estado de la conexión, el comportamiento del handshake, metadatos de la aplicación o atributos derivados de TLS/HTTP. Las representaciones a nivel de flujo son más compactas: agregan paquetes en una ventana temporal o registro similar a una conexión, y resumen el comportamiento mediante duración, bytes, paquetes, direccionalidad, tiempos entre llegadas, estadísticas de longitud de paquetes, conteos de flags TCP y periodos activos o inactivos (Sperotto et al. [2010](#); Umer et al. [2017](#)).

Los registros de estilo NetFlow e IPFIX son el punto de referencia operativo para la monitorización de flujos escalable. Routers, switches, sondas y colectores exportan registros de flujo compactos que pueden retenerse durante largos periodos y procesarse a escala. Estos registros son atractivos para el NIDS

porque reducen el volumen y evitan la inspección de la carga útil. También tienen limitaciones porque los despliegues exportan a menudo solo un subconjunto de los campos posibles, pueden muestrear el tráfico y pueden omitir estadísticas más ricas utilizadas en conjuntos de datos académicos. Esto crea una brecha entre la telemetría de flujos en producción y los CSV de benchmark con abundantes características.

Las características al estilo CICFlowMeter se sitúan en el lado más rico y orientado a la investigación de esa brecha. CICIDS2017 proporciona PCAPs y CSV de flujos bidireccionales generados, con características como la duración del flujo, conteos de paquetes y bytes en sentido directo e inverso, estadísticas de tiempos entre llegadas, estadísticas de longitud de paquetes, conteos de flags TCP, agregados de longitud de cabecera, tasas de flujo y temporización activa/inactiva (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Estas características son convenientes para ML tabular porque cada flujo se convierte en una fila de anchura fija, pero dependen de la reconstrucción offline de PCAPs y de las decisiones de extracción de características.

La idoneidad de la representación a nivel de flujo es, por tanto, condicional. Es adecuada para esta tesis porque el defensor implementado consume vectores numéricos de anchura fija, porque CICIDS2017 ya está disponible en formato de flujo, y porque la ruta prevista en Fase 2 también extrae CSV de flujos antes de la inferencia. Está limitada porque se pierden la semántica de la carga útil, el ordenamiento exacto de paquetes, el contenido de la aplicación y algunas dependencias temporales de múltiples flujos. Los ataques cuya evidencia es principalmente a nivel de contenido, a nivel de host o de comportamiento a largo plazo pueden ser solo indirectamente visibles para un clasificador de flujos (Sperotto et al. 2010; Umer et al. 2017).

La disponibilidad de características es otra limitación. Una característica calculada por CICFlowMeter a partir de PCAPs completos puede no estar disponible en un exportador NetFlow/IPFIX en producción, o puede calcularse

de manera diferente. Sarhan et al. abordan este problema mapeando conjuntos heterogéneos de características de NIDS hacia una representación estándar alineada con NetFlow para mejorar la generalizabilidad y la explicabilidad (Sarhan et al. 2021). Esta tesis no adopta el conjunto exacto de características estándar de Sarhan, pero sigue la misma motivación metodológica: definir un contrato de características estable, documentar qué está presente o ausente, y evitar tratar un esquema CSV específico de un benchmark como si fuera universal.

El proyecto actual utiliza un esquema canónico de flujo con 76 valores de características y una máscara de presencia/ausencia de 76 valores, produciendo observaciones de 152 dimensiones. La máscara registra si cada valor canónico estaba presente y era válido, en lugar de ocultar silenciosamente los campos ausentes o imputados. Esto importa al pasar de CICIDS2017 a tráfico de laboratorio porque diferentes pipelines de extracción pueden no producir exactamente las mismas columnas. La máscara no hace que los datos faltantes sean inofensivos, y no resuelve el cambio de dominio. Hace auditable la entrada del modelo: un lector puede distinguir un cero medido de un marcador de posición imputado.

La representación a nivel de flujo también modifica el perfil de privacidad y de fugas. La eliminación de las cargas útiles puede reducir la exposición de privacidad, pero los metadatos aún pueden identificar usuarios, servicios, horarios y escenarios. Las direcciones IP, las marcas temporales absolutas, los Flow ID, la identidad de los endpoints, la estructura del día de captura y los puertos usados como proxies de escenario pueden filtrar etiquetas en lugar de representar comportamiento de ataque (Arp et al. 2020; Kaufman et al. 2011). La tesis trata por tanto la representación y la selección de características como decisiones de diseño relevantes para la seguridad, no solo como detalles de preprocesamiento.

3.4 Conjuntos de datos públicos para la detección de intrusiones en red

Los conjuntos de datos públicos de NIDS permiten la experimentación compartida, la reproducibilidad y la comparación entre trabajos (Ring et al. 2019; Thakkar et al. 2020). También condicionan las conclusiones. Un conjunto de datos refleja su generador de tráfico, su topología, su proceso de captura, su extractor de características, sus etiquetas, sus scripts de ataque y su formato de distribución. El rendimiento sobre un benchmark público es, por tanto, evidencia sobre una tarea controlada, no evidencia directa de preparación para el despliegue (Apruzzese et al. 2022; Layeghy et al. 2023).

KDDCup99 y NSL-KDD son históricamente importantes porque dieron forma a la evaluación temprana de IDS (KDD Cup 1999; Tavallae et al. 2009). KDDCup99 contiene registros de conexión derivados del tráfico de la era DARPA y utiliza un esquema de 41 características con categorías de ataque como DoS, Probe, R2L y U2R. NSL-KDD redujo algunos problemas de redundancia y facilitó el uso del benchmark, pero mantuvo el mismo origen básico y la misma lógica de características. Estos conjuntos de datos son útiles para comprender la historia de la evaluación de IDS, pero son evidencia débil para la defensa moderna basada en flujos, porque su tráfico, comportamiento de ataques y esquema son antiguos (Tavallae et al. 2009; Ring et al. 2019). En este repositorio, NSL-KDD es, por tanto, contexto histórico, no la ruta final para el defensor actual.

UNSW-NB15 es un conjunto de datos de cyber-range más reciente generado con tráfico benigno y malicioso contemporáneo, capturas sin procesar y características de ingeniería (Moustafa y Slay 2015; Moustafa y Slay 2016). Sigue siendo útil porque amplió el campo más allá de los benchmarks al estilo KDD e incluye un conjunto más rico de categorías de ataque. CICIDS2017 y CSE-CIC-IDS2018 son conjuntos de datos de la familia CIC ampliamente utilizados para

la investigación de NIDS basada en flujos; ambos proporcionan escenarios de ataque generados en laboratorio y características de flujo derivadas de capturas de paquetes (Sharafaldin et al. 2018; Communications Security Establishment et al. 2018). Bot-IoT y ToN_IoT amplían el panorama hacia el tráfico IoT e IIoT, donde el comportamiento de los dispositivos, la telemetría y los patrones de ataque difieren de las redes empresariales (Koroniotis et al. 2019; Moustafa, Ahmed et al. 2020; Alsaedi et al. 2020).

Conjunto de datos	Papel como benchmark	Representación	Principales fortalezas	Principales advertencias
KDDCup99	Benchmark histórico heredado	Registros de conexión con 41 características artesanales	Muy ampliamente utilizado; línea base sencilla para la bibliografía clásica de IDS (KDD Cup 1999)	Tráfico obsoleto; redundancia; débil como aproximación al NIDS moderno (Ring et al. 2019)
NSL-KDD	Benchmark heredado depurado	Mismo esquema de conexión al estilo KDD	Reduce algunos problemas de registros duplicados; útil para comparación histórica (Tavallaei et al. 2009)	Sigue heredando las suposiciones antiguas de DARPA/KDD y el diseño de características
UNSW-NB15	Conjunto de datos moderno de laboratorio/cyber range	PCAPs, registros y características tabulares de ingeniería	Taxonomía de ataques más contemporánea y características más ricas (Moustafa y Slay 2015)	Entorno controlado; no constituye evidencia directa de producción
CICIDS2017	Benchmark interno principal de la tesis	PCAPs y CSV de flujos CICFlowMeter	PCAPs públicos; múltiples días de ataque; ricas características de flujo (Sharafaldin et al. 2018)	Problemas conocidos de etiquetado, extracción, duplicación y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023)
CSE-CIC-IDS2018	Benchmark de mayor tamaño de la familia CIC	PCAPs y flujos al estilo CICFlowMeter	Colaboración más amplia CIC/CSE; comparación moderna útil (Communications Security Establishment et al. 2018)	Advertencias similares sobre generación en laboratorio y artefactos de flujo (L. Liu et al. 2022)
Bot-IoT	Benchmark de IoT/botnets	PCAPs, flujos al estilo Argus, exportaciones CSV	Escenarios de botnet, escaneo, DoS/DDoS y exfiltración orientados a IoT (Koroniotis et al. 2019)	Fuerte desbalanceo y artefactos de topología específicos del conjunto de datos
ToN_IoT	Benchmark de telemetría IoT/IIoT	Flujos de red, registros de host y telemetría de sensores	Perspectiva multimodal de IoT/IIoT (Alsaedi et al. 2020)	Entorno de nicho; los artefactos de identificadores y escenarios siguen siendo posibles

Tabla 3.1: Comparación compacta de los conjuntos de datos públicos de NIDS relevantes para la tesis.

La Tabla 3.1 no es una clasificación por rango. Muestra por qué la elección

del conjunto de datos debe alinearse con la solidez de las afirmaciones. Los benchmarks históricos ayudan a situar el campo; los conjuntos de datos modernos de laboratorio permiten la comparación controlada; los conjuntos de datos de IoT prueban suposiciones de tráfico diferentes; el tráfico de laboratorio externo o similar al de producción es necesario para una evidencia de generalización más sólida. Ninguno de estos conjuntos de datos debe tratarse como un sustituto completo de todas las redes reales. Su valor es máximo cuando se usan con preprocesamiento transparente, líneas base sólidas y protocolos de validación que prueben el cambio de distribución en lugar de únicamente el rendimiento sobre distribuciones emparejadas aleatorias.

3.5 CICIDS2017 como benchmark interno principal

CICIDS2017 es el benchmark interno principal de esta tesis porque proporciona tráfico benigno y de ataque etiquetado, PCAPs sin procesar y CSV de flujos generados con características al estilo CICFlowMeter (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Fue diseñado para mejorar conjuntos de datos de IDS más antiguos mediante perfiles de tráfico contemporáneos, perfiles de actividad benigna realistas y múltiples escenarios de ataque (Sharafaldin et al. 2018; Sharafaldin et al. 2019). Estas propiedades lo hacen adecuado para la experimentación controlada con modelos de NIDS a nivel de flujo.

Su estructura también lo hace atractivo para el trabajo de tesis. El conjunto de datos abarca múltiples días de captura y familias de ataque, y está disponible tanto como capturas de paquetes sin procesar como en archivos CSV generados. Los PCAPs están más cerca de la evidencia de medición original. Los CSV son más fáciles de usar para ML porque CICFlowMeter ya ha reconstruido los flujos

bidireccionales y calculado las características estadísticas. Esta comodidad es también un riesgo: una vez que los investigadores trabajan solo con CSV resumidos, pueden dejar de comprobar cómo se generaron los flujos, cómo se asignaron las etiquetas y si las filas preservan la independencia entre las particiones de entrenamiento y prueba.

CICFlowMeter reconstruye flujos a partir de trazas de paquetes utilizando claves de endpoint, direccionalidad, lógica de timeout y cálculos de características. Pequeñas diferencias en los parámetros de extracción pueden cambiar las filas resultantes, las duraciones, los conteos y las etiquetas. Engelen et al. muestran que CICIDS2017 contiene problemas prácticos en la reconstrucción de flujos y la consistencia del conjunto de datos (Engelen et al. 2021). Lanvin et al. reportan además errores que afectan al rendimiento de detección y muestran que correcciones aparentemente pequeñas del conjunto de datos pueden producir diferencias métricas significativas (Lanvin et al. 2023; Lanvin et al. 2022). Liu et al. analizan la prevalencia de errores en CIC-IDS-2017 y CSE-CIC-IDS-2018, mientras que Rosay et al. ofrecen un análisis más amplio y una discusión orientada a la reconstrucción (L. Liu et al. 2022; Rosay et al. 2022). La crítica de Dube es especialmente relevante para la disciplina en las afirmaciones, porque cuestiona las interpretaciones habituales de las puntuaciones elevadas sobre los datos resumidos de CIC-IDS 2017 (Dube 2024).

Estas críticas no hacen que CICIDS2017 sea inútil. Hacen que el uso irresponsable sea más fácil de identificar. Los patrones de uso incorrecto más comunes incluyen tratar los CSV pregenerados como verdad incuestionable, usar particiones aleatorias por filas en todos los días, conservar identificadores o marcas temporales que exponen el contexto del escenario, informar solo de la exactitud, ignorar el desbalanceo de clases y no documentar si los datos son originales, corregidos, reconstruidos o curados. Las variantes corregidas o reconstruidas pueden ser valiosas, pero no son intercambiables con la distribución original de CSV. Una tesis debe nombrar qué variante utiliza y evitar mezclar resultados de diferentes variantes como si procedieran de un único conjunto de datos.

El repositorio actual responde a estas preocupaciones mediante una copia del conjunto de datos rastreada y curada, una copia de referencia sin rastrear, y una limpieza a nivel de cargador en `src/load_cicids2017.py`. El adaptador realiza coerción numérica, tratamiento de valores no válidos, mapeo canónico y exclusión anti-fugas de campos que no deben ser características del modelo. El código también ajusta el escalado sobre la partición de entrenamiento y aplica la transformación ajustada a la partición de prueba, en lugar de ajustar sobre todos los datos. El mapeo canónico en `src/canonical_schema.py` excluye IPs, marcas temporales, Flow ID y puertos específicos de las características canónicas.

El código de evaluación también admite particiones aleatorias, particiones por CSV/día y validación *leave-one-CSV-out*. El rendimiento con partición aleatoria sigue siendo útil para comprobar si el modelo y la implementación son capaces de aprender el benchmark interno. Las particiones por día/archivo más difíciles, las comprobaciones de etiquetas barajadas, la validación *leave-one-CSV-out* y la inferencia offline sobre flujos de laboratorio prueban diferentes modos de fallo. Una puntuación alta en el escenario más sencillo no es suficiente para una afirmación de despliegue. En esta tesis, CICIDS2017 debe leerse como el benchmark interno principal, no como sustituto de la validación con tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial.

3.6 Aprendizaje supervisado y aprendizaje profundo para NIDS

La mayoría del trabajo de NIDS basado en flujos se formula naturalmente como aprendizaje supervisado: cada registro de tráfico se mapea a una etiqueta benigna, maliciosa o de familia de ataque (H. Liu et al. 2019; Ahmad et al. 2021). Los métodos supervisados clásicos incluyen árboles de decisión, Random

Forests, *SVMs*, *k-NN*, regresión logística, *Naive Bayes* y conjuntos de árboles con *gradient boosting*. Estos modelos difieren en sus suposiciones y en su coste operativo. La regresión logística es simple e interpretable, pero está limitada ante interacciones de características no lineales. *Naive Bayes* es rápido y útil como línea base débil, pero sus suposiciones de independencia son irrealistas para estadísticas de flujo correlacionadas. *k-NN* puede capturar vecindades locales, pero escala mal y es sensible al escalado de características. Las *SVMs* pueden ser potentes en conjuntos de datos más pequeños, pero se vuelven costosas con tablas de flujos muy grandes y requieren un escalado cuidadoso y la selección del kernel.

Los modelos basados en árboles siguen siendo especialmente importantes para el NIDS tabular. Los árboles de decisión y Random Forests manejan bien las interacciones no lineales, las escalas de características heterogéneas, las variables irrelevantes y las distribuciones de características mixtas (H. Liu et al. 2019; Apruzzese et al. 2022). Las familias de árboles de decisión con *gradient boosting* como *XGBoost*, *LightGBM* y *CatBoost* son a menudo potentes aprendices tabulares, pero este capítulo no necesita afirmaciones separadas sobre cada implementación a menos que la tesis los evalúe directamente. El punto metodológico importante es más sencillo: un clasificador de flujos basado en RL debe compararse con líneas base supervisadas tabulares competentes, no solo con métodos débiles u obsoletos.

Esto es directamente relevante para la tesis. Si cada observación es una fila de flujo etiquetada y la recompensa se deriva de la corrección de la clasificación, un clasificador supervisado es la línea base natural. Un agente QRDQN no puede evaluarse únicamente contra su propia curva de recompensa, su pérdida de entrenamiento o una línea base de clase mayoritaria. Debe compararse con métodos supervisados sólidos bajo el mismo esquema de características, protocolo de partición, preprocesamiento y métricas. El repositorio actual incluye un protocolo de línea base de Random Forest alineado con las particiones de QRDQN, mientras que la instantánea de resultados mantenida sigue marcando

las últimas métricas completas como pendientes. Hasta que existan esos artefactos, el capítulo debe describir la obligación de comparación con líneas base en lugar de afirmar que la comparación está completada.

El aprendizaje profundo amplía el espacio de diseño mediante MLPs, CNNs, RNNs, LSTMs, autoencoders, mecanismos de atención, modelos al estilo transformer y enfoques basados en grafos (Xiao et al. 2021; Asadullah et al. 2023; Sayeeduddin et al. 2022; Nguyen et al. 2022). Los MLPs tratan las características de flujo como vectores numéricos genéricos. Las CNNs se utilizan cuando los bytes de paquetes, las matrices de características o los patrones de características locales se codifican en una forma estructurada. Las RNNs y LSTMs se orientan a secuencias temporales de paquetes, flujos o eventos. Los autoencoders apoyan frecuentemente la detección de anomalías aprendiendo representaciones comprimidas del comportamiento benigno y señalando los errores de reconstrucción. Los modelos al estilo transformer y las redes neuronales en grafo son direcciones más recientes para representaciones de tráfico de secuencia y estructurales.

Estas arquitecturas pueden reportar puntuaciones de benchmark muy elevadas, especialmente en conjuntos de datos públicos, pero las revisiones bibliográficas también identifican debilidades recurrentes: preprocesamiento poco claro, particiones aleatorias favorables, análisis insuficiente del desbalanceo de clases, validación externa limitada y reproducibilidad insuficiente (H. Liu et al. 2019; Ring et al. 2019; Arp et al. 2020). Un modelo profundo puede sobreajustarse a los artefactos del benchmark con la misma facilidad que un modelo superficial. La familia de modelos por sí sola no resuelve las fugas, el desbalanceo de clases, la disponibilidad de características ni el cambio de dominio.

La estandarización de características es otra lección importante del aprendizaje supervisado. Diferentes conjuntos de datos y herramientas exponen conjuntos de características solapados pero no idénticos, lo que dificulta la comparación entre conjuntos de datos. Sarhan et al. propusieron un mapeo de características alineado con NetFlow para mejorar la generalizabilidad y

la explicabilidad en conjuntos de datos de NIDS (Sarhan et al. 2021). Esta tesis no adopta ese estándar exacto, pero adopta la misma motivación: un esquema canónico fijo hace auditable la interfaz del modelo y permite comparar las entradas de flujo de CICIDS2017 y de laboratorio a través de un único contrato de preprocesamiento. La máscara de presencia/ausencia extiende esta idea haciendo visibles al modelo y al evaluador las características ausentes.

3.7 Fundamentos del aprendizaje por refuerzo

El aprendizaje por refuerzo estudia cómo un agente selecciona acciones en un entorno para maximizar el retorno (Sutton et al. 2018). Sus conceptos centrales son los estados u observaciones, las acciones, las recompensas, las políticas, las funciones de valor y las transiciones. En un proceso de decisión de Markov, el estado actual resume la información necesaria para la toma de decisiones futura, y las acciones influyen tanto en la recompensa inmediata como en los estados posteriores (Sutton et al. 2018). Esta propiedad secuencial es lo que distingue el RL de la clasificación supervisada ordinaria.

Una política especifica cómo el agente elige acciones a partir de las observaciones. Una función de valor estima el retorno esperado desde un estado, mientras que una función de valor-acción estima el retorno esperado tras tomar una acción particular en un estado. Las recompensas definen el objetivo de la tarea, y el retorno agrega las recompensas futuras, generalmente con descuento. Estas definiciones importan porque cambiar la función de recompensa cambia aquello para lo que el agente está optimizado. En un contexto de seguridad, una recompensa que penaliza más los falsos negativos que los falsos positivos codifica una preferencia operativa concreta; no es un hecho neutral sobre todas las redes.

La distinción entre tareas episódicas y continuas también es relevante. Las tareas episódicas tienen límites y estados terminales, como un episodio de

juego o un escenario simulado de respuesta a incidentes. Las tareas continuas modelan la operación continua, como la monitorización continua de la red. Una formulación estática de conjunto de datos como entorno impone a menudo episodios artificiales: un episodio puede ser un número fijo de filas muestreadas, un recorrido por un conjunto de datos o un archivo. Eso es aceptable para la experimentación, pero es más débil que un entorno donde la acción del defensor modifica el estado siguiente.

El Q-learning clásico es un método de diferencia temporal model-free y off-policy que aprende valores de acción sin requerir un modelo de la dinámica del entorno (Watkins et al. 1992). Los métodos model-free aprenden directamente de la interacción o de transiciones registradas, mientras que los métodos basados en modelo aprenden o utilizan un modelo de transición para la planificación. Los métodos on-policy aprenden sobre la política que está generando el comportamiento actual; los métodos off-policy pueden aprender sobre una política objetivo mientras el comportamiento es generado de manera diferente. DQN y QRDQN heredan el enfoque basado en valores y off-policy. La repetición de experiencias, introducida anteriormente como mecanismo para reutilizar experiencias pasadas, también se volvió central en los sistemas de Q-learning profundo posteriores (L.-J. Lin 1992).

El RL tabular no es apropiado cuando las observaciones son vectores numéricos de alta dimensión. Una tabla de búsqueda sobre todas las posibles combinaciones de características de flujo es inviable, y pequeños cambios numéricos podrían crear efectivamente estados nuevos. La aproximación de funciones es, por tanto, necesaria. El DRL utiliza redes neuronales para aproximar políticas o funciones de valor sobre espacios de observación grandes. En esta tesis, la observación es un vector de 152 dimensiones: 76 valores canónicos de flujo más 76 valores de máscara de presencia/ausencia. Eso justifica un algoritmo de la familia DQN más que un algoritmo de RL tabular, pero no justifica por sí solo el RL frente al aprendizaje supervisado.

La ciberseguridad puede contener problemas de RL secuenciales genuinos.

Un defensor puede aislar un host, parchear un servicio, cambiar el enrutamiento, desplegar una contramedida o elegir dónde inspeccionar a continuación, y esas acciones pueden afectar a las observaciones posteriores y a las oportunidades del atacante. Sin embargo, un conjunto de datos estático de flujos etiquetados es un escenario más débil. Si se muestrea una fila de flujo, se clasifica y va seguida de otra fila que no es causalmente afectada por la acción anterior, el problema se parece más a una clasificación sensible al coste o a un proceso de decisión contextual que a una defensa cibernética autónoma completa (Levine et al. 2020; Fujimoto et al. 2019). La tesis debe mantener esta distinción de forma explícita.

3.8 Q-Learning profundo y aprendizaje por refuerzo distribucional

El Q-Learning Profundo sustituye una función de valor-acción tabular por una red neuronal, haciendo aplicable el RL basado en valores a observaciones de alta dimensión (Mnih et al. 2015). DQN popularizó dos mecanismos estabilizadores: la memoria de repetición y una red objetivo separada (Mnih et al. 2015). La memoria de repetición rompe las correlaciones a corto plazo muestreando transiciones pasadas para el entrenamiento y mejora la reutilización de datos. Una red objetivo estabiliza los objetivos de bootstrapping manteniendo el cálculo del objetivo separado de los pesos de la red online, que cambian rápidamente.

Variantes posteriores de DQN abordaron debilidades conocidas. Double DQN reduce el sesgo de sobreestimación desacoplando la selección de acciones de la evaluación del objetivo (Hasselt et al. 2016). Dueling DQN separa la estimación del valor del estado y la ventaja de la acción, lo que puede ayudar cuando muchas acciones tienen valores similares en un estado (Wang et al.

2016). Prioritized Experience Replay muestrea transiciones según la señal de aprendizaje en lugar de hacerlo de forma uniforme (Schaul et al. 2016). Rainbow combinó varias mejoras en un único agente de la familia DQN, incluyendo double Q-learning, redes dueling, repetición priorizada, retornos de múltiples pasos, aprendizaje distribucional y exploración con ruido (Hessel et al. 2018).

El aprendizaje por refuerzo distribucional cambia el objeto que se estima. En lugar de estimar únicamente el retorno esperado, aproxima la distribución de los retornos posibles (Bellemare et al. 2017). C51 representa la distribución del retorno sobre un soporte categórico fijo. QRDQN usa regresión cuantílica para aproximar los cuantiles del retorno, evitando la misma suposición de soporte fijo (Dabney et al. 2018). QRDQN no es, por tanto, un paradigma separado de DQN; es un miembro distribucional y basado en valores de la familia DQN.

La motivación de seguridad es intuitiva, pero debe formularse con cuidado. Los falsos positivos y los falsos negativos tienen consecuencias asimétricas, y los errores raros de alto coste pueden importar más de lo que sugiere una puntuación media. Una estimación distribucional del valor puede ser útil para estudiar la estructura del retorno bajo recompensas asimétricas. Puede revelar si una decisión tiene una distribución de retornos estrecha o amplia bajo el modelo aprendido. Sin embargo, el RL distribucional no proporciona automáticamente incertidumbre calibrada, seguridad operativa, comportamiento sensible al riesgo ni superioridad sobre el DQN escalar. Esas propiedades requerirían reglas de decisión explícitas, calibración, objetivos de riesgo en la cola o criterios de evaluación más allá de simplemente entrenar un agente distribucional (Bellemare et al. 2017; Dabney et al. 2018).

En esta tesis, QRDQN es una elección algorítmica exploratoria. Es adecuado para un espacio de acciones binario discreto y observaciones de flujo de dimensión moderadamente alta, y está disponible a través de la implementación de SB3-Contrib utilizada por el proyecto (Stable-Baselines3 Contributors 2023; Farama Foundation 2023). La elección algorítmica es coherente: dos acciones, observaciones numéricas, aprendizaje de valores off-policy, repetición y un obje-

tivo distribucional. La afirmación evidencial sigue siendo empírica. QRDQN no debe describirse como probadamente superior para NIDS a menos que las comparaciones con el mismo protocolo frente a DQN, Random Forest y otras líneas base apoyen esa afirmación.

3.9 Aprendizaje por refuerzo para la detección de intrusiones y la defensa cibernética

El RL y el DRL para la detección de intrusiones ya forman una bibliografía activa (Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024; Adawadkar et al. 2022). Los trabajos previos incluyen clasificadores de intrusiones al estilo DQN, enfoques actor-crítico, formulaciones de entorno adversarial, agentes de selección de características, sistemas orientados a SDN o IoT, y simulaciones de defensa cibernética autónoma. El campo es heterogéneo, por lo que los trabajos no deben tratarse como directamente comparables a menos que su formulación de tarea, conjuntos de datos, acciones, recompensas y protocolos de validación estén alineados.

Una rama está próxima a esta tesis: la detección de intrusiones con conjunto de datos como entorno. Lopez-Martin et al. reformularon la detección supervisada de intrusiones como un problema de RL profundo tratando los registros etiquetados como estados, las predicciones de clase como acciones y la recompensa como función de la corrección (Lopez-Martin, Carro et al. 2020). Su trabajo posterior extendió una formulación relacionada de RL offline a varios conjuntos de datos, incluyendo CICIDS2017 y UNSW-NB15 (Lopez-Martin, Sanchez-Esguevillas et al. 2021). Ren et al. utilizaron DRL para la selección de características y la clasificación en CSE-CIC-IDS2018 (Ren et al. 2022). Estos trabajos muestran que las formulaciones de RL para IDS tienen precedentes.

Otros trabajos de RL orientados a IDS exploran DQN, Double DQN, DQN

adversarial, actor-crítico, SAC, DDPG y diseños al estilo Rainbow (Caminero et al. 2019; Han et al. 2020; Hsu et al. 2023; Alavizadeh et al. 2021; Hossain et al. 2025). Los detalles varían ampliamente. Algunos trabajos utilizan clasificación binaria, otros etiquetas de ataque multiclase, selección de características, control de umbral o acciones específicas de SDN. Muchos siguen evaluándose sobre conjuntos de datos públicos estáticos. Eso los convierte en antecedentes relevantes para la tesis, pero también limita la comparabilidad directa. Un trabajo que usa NSL-KDD con particiones aleatorias y recompensas de corrección no es evidencia de que un defensor QRDQN offline generalice de CICIDS2017 a tráfico de laboratorio.

Una segunda rama estudia la defensa cibernética genuinamente secuencial. Los enfoques POMDP y de grafo de ataque modelan acciones defensivas bajo incertidumbre (Hu et al. 2020). CybORG proporciona un entorno al estilo gym para agentes cibernéticos autónomos (Standen et al. 2021). Los trabajos más amplios de defensa cibernética autónoma discuten agentes que seleccionan contramedidas en redes simuladas o emuladas (Palmer et al. 2023; Center for Security and Emerging Technology et al. 2023). Estos escenarios son conceptualmente diferentes de esta tesis porque las acciones del defensor pueden alterar el estado futuro. Representan una forma más fuerte de problema de RL que la clasificación estática por filas.

La tesis pertenece, por tanto, principalmente a la primera rama: detección offline a nivel de flujo formulada a través de una interfaz de RL. Puede referenciar la defensa cibernética autónoma como contexto más amplio y dirección futura, pero no debe implicar que el sistema actual realice aprendizaje multiagente, adversarial o de control de red en vivo. La formulación más precisa es que la tesis implementa un benchmark de NIDS con conjunto de datos como entorno y una interfaz de acción de seguridad binaria sensible al coste, no un defensor de cyber-range completo.

3.10 Clasificación como RL y decisiones de seguridad binarias

La formulación PERMIT/BLOCK pertenece a la discusión de clasificación como RL. En el entorno actual, cada observación de flujo se presenta al agente, la acción es binaria y la recompensa se calcula a partir de la acción y la etiqueta de verdad. PERMIT corresponde a aceptar el tráfico como benigno, mientras que BLOCK corresponde a identificarlo como ataque. Un falso negativo permite un flujo de ataque; un falso positivo bloquea tráfico benigno.

Este enfoque tiene un beneficio metodológico real: hace explícitas las decisiones de seguridad sensibles al coste. Los errores de IDS no son simétricos en la práctica. Omitir un ataque y bloquear tráfico benigno tienen significados operativos diferentes, y la compensación preferida depende del entorno defendido (Lee et al. 2002; Cárdenas et al. 2006). El diseño de la recompensa puede codificar esta asimetría directamente, lo que es útil para una tesis que estudia reglas de decisión defensivas en lugar de solo la predicción de etiquetas.

La limitación es igualmente importante. Si el conjunto de datos no reacciona a la acción, entonces PERMIT/BLOCK no es una acción de control activa. Es una etiqueta de decisión offline sobre una fila de flujo registrada o extraída. La fila siguiente no cambia porque el agente haya bloqueado o permitido la fila anterior. La formulación es, por tanto, más próxima a una clasificación con recompensa diseñada que a un firewall, un IPS, un controlador SDN o un agente de defensa cibernética autónoma. Esto no es un defecto si se declara con claridad. Solo se convierte en un defecto si el capítulo reclama en exceso el bloqueo en vivo, la preparación para el despliegue o el control secuencial completo.

La función de recompensa también necesita una interpretación cuidadosa. Una recompensa positiva para un verdadero positivo y una penalización mayor para un falso negativo puede expresar la idea de que los ataques omitidos

son costosos. Una penalización por falso positivo puede expresar el coste de interrumpir el tráfico benigno o de sobrecargar a los analistas. Una recompensa de cero o pequeño valor para los verdaderos negativos puede mantener el foco en la detección de ataques. Estas son decisiones de diseño, no economía universal de seguridad. Diferentes organizaciones elegirían distintos costes dependiendo de la continuidad del negocio, el modelo de amenaza, la capacidad del analista y la tolerancia a los ataques omitidos.

Dado que la formulación está próxima a la clasificación sensible al coste, las líneas base supervisadas son obligatorias. Un Random Forest sólido, e idealmente otras líneas base tabulares, ayudan a determinar si QRDQN añade valor empírico más allá de las características canónicas, el preprocesamiento y el diseño de la recompensa. Las mismas métricas deben calcularse tanto a partir de las salidas de RL como de las supervisadas. La recompensa de entrenamiento sola no es suficiente, porque una curva de recompensa puede mejorar mientras la precisión, el recall o el comportamiento de falsos positivos sigue siendo operativamente deficiente. La tesis debe interpretar cualquier ventaja de RL como específica del benchmark y el protocolo, a menos que se disponga de evidencia más amplia.

3.11 Riesgos metodológicos en NIDS basado en ML/DL/RL

La bibliografía de NIDS contiene advertencias repetidas contra la sobreinterpretación de la exactitud en el benchmark. Sommer y Paxson argumentaron que la detección de intrusiones basada en ML es difícil de validar en entornos realistas porque el tráfico benigno es diverso, los ataques son raros, las etiquetas son difíciles de obtener y las tasas base operativas difieren marcadamente de las condiciones del benchmark (Sommer et al. 2010). Arp et al. identifican riesgos

similares en el aprendizaje automático para seguridad informática, incluyen fuga de información, sesgo de muestreo, métricas inapropiadas y diseño experimental débil (Arp et al. 2020).

La fuga de información es más amplia que colocar accidentalmente la misma fila en entrenamiento y prueba. Incluye cualquier información relacionada con el objetivo que no estaría legítimamente disponible en el momento de la predicción (Kaufman et al. 2011). En NIDS, la fuga puede introducirse a través de identificadores de endpoint, marcas temporales absolutas, Flow ID, puertos específicos del escenario, preprocesamiento global ajustado antes de la partición, flujos duplicados o casi duplicados, y particiones de entrenamiento/prueba que comparten contexto de captura. Un modelo que aprende que una IP, un puerto, un día o un Flow ID concreto corresponde a un ataque puede obtener buenas puntuaciones sin haber aprendido comportamiento malicioso portable.

La fuga temporal y de escenario es especialmente importante. Los conjuntos de datos públicos de NIDS se organizan frecuentemente por día, archivo de captura, ventana de ataque o escenario con script. Una partición aleatoria por filas puede situar tráfico muy relacionado en ambos lados de la frontera entrenamiento/prueba. Incluso cuando las filas no son duplicados exactos, pueden compartir hosts, herramientas de ataque, regímenes de temporización o artefactos de extracción. Esto es especialmente importante para CICIDS2017 porque estudios posteriores reportan errores o artefactos en la extracción de flujos, el etiquetado y el uso de CSV resumidos (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022; Rosay et al. 2022; Dube 2024).

El preprocesamiento también puede introducir fugas. El escalado, la imputación, la selección de características, el recorte de valores atípicos o la reducción de dimensionalidad deben ajustarse sobre la partición de entrenamiento y aplicarse luego a los datos de validación o prueba. Si un escalador se ajusta sobre el conjunto de datos completo antes de particionar, la información distribucional del conjunto de prueba ha entrado en el entrenamiento. Esto puede parecer inofensivo en comparación con la fuga de etiquetas, pero en conjuntos de datos

de seguridad con estructura fuerte por día o escenario puede igualmente inflar la confianza. El uso en el repositorio de objetos `StandardScaler` ajustados sobre el entrenamiento es, por tanto, parte del diseño metodológico, no solo una comodidad de codificación.

El desbalanceo de clases y el problema de la tasa base complican la interpretación. En redes operativas, los ataques son típicamente raros en relación con el tráfico benigno. Un detector puede lograr una exactitud elevada mientras omite muchos ataques, o puede lograr un recall elevado mientras produce demasiadas alertas falsas como para investigarlas. El argumento de la tasa base de Axelsson sigue siendo relevante porque incluso tasas de falsos positivos bajas pueden dominar los flujos de alertas cuando la prevalencia real de los ataques es baja (Axelsson 2000). Por eso la exactitud no debe ser la medida principal para NIDS.

El RL introduce riesgos adicionales. El entrenamiento de RL profundo puede ser estocástico y sensible a las semillas aleatorias, los hiperparámetros, el modelado de la recompensa, la configuración del buffer de repetición, la exploración y los detalles del entorno. En un escenario estático de conjunto de datos como entorno, los resultados sólidos pueden reflejar las mismas ventajas de características y partición que benefician a los modelos supervisados. Por tanto, cualquier afirmación de que QRDQN es mejor que una línea base supervisada debe estar respaldada por una comparación con el mismo protocolo y debe reconocer la varianza entre ejecuciones cuando no se dispone de semillas repetidas. El capítulo debe tratar las ganancias de una única ejecución como preliminares a menos que estén respaldadas por experimentos repetidos o artefactos de validación sólidos.

3.12 Protocolos de evaluación, métricas y reproducibilidad

La evaluación debe ajustarse a la solidez de la afirmación. Las particiones aleatorias sobre el mismo conjunto de datos son aceptables para comprobaciones internas de aprendizaje, pero son evidencia débil para el despliegue. Una escalera de validación práctica para esta tesis comienza con particiones aleatorias estratificadas, avanza hacia particiones temporales o por CSV/día, la validación leave-one-scenario o leave-one-CSV-out, las pruebas entre conjuntos de datos con un esquema de características armonizado, y finalmente el tráfico capturado de forma independiente o similar al de producción. La tesis no alcanza este último peldaño; su inferencia sobre tráfico de laboratorio, generado por el operador en un entorno doméstico cerrado y no adversarial, se sitúa por debajo. Cada paso aumenta la separación distribucional entre datos de entrenamiento y prueba, y por tanto permite afirmaciones de generalización más sólidas.

El repositorio actual refleja esta escalera. El entrenamiento con partición aleatoria verifica que la implementación de QRDQN, el esquema de características, la función de recompensa y el bucle de entrenamiento pueden aprender el benchmark controlado. La comprobación B con etiquetas barajadas comprueba si el rendimiento colapsa hacia una línea base de clase mayoritaria cuando las etiquetas ya no contienen señal real. La comprobación C utiliza una partición por CSV/día más difícil. El script leave-one-CSV-out comprueba si retener un CSV original de CICIDS2017 modifica el comportamiento. La inferencia offline de Fase 2 sobre CSV de flujos de laboratorio prueba la cuestión separada de si un modelo entrenado sobre el benchmark puede procesar flujos capturados por el operador en un laboratorio doméstico cerrado.

Las métricas también deben interpretarse operativamente. La exactitud es útil pero insuficiente para NIDS desequilibrados. La precisión indica cuán fiables son las alertas positivas. El recall indica qué proporción del tráfico de

ataque es detectada. F1 proporciona un equilibrio compacto, pero oculta la compensación real entre falsos positivos y falsos negativos. La tasa de falsos positivos mapea al tráfico benigno bloqueado o alertado incorrectamente. La tasa de falsos negativos mapea al tráfico malicioso omitido por el detector (Axelsson 2000; Fawcett 2006; Saito et al. 2015). Las matrices de confusión, la precisión, el recall, F1, TP, FP, FN, TN, FPR y FNR por clase deben preferirse sobre un único número resumen (Milenkoski et al. 2015).

La evaluación sensible al coste es relevante porque el punto de operación correcto depende del contexto. La guía del NIST y la bibliografía de evaluación de IDS enfatizan ambas las compensaciones entre falsos positivos y falsos negativos (Scarfone et al. 2007; Cárdenas et al. 2006). En esta tesis, el diseño de recompensa asimétrico debe tratarse como una suposición de escenario y evaluarse empíricamente, no como un modelo de coste universal para todas las redes. Si la recompensa penaliza fuertemente los falsos negativos, la política resultante puede preferir bloquear más flujos. Ese comportamiento debe juzgarse mediante precisión, recall, tasa de falsos positivos y las suposiciones de coste específicas declaradas en el experimento.

La reproducibilidad es parte de la evidencia, no un detalle administrativo. Los resultados de ML en ciberseguridad deben preservar la versión del conjunto de datos, la variante exacta del conjunto de datos, el esquema de características, los campos excluidos, el orden del preprocesamiento, la lógica de partición, el estado del escalador, las semillas aleatorias, la configuración del modelo, la configuración de la recompensa, el identificador de ejecución, las métricas y las predicciones donde sea factible. Olszewski et al. muestran que la reproducibilidad sigue siendo un problema importante en la investigación de ML en seguridad (Olszewski et al. 2023). El uso en el repositorio de configuraciones guardadas, métricas, artefactos del modelo, salidas de validación y carpetas de ejecución es, por tanto, una fortaleza metodológica, pero los artefactos faltantes deben seguir claramente marcados como pendientes.

Esta disciplina en los informes también protege contra las afirmaciones

excesivas accidentales. Un resultado vinculado a la ejecución C03 de QRDQN con partición aleatoria completa es un resultado para esa ejecución, con esa partición, configuración de recompensa, ruta de preprocesamiento y estado de artefactos. No debe convertirse silenciosamente en una afirmación sobre todas las configuraciones de QRDQN, todas las variantes de CICIDS2017 o todo el tráfico de red. Cuanto más sólida sea la afirmación, más completo debe ser el rastro de artefactos.

3.13 Eficiencia de datos y evaluación a escala de entrenamiento

La eficiencia de datos pregunta cuánto tráfico etiquetado se necesita para alcanzar un rendimiento útil, y cómo cambia la curva de aprendizaje a medida que crece el conjunto de entrenamiento. Esta pregunta es importante para NIDS porque el tráfico de ataque etiquetado es costoso, las distribuciones de tráfico cambian y la abundancia de benchmarks públicos no implica abundancia de datos de despliegue (Ring et al. 2019; Apruzzese et al. 2022). También es importante para el RL porque los métodos de RL profundo pueden ser exigentes en muestras, especialmente cuando las señales de recompensa son escasas, ruidosas, retardadas o altamente diseñadas (Sutton et al. 2018; Hessel et al. 2018).

La bibliografía existente de RL-IDS con frecuencia entrena sobre conjuntos de datos públicos completos e informa de puntuaciones finales en el benchmark, mientras que las ablaciones controladas a escala de entrenamiento son menos comunes (Yang et al. 2026; Kheddar et al. 2025). El trabajo de NIDS supervisado con pocos ejemplos y aprendizaje incremental por clases aborda preguntas relacionadas desde una perspectiva diferente (Di Monda et al. 2024), pero la pregunta de la tesis es más estrecha: bajo un mismo esquema de características

y protocolo de evaluación, ¿cómo escala QRDQN con las filas de entrenamiento disponibles, y cómo se compara eso con las líneas base supervisadas?

La respuesta importa porque la elección del modelo y las necesidades de datos están conectadas. Si un Random Forest alcanza un rendimiento estable con muchas menos filas que QRDQN bajo la misma partición, eso es evidencia importante incluso si QRDQN lo alcanza posteriormente. Si QRDQN es más sensible a los datos reducidos o al desbalanceo de la clase de ataque, la tesis debe reportarlo como una limitación. Si QRDQN es competitivo con presupuestos menores, la evidencia debe seguir vinculada al benchmark y al protocolo de validación, no generalizada a todos los escenarios de NIDS.

La tesis posiciona los experimentos a escala de entrenamiento como evidencia metodológica. Presupuestos como 100k, 250k, 500k, 1M y 2M muestras pueden comprobar si la formulación de RL requiere sustancialmente más datos que una línea base supervisada para alcanzar un rendimiento comparable. Tales experimentos no deben enmarcarse como prueba de aprendizaje con pocos ejemplos ni de preparación para el despliegue. Son evidencia interna sobre los requisitos de datos de esta formulación particular. La salida útil es una comparación de curvas de aprendizaje con preprocesamiento consistente, semillas aleatorias donde sea factible, métricas compartidas y rutas de artefactos explícitas.

3.14 Brecha de investigación y posicionamiento de esta tesis

La bibliografía respalda una brecha de investigación estrecha pero significativa. El NIDS y el ML basado en flujos son áreas maduras; conjuntos de datos públicos como CICIDS2017 son útiles pero frágiles; las líneas base tabulares supervisadas son sólidas; el RL y el DRL para IDS ya existen; y la defensa

cibernética autónoma es un campo más amplio y más secuencial que la implementación actual (Ring et al. 2019; H. Liu et al. 2019; Yang et al. 2026; Kheddar et al. 2025). La brecha no es la ausencia de RL para la detección de intrusiones.

La contribución de esta tesis es un estudio experimental reproducible de una formulación offline de decisión de seguridad binaria a nivel de flujo. El sistema mapea CICIDS2017 y entradas posteriores de flujos de laboratorio en un esquema canónico fijo de 76 características, aumenta las observaciones con una máscara de presencia/ausencia para producir entradas de 152 dimensiones, expone las filas a través de un entorno compatible con Gymnasium, y entrena un agente QRDQN para elegir entre PERMIT y BLOCK. Esto hace explícita la interfaz de decisión sensible al coste mientras preserva una obligación de comparación clara contra las líneas base supervisadas.

La tesis también está posicionada metodológicamente. Trata a CICIDS2017 como el benchmark interno principal, no como prueba de preparación para la producción. Separa los resultados con partición aleatoria de la validación más estricta. Utiliza preprocesamiento con control de fugas y comprobaciones de validación para reducir el aprendizaje evidente de artefactos. Trata el tráfico de laboratorio capturado externamente como el siguiente paso evidencial preferido, reconociendo al mismo tiempo que la inferencia actual de Fase 2 es offline y no realiza bloqueo activo.

La brecha final es, por tanto, una brecha de integración: un benchmark QRDQN reproducible, con control de fugas y sensible al coste para el NIDS a nivel de flujo, evaluado contra líneas base supervisadas e interpretado a través de una escalera de validación. La tesis puede contribuir haciendo explícito el contrato de características, guardando artefactos de ejecución, documentando las suposiciones de la recompensa, comparando contra Random Forest bajo particiones emparejadas, y mostrando cómo el rendimiento interno de CICIDS2017 cambia bajo una validación más estricta y restricciones de escala de entrenamiento.

La afirmación final defendible no es que QRDQN sea universalmente mejor que los modelos de NIDS supervisados, ni que el proyecto implemente defensa cibernética autónoma. La afirmación defendible es que la tesis evalúa un defensor a nivel de flujo basado en QRDQN, sensible al coste, bajo un benchmark reproducible y un pipeline de validación, y utiliza líneas base supervisadas, comprobaciones con control de fugas, análisis de escala de entrenamiento e inferencia sobre tráfico de laboratorio para determinar hasta qué punto puede confiarse en esa formulación. Si experimentos posteriores muestran un rendimiento débil entre archivos o con tráfico de laboratorio, eso no invalida la tesis. Agudiza su contribución al demostrar los límites del RL entrenado sobre benchmark bajo un cambio de distribución más realista.

Diseño del sistema

4.1 Visión general metodológica

Este capítulo describe el diseño metodológico empleado para construir y evaluar el defensor de ciberseguridad basado en aprendizaje por refuerzo (RL) propuesto. El objetivo no es presentar un sistema de prevención de intrusiones (IPS) listo para producción, sino definir un marco experimental controlado, reproducible y offline para decisiones de seguridad binarias sobre registros de flujos de red. Cada flujo se trata como un único punto de decisión: el defensor observa una representación numérica del flujo y decide si permitirlo (**PERMIT**) como tráfico benigno o bloquearlo (**BLOCK**) como tráfico malicioso.

La metodología es deliberadamente conservadora. Los benchmarks públicos de detección de intrusiones pueden producir resultados inflados cuando el preprocesamiento, la construcción de las particiones, los controles de fuga de información y las comparaciones con líneas base son débiles (Arp et al. 2020; Engelen et al. 2021; Lanvin et al. 2023). Por ello, el pipeline se ha diseñado en torno a contratos de datos explícitos, artefactos de ejecución reproducibles, normalización ajustada únicamente sobre datos de entrenamiento, y una validación progresivamente más estricta. La misma representación canónica se utiliza

tanto para el benchmark interno de CICIDS2017 como para la inferencia offline sobre tráfico de laboratorio en la Fase 2, de modo que la pregunta experimental no queda ligada a una convención de nomenclatura de CSV concreta ni a un extractor de características particular.

A grandes rasgos, la metodología comprende seis etapas encadenadas:

1. ingerir los datos CSV de CICIDS2017 a nivel de flujo y mapear las etiquetas a una tarea binaria de defensa;
2. eliminar los identificadores susceptibles de provocar fuga de información y normalizar los valores numéricos malformados;
3. mapear las columnas específicas de cada extractor a un esquema canónico de características fijo de 76 características;
4. añadir una máscara de presencia/ausencia de 76 dimensiones, produciendo observaciones de 152 dimensiones;
5. entrenar y evaluar un defensor QRDQN compatible con Gymnasium bajo protocolos de partición controlados;
6. comprobar la robustez mediante comprobaciones de validación, líneas base supervisadas e inferencia offline en la Fase 2.

Este diseño mantiene la tesis dentro de un marco experimental offline. Las salidas PERMIT y BLOCK son predicciones del modelo sobre registros de flujo almacenados. No son órdenes de cortafuegos y no modifican ninguna ruta de red activa.

La Figura 4.1 resume esta arquitectura en dos recorridos separados: entrenamiento y evaluación sobre CICIDS2017, e inferencia offline posterior sobre tráfico de laboratorio. La separación visual refuerza una idea metodológica central: la Fase 2 reutiliza el esquema canónico, el escalador y el modelo entrenado, pero no reentrena ni reajusta el pipeline sobre la captura de laboratorio.

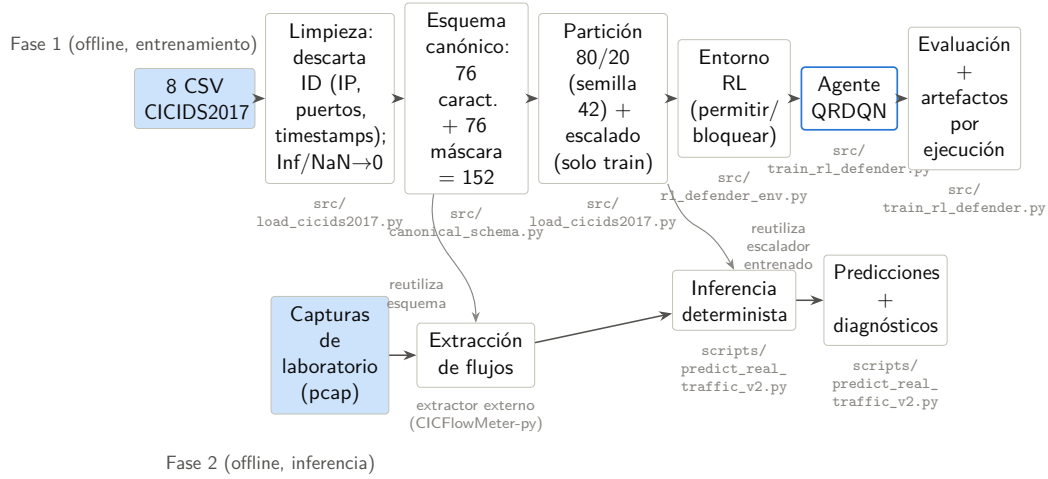


Figura 4.1: Pipeline general del sistema: entrenamiento offline sobre CICIDS2017 e inferencia offline en la Fase 2. Fuente: elaboración propia a partir de `src/load_cicids2017.py`, `src/canonical_schema.py`, `src/train_rl_defender.py` y `scripts/predict_real_traffic_v2.py`.

4.2 Fuente de datos y representación de la entrada

La tesis utiliza una representación tabular a nivel de flujo. Esta representación es un compromiso deliberado entre relevancia operativa, manejabilidad y privacidad. Captura el comportamiento estadístico a lo largo de secuencias de paquetes evitando la inspección de la carga útil.

4.2.1 CSVs de flujos de CICIDS2017

La fuente de datos principal es la publicación de CSVs de flujos de CICIDS2017. Cada fila de un CSV corresponde a un flujo de red bidireccional resumido mediante estadísticas numéricas como duración, conteos de paquetes, totales de bytes, tiempos de llegada entre paquetes, estadísticas de longitud de paquete y conteos de flags TCP. Las representaciones basadas en flujos se utilizan ampliamente en la investigación sobre NIDS porque comprimen el comportamiento de la red en filas numéricas de anchura fija que pueden

procesarse mediante métodos estándar de aprendizaje automático (Sperotto et al. 2010; Umer et al. 2017).

El pipeline de carga local espera los archivos CSV oficiales de CICIDS2017 y admite presupuestos de filas tanto completos como limitados. Los presupuestos limitados permiten pruebas rápidas de humo e iteración ágil; los presupuestos completos son los utilizados en las ejecuciones finales del benchmark. El cargador lee los CSVs grandes en fragmentos, estandariza los nombres de columna eliminando espacios en blanco inconsistentes, identifica la columna de etiqueta de forma robusta y aplica después la limpieza de características antes de cualquier escalado dependiente de la partición.

4.2.2 Mapeo de etiquetas binarias

CICIDS2017 incluye varias etiquetas de ataque. La tesis las colapsa en una decisión binaria del defensor:

- etiqueta 0: BENIGN, mapeada a la acción deseada PERMIT;
- etiqueta 1: cualquier clase de ataque, mapeada a la acción deseada BLOCK.

Este mapeo binario se corresponde con el modelo de decisión de primera línea estudiado aquí. Además, hace que el espacio de acciones de RL sea sencillo y auditable. La limitación es que la información sobre la familia de ataque no es el objetivo de aprendizaje principal en la ruta de RL actual. Cualquier análisis de errores por familia debe por tanto estar respaldado por etiquetas de familia preservadas o artefactos independientes; no puede inferirse a partir de las etiquetas binarias por sí solas.

4.2.3 Representación tabular a nivel de flujo

El sustrato de aprendizaje es una tabla a nivel de flujo en lugar de paquetes en bruto o bytes de carga útil. Esto reduce la dimensionalidad y hace factible

el entrenamiento offline a gran escala. También evita algunas complicaciones legales y técnicas de la inspección profunda de paquetes, especialmente cuando las cargas útiles están cifradas (Sperotto et al. 2010; Umer et al. 2017). Sin embargo, la abstracción de flujo pierde el contenido, el orden preciso de los paquetes y parte del contexto de largo alcance entre múltiples flujos. La metodología estudia por tanto un defensor basado en flujos, no un sistema completo de análisis forense de red.

4.3 Limpieza de datos y preprocesamiento

El preprocesamiento se trata como parte del método experimental, no como un detalle de implementación. En los benchmarks de NIDS, pequeñas decisiones de preprocesamiento pueden cambiar el significado de la tarea al filtrar etiquetas, suprimir flujos malformados o hacer que los datos de prueba influyan en el entrenamiento.

4.3.1 Normalización de columnas y coerción numérica

Las columnas CSV en bruto pueden contener espacios iniciales, capitalización inconsistente y tipos de datos mixtos. El cargador primero normaliza los nombres de columna y después convierte las columnas de características a valores numéricos. La matriz de características final se representa como `float32`, mientras que las etiquetas se representan como `int64`. Este contrato de tipos mantiene estable la interfaz del entorno, el escalador y el modelo en las etapas posteriores.

No se permite que las filas introduzcan cadenas de texto arbitrarias en el modelo. Las columnas de características no numéricas que sobreviven a la eliminación de identificadores quedan excluidas. Esto evita la introducción accidental de cadenas categóricas cuya codificación sería no controlada o específica

de un conjunto de datos.

4.3.2 Valores inválidos, valores ausentes e imputación

La extracción de flujos de red puede producir valores numéricos inválidos. Las características de tasa, por ejemplo, pueden volverse infinitas o indefinidas cuando se calculan sobre una duración nula o casi nula. El pipeline sustituye los infinitos y los valores numéricos ausentes por cero antes del mapeo canónico. Esta elección es pragmática: muchos contadores de flujo, tasas y estadísticas agregadas pueden utilizar de forma segura el cero como marcador de posición neutro cuando se acompaña de una señal explícita de presencia/ausencia.

La imputación por cero sola sería insegura porque el modelo no podría distinguir un cero medido real de un marcador sintético. El esquema canónico empareja por tanto la imputación con una máscara de presencia/ausencia. La máscara hace visible la disponibilidad de características tanto para el modelo como para los diagnósticos posteriores.

4.3.3 Exclusión de campos susceptibles de fuga

El pipeline de preprocesamiento aplica una política anti-fuga fundamentada en los riesgos de evaluación conocidos en seguridad con ML (Arp et al. 2020; Kaufman et al. 2011). Se excluyen las direcciones IP de origen y destino, los Flow IDs, las marcas temporales absolutas y los identificadores de endpoints. Los campos de puerto directo también se excluyen porque pueden actuar como indicadores del escenario en conjuntos de datos controlados por scripts. Por ejemplo, un día de ataque puede concentrar tráfico malicioso en servicios concretos, lo que permitiría al modelo memorizar un script de captura en lugar de aprender estadísticas de comportamiento de flujo.

Esta exclusión se aplica antes del mapeo canónico. El modelo recibe por tanto únicamente características estadísticas de flujo y los valores de máscara

correspondientes. Si en el futuro se introducen nuevos conjuntos de datos o extractores, sus adaptadores deben preservar la misma política: sin identificadores, sin campos de tiempo absoluto y sin puertos utilizados directamente como atajos de etiqueta.

4.4 Esquema canónico de características

El esquema canónico es el contrato de datos central de la tesis. Separa la interfaz del modelo de los nombres de columna, el orden y la disponibilidad de características de un extractor concreto.

4.4.1 Características canónicas de flujo

El esquema define 76 características numéricas de flujo ordenadas. Incluyen duración, conteos de paquetes hacia adelante y hacia atrás, totales de bytes, estadísticas de longitud de paquete, estadísticas de tiempo entre llegadas de flujo y por dirección, tasas de paquetes y bytes, conteos de flags TCP, longitudes de cabecera, tamaños de ventana, estadísticas de subflujos y características de temporización activa/inactiva. Estas características se han elegido porque son estadísticas de flujo y no identificadores, y porque son ampliamente compatibles con la extracción al estilo CICFlowMeter.

Para CICIDS2017, un adaptador mapea las columnas originales del CSV a estos nombres canónicos. Para la Fase 2, el script de inferencia robusta mapea las columnas del extractor de flujos de laboratorio a los mismos nombres canónicos. El modelo se entrena y evalúa por tanto frente a un contrato de características fijo. Este diseño sigue la motivación metodológica más amplia de la estandarización de características en NIDS: la comparación entre dominios resulta más auditable cuando las definiciones de características son explícitas y estables (Sarhan et al. [2021](#)).

La Tabla 4.1 enumera los grupos lógicos dentro del esquema de 76 características y el número de características aportadas por cada grupo. La agrupación es informativa; el modelo recibe los 76 valores como un vector plano sin señales de agrupación estructural.

Grupo de características	Cantidad	Estadísticas representativas
Duración del flujo	1	Duración total del flujo bidireccional
Estadísticas de paquetes hacia adelante	12	Conteo de paquetes, bytes totales, media/desv. típica/mínimo/máximo de longitudes de paquete
Estadísticas de paquetes hacia atrás	12	Las mismas estadísticas para la dirección inversa
Agregados bidireccionales	8	Conteos combinados, bytes totales, promedios de tasa de ráfaga
Estadísticas de tiempo entre llegadas	10	Media, desv. típica, mínimo y máximo del tiempo entre llegadas del flujo, hacia adelante y hacia atrás
Conteos de flags TCP	8	Conteos de FIN, SYN, RST, PSH, ACK, URG, CWE, ECE
Agregados de longitud de cabecera	4	Totales de bytes de cabecera hacia adelante y hacia atrás
Estadísticas de tamaño de ventana	3	Tamaño de ventana inicial y medio por dirección
Características de tasa de flujo	6	Bytes/s, paquetes/s por dirección y combinados
Estadísticas de subflujos	6	Promedios de paquetes y bytes de subflujo por dirección
Temporización activa/inactiva	6	Media, desv. típica y máximo de las duraciones de los periodos activos e inactivos

Tabla 4.1: Agrupación lógica de las 76 características canónicas de flujo. Todos los grupos se fusionan en un único vector plano antes de la entrada al modelo.

4.4.2 Máscara de presencia/ausencia

Diferentes extractores pueden no proporcionar todas las características canónicas, e incluso una columna fuente presente puede contener valores inválidos.

El pipeline produce por tanto una máscara de presencia/ausencia de 76 dimensiones. Para cada característica canónica x_i , el valor de máscara correspondiente m_i es:

- $m_i = 1$ cuando la característica fuente está presente y es válida;
- $m_i = 0$ cuando la característica está ausente o ha tenido que ser imputada.

La máscara desempeña dos funciones metodológicas. En primer lugar, evita la imputación silenciosa: el modelo puede condicionar su decisión a si un valor fue observado o rellenado. En segundo lugar, facilita los diagnósticos de la Fase 2, porque un extractor de laboratorio que carece de muchas características al estilo CICIDS2017 puede detectarse mediante un patrón de máscara diferente en lugar de quedar oculto dentro de un vector numérico denso.

Conviene precisar la semántica exacta de la máscara en la implementación actual. La máscara se calcula *después* de la limpieza que sustituye los valores no finitos por cero (`src/load_cicids2017.py`: $\pm\infty \rightarrow \text{NaN} \rightarrow 0$), de modo que $m_i = 1$ siempre que la característica canónica tiene una columna fuente mapeada. Como el mapeo $\text{CICIDS2017} \rightarrow \text{canónico}$ cubre las 76 características, sobre los datos nativos de CICIDS2017 la máscara es **constante e igual a 1** en todas las filas: codifica la *presencia de la columna fuente*, no la ausencia de valores fila a fila. La máscara solo se vuelve informativa en la inferencia entre dominios o en la Fase 2 de laboratorio, donde algunas columnas fuente no están disponibles y su valor de máscara permanece en 0; un mapeo histórico como NSL-KDD, por ejemplo, solo cubre 3 de las 76 características canónicas. Así, las 152 dimensiones de la observación se componen de 76 valores de características más 76 indicadores de presencia.

4.4.3 Vector de observación final

La observación final es:

$$o = [x_1, x_2, \dots, x_{76}, m_1, m_2, \dots, m_{76}]$$

Esto produce un vector `float32` de 152 dimensiones. La primera mitad contiene los valores de las características, y la segunda mitad contiene la máscara. El tamaño de la observación es por tanto invariante entre CICIDS2017, las particiones de validación y la inferencia en la Fase 2. Esta invariancia es importante porque la red de política del QRDQN espera una dimensión de entrada fija.

El Algoritmo 1 resume de forma operativa el proceso descrito en los párrafos anteriores: para cada fila del DataFrame de entrada construye, característica a característica, el valor x_i y el indicador de presencia m_i a partir del mapeo columna-fuente $\rightarrow$ nombre canónico, y concatena ambos vectores en la observación final de 152 dimensiones.

La Figura 4.2 muestra la misma construcción desde el punto de vista de la entrada al modelo: columnas del extractor, mapeo al contrato canónico, 76 valores de características, 76 indicadores de presencia y vector final de 152 dimensiones.

4.5 Formulación del aprendizaje por refuerzo

El defensor se implementa como un entorno compatible con Gymnasium (Farama Foundation 2023). Esto permite que el código estándar de RL basado en valores interactúe con el conjunto de datos de flujos a través de la interfaz habitual `reset()` y `step()`.

Algoritmo 1 Mapeo al esquema canónico y construcción de la observación. Fuente: elaboración propia a partir de `src/canonical_schema.py`.

Require: DataFrame D con las columnas producidas por el extractor; mapeo μ : columna_fuente $\rightarrow$ nombre_canónico

Ensure: Matriz de observaciones O , con una fila $o \in \mathbb{R}^{152}$ (`float32`) por cada fila de D

```

1: for cada fila  $d$  de  $D$  do
2:    $x \leftarrow$  vector nulo de dimensión 76
3:    $m \leftarrow$  vector nulo de dimensión 76
4:   for  $i = 1$  hasta 76 do
5:      $c_i \leftarrow$  nombre canónico  $i$ -ésimo de FEATURES_CANON
6:     if existe columna_fuente tal que  $\mu(\text{columna\_fuente}) = c_i$  y columna_fuente  $\in$  columnas( $D$ ) then
7:        $v \leftarrow$  CoerciónNumérica( $d[\text{columna\_fuente}]$ )  $\triangleright$  no numérico  $\rightarrow$  NaN
8:       if  $v$  no es finito (NaN o  $\pm\infty$ ) then
9:          $v \leftarrow 0$ 
10:      end if
11:       $x_i \leftarrow v$ 
12:       $m_i \leftarrow 1$   $\triangleright$  columna fuente mapeada y presente
13:    else
14:       $x_i \leftarrow 0$   $\triangleright$  imputación por ausencia de columna fuente
15:       $m_i \leftarrow 0$ 
16:    end if
17:  end for
18:   $o \leftarrow \text{concat}(x_1, \dots, x_{76}, m_1, \dots, m_{76})$   $\triangleright$  vector float32 de 152 dimensiones
19:  añadir  $o$  a  $O$ 
20: end for
21: return  $O$ 

```

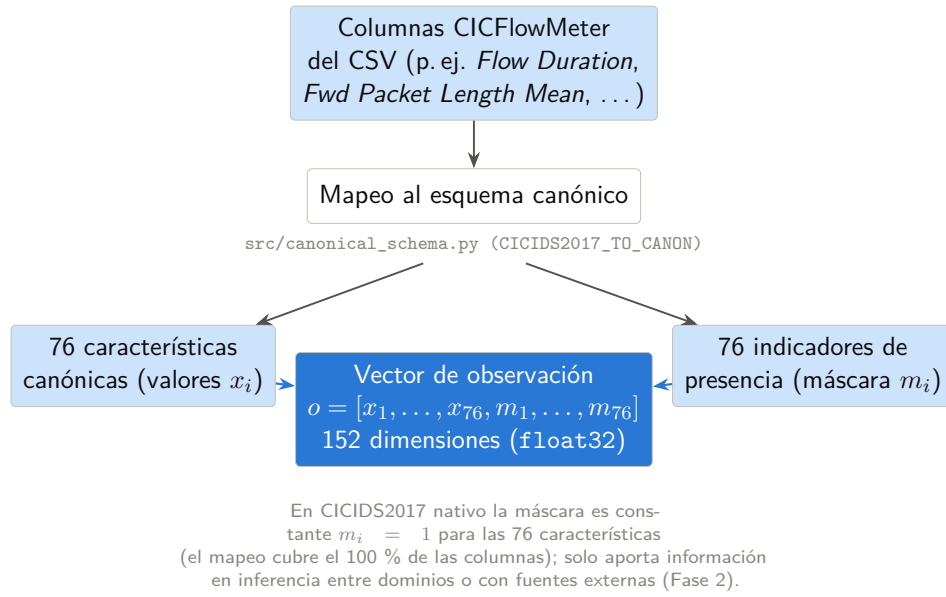


Figura 4.2: Construcción del vector de observación canónico de 152 dimensiones. Fuente: elaboración propia a partir de `src/canonical_schema.py`.

4.5.1 Espacio de observación

El espacio de observación es un espacio vectorial continuo de dimensión 152. Cada observación representa un vector de flujo canónico escalado junto con su máscara de presencia/ausencia. Durante el entrenamiento, las observaciones se extraen de la partición de entrenamiento. Durante la evaluación determinista, el entorno itera a través de la partición de prueba sin barajado, lo que permite comparar las predicciones directamente con las etiquetas verdaderas.

4.5.2 Espacio de acciones

El espacio de acciones es discreto con dos acciones:

- **Acción 0 (PERMIT):** clasificar el flujo como benigno y permitirlo en el modelo de decisión experimental;
- **Acción 1 (BLOCK):** clasificar el flujo como malicioso y bloquearlo o generar una alerta en el modelo de decisión experimental.

El valor de la acción está alineado intencionalmente con el valor de la etiqueta binaria: benigno es 0, ataque es 1. Esto simplifica la evaluación directa porque una acción predicha puede compararse con la etiqueta objetivo sin necesidad de una capa de decodificación adicional.

4.5.3 Función de recompensa

La función de recompensa codifica costes de seguridad asimétricos (Lee et al. 2002; Cárdenas et al. 2006). Un falso negativo significa que se permite un ataque. Un falso positivo significa que se bloquea tráfico benigno. En muchos escenarios defensivos, los ataques no detectados son más graves que los bloqueos innecesarios, aunque la relación exacta depende del contexto operativo.

La implementación actual utiliza la siguiente configuración de recompensa por defecto:

$$TP = 1.5, \quad FP = -2.0, \quad FN = -5.0, \quad TN/\text{omisión} = 0.0.$$

La Tabla 4.2 organiza estos valores como una matriz de decisión para los cuatro resultados posibles.

Decisión del agente	Etiqueta real: Ataque	Etiqueta real: Benigno
BLOCK (Acción 1)	+1.5 (Verdadero Positivo)	-2.0 (Falso Positivo)
PERMIT (Acción 0)	-5.0 (Falso Negativo)	0.0 (Verdadero Negativo)

Tabla 4.2: Matriz de recompensa por defecto para el entorno del defensor binario. Los valores son supuestos experimentales registrados con cada ejecución.

La Figura 4.3 representa el bucle agente–entorno usado en esta formulación. Aunque se utiliza una interfaz de RL estándar, la recompensa depende únicamente de la acción actual y de la etiqueta real del flujo evaluado; con $\gamma = 0$, el objetivo aprendido se reduce a la recompensa inmediata.

La penalización por falso negativo domina la señal de recompensa. Un agente

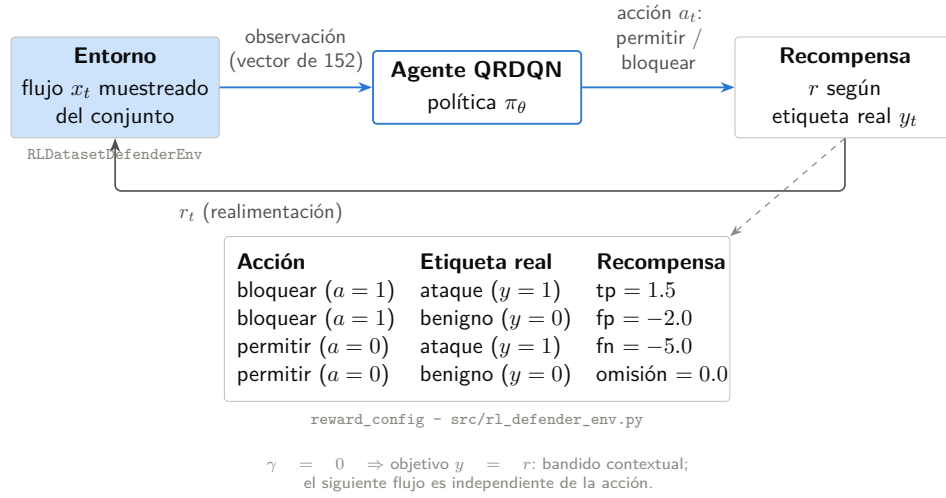


Figura 4.3: Interacción agente-entorno y matriz de recompensa de la decisión PERMIT/BLOCK. Fuente: elaboración propia a partir de `src/rl_defender_env.py` y de la configuración MAIN.

que emite PERMIT en cada observación acumula un retorno muy negativo en tráfico con alta proporción de ataques, mientras que un agente que emite BLOCK en cada observación acumula penalizaciones por falsos positivos sin obtener crédito por detección de ataques. La estructura de recompensa está por tanto diseñada para alejar al agente de políticas constantes degeneradas y conducirlo hacia un comportamiento discriminativo. La recompensa cero para el verdadero negativo es deliberada: recompensar explícitamente cada paso benigno correcto sesgaría al agente hacia maximizar la tasa de permiso en lugar de la calidad de detección. Estos valores son supuestos experimentales, no costes empresariales medidos. Se registran con cada ejecución para que las ejecuciones históricas puedan distinguirse de los valores por defecto actuales.

Los valores de la Tabla 4.2 admiten además una lectura como compromiso multiobjetivo implícito entre dos objetivos operativos, seguridad (minimizar ataques admitidos) e impacto (minimizar el bloqueo de tráfico legítimo), que se resume en la Tabla 4.3.

Resultado	Recompensa	Objetivo operativo	Lectura
FN (ataque permitido)	-5.0	Seguridad	Coste dominante: un ataque queda admitido
FP (benigno bloqueado)	-2.0	Impacto	Penaliza el bloqueo de tráfico legítimo
TP (ataque bloqueado)	+1.5	Seguridad	Crédito positivo por detección correcta de un ataque
TN/omisión (benigno permitido)	0.0	Neutralidad deliberada	Evita sesgar al agente hacia maximizar la tasa de permiso

Tabla 4.3: Lectura multiobjetivo de la función de recompensa del entorno del defensor binario: cada resultado se asocia al objetivo operativo de seguridad o de impacto al que responde. Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

4.5.4 Estructura del episodio

Un episodio es una secuencia de decisiones sobre flujos. Al reiniciar, el entorno inicializa un índice sobre la partición seleccionada. Cuando el barajado está activado, el orden de entrenamiento se aleatoriza. En cada paso, el agente recibe una observación, selecciona `PERMIT` o `BLOCK`, recibe una recompensa calculada a partir de la etiqueta real y la acción, y avanza al flujo siguiente. Un episodio termina cuando se agota el conjunto de datos o se alcanza un número máximo de pasos configurado.

Este diseño proporciona a los algoritmos estándar de RL un flujo de transiciones manteniendo el entrenamiento offline y seguro. Ninguna acción afecta al conjunto de datos fuente, al flujo siguiente ni al entorno de red.

4.5.5 Estado del entorno y protocolo de transición

En cada llamada a `reset()`, el entorno selecciona la partición asignada al modo actual (entrenamiento o evaluación), baraja opcionalmente el índice si el

modo de barajado está activo, reinicia el contador de pasos interno y devuelve la primera observación junto con un diccionario de metadatos. En cada llamada a `step()`, el entorno registra la acción del agente, recupera la etiqueta de verdad del índice de flujo actual, calcula la recompensa escalar a partir de la matriz de recompensa, avanza el índice y devuelve la siguiente observación, la recompensa, un indicador de terminación, un indicador de truncación y el diccionario de metadatos.

La condición de terminación es el agotamiento del conjunto de datos o un número máximo de pasos configurable. En el modo de entrenamiento, el orden barajado garantiza que las transiciones del minibatch muestreen una distribución amplia de tipos de flujo durante un único episodio. En el modo de evaluación, el entorno es determinista: la misma partición recorrida en el mismo orden produce la misma secuencia de observaciones, recompensas y etiquetas. Este determinismo es esencial para la evaluación reproducible porque implica que las diferencias en métricas entre dos modelos guardados reflejan únicamente el comportamiento del modelo y no el ruido de muestreo en el bucle de evaluación.

La separación entre los modos de entrenamiento y evaluación se aplica a nivel del entorno y no se deja al llamante. Pasar el indicador de modo correcto en el momento de la construcción establece tanto el conjunto de índices como el comportamiento de barajado, lo que evita un error frecuente en el que la evaluación hereda accidentalmente el orden barajado de un entorno de entrenamiento.

El Algoritmo 2 muestra el pseudocódigo del paso de decisión `step(a)` tal y como está implementado, que aplica la matriz de recompensas de la Tabla 4.2 a la etiqueta real del índice actual, hace avanzar el puntero de muestras y evalúa de forma independiente las condiciones de terminación (agotamiento del conjunto de índices) y truncación (límite de pasos por episodio).

Algoritmo 2 Paso de decisión del entorno (`env.step`). Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

Require: Índice actual i , contador de pasos t y vector de índices $\mathcal{I}$, inicializados en RESET (con $\mathcal{I}$ barajado si el indicador *shuffle* está activo, según el modo fijado en la construcción del entorno)

```

1: function STEP( $a$ )
2:   if  $a \notin \{0, 1\}$  then
3:     return error (acción inválida)
4:   end if
5:    $idx \leftarrow \mathcal{I}[i]$ 
6:    $y \leftarrow Y[idx]$  ▷ etiqueta real de la muestra actual
7:   if  $y = \text{ataque}$  then
8:     if  $a = 1$  (BLOCK) then
9:        $r \leftarrow tp = 1.5$  ▷ verdadero positivo
10:    else
11:       $r \leftarrow fn = -5.0$  ▷ falso negativo
12:    end if
13:  else ▷  $y = \text{benigno}$ 
14:    if  $a = 0$  (PERMIT) then
15:       $r \leftarrow \text{omission} = 0.0$  ▷ verdadero negativo
16:    else
17:       $r \leftarrow fp = -2.0$  ▷ falso positivo
18:    end if
19:  end if
20:   $i \leftarrow i + 1$ ;  $t \leftarrow t + 1$  ▷ avance del índice y del contador de pasos
21:   $terminated \leftarrow (i \geq N)$  ▷  $N$ : tamaño del conjunto de muestras
22:   $truncated \leftarrow (t \geq T_{\text{máx}})$  ▷  $T_{\text{máx}}$ : máximo de pasos por episodio
23:  if  $terminated \vee truncated$  then
24:     $s' \leftarrow X[idx]$  ▷ se repite la observación de la muestra actual
25:  else
26:     $s' \leftarrow X[\mathcal{I}[i]]$  ▷ observación de la siguiente muestra
27:  end if
28:   $info \leftarrow \{sample\_index : idx, true\_label : y\}$ 
29:  return ( $s', r, terminated, truncated, info$ )
30: end function

```

4.5.6 Limitaciones de la formulación conjunto-de-datos-como-entorno

Dado que el entorno es un conjunto de datos estático, las acciones del agente no afectan a los estados futuros. Esto no es un problema completo de defensa cibernética autónoma en el que bloquear un flujo podría cambiar el comportamiento posterior del atacante, el estado del host, el enrutamiento o el contexto de alertas. La formulación se asemeja más a un problema de decisión contextual sensible al coste que a un proceso de decisión de Markov rico (Sutton et al. 2018; Levine et al. 2020).

Esta limitación es explícita en la metodología. El RL se utiliza aquí para estudiar el aprendizaje de decisiones basadas en valor y sensibles al coste sobre registros de flujo, y para establecer un pipeline offline seguro. Las afirmaciones sobre respuesta adversarial en múltiples pasos, adaptación en línea o defensa cibernética activa quedan fuera del alcance implementado.

4.6 Agente QRDQN

El algoritmo de RL principal es Quantile Regression Deep Q-Network (QRDQN) (Dabney et al. 2018). QRDQN pertenece a la familia del RL distribucional, que modela una distribución sobre los retornos en lugar de únicamente un retorno esperado (Bellemare et al. 2017).

4.6.1 Papel algorítmico

El DQN estándar aproxima los valores-acción con una red neuronal y selecciona acciones según el retorno esperado estimado (Mnih et al. 2015). QRDQN extiende esta idea aprendiendo los cuantiles de la distribución del retorno. Esto es relevante para la tesis porque la estructura de recompensa

es asimétrica: los falsos negativos conllevan una penalización mucho mayor que la recompensa que generan los permisos benignos correctos. Una visión distribucional puede representar más información sobre la variabilidad del retorno que una única estimación de la media.

La metodología no asume que QRDQN sea inherentemente superior a los clasificadores supervisados para datos de flujo tabulares. En cambio, evalúa QRDQN como el representante principal de RL bajo el mismo preprocesamiento, las mismas particiones y las mismas métricas utilizadas por las líneas base.

La arquitectura de red concreta de la ejecución MAIN se resume en la Figura 4.4: una entrada de 152 dimensiones, tres capas densas ocultas y una salida distribucional de 200 cuantiles por cada una de las dos acciones.

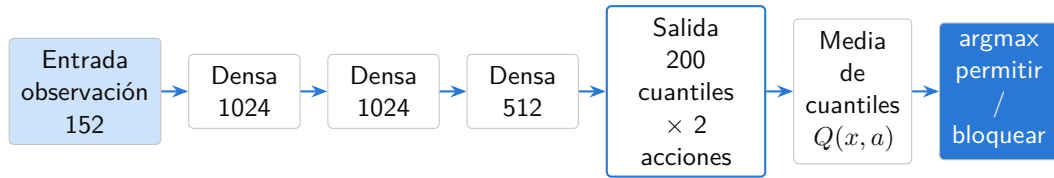


Figura 4.4: Arquitectura QRDQN de la ejecución MAIN: observación canónica, capas densas, cuantiles por acción y decisión final por media de cuantiles. Fuente: elaboración propia a partir de `config.json` de la ejecución MAIN.

4.6.2 Función de pérdida de regresión cuantílica

QRDQN representa la distribución del retorno de acción mediante N estimaciones de cuantiles aprendidas $\{\theta_i(s, a)\}_{i=1}^N$ en puntos medios de cuantil fijos $\hat{\tau}_i = \frac{2i-1}{2N}$ (Dabney et al. 2018). Durante la selección de acción, estos cuantiles se promedian para producir una estimación del retorno esperado, recuperando la regla codiciosa estándar del DQN bajo expectativa.

Para una única transición (s, a, r, s') , sea $a^* = \arg \max_a \sum_j \theta_j^-(s', a)$ la acción codiciosa bajo la red objetivo, y sea

$$\delta_{ij} = r + \gamma \theta_j^-(s', a^*) - \theta_i(s, a)$$

el error de diferencia temporal entre el cuantil fuente i y el cuantil objetivo j calculado mediante bootstrapping. QRDQN minimiza la función de pérdida de regresión cuantílica de Huber sumada sobre todos los N^2 pares de cuantiles:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \left| \hat{\tau}_i - \mathbf{1}[\delta_{ij} < 0] \right| \cdot L_{\kappa}(\delta_{ij}),$$

donde L_{κ} es la pérdida de Huber elemento a elemento con umbral κ :

$$L_{\kappa}(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{si } |\delta| \leq \kappa, \\ \kappa\left(|\delta| - \frac{1}{2}\kappa\right) & \text{en caso contrario.} \end{cases}$$

A diferencia de C51, que representa los retornos sobre un soporte categórico fijo, QRDQN no impone ninguna restricción sobre el soporte de la distribución del retorno y se adapta automáticamente a la escala de las recompensas (Bellemare et al. 2017). En el entorno de recompensa asimétrica de esta tesis, la distribución del retorno puede desarrollar colas negativas pronunciadas cuando los eventos de falso negativo son frecuentes, lo que hace que una visión distribucional sea más informativa que una estimación escalar de la media para comprender el comportamiento del agente en torno a la frontera de decisión PERMIT/BLOCK.

4.6.3 Factor de descuento $\gamma = 0$: formulación de un solo paso

En todas las ejecuciones de esta tesis, incluida la ejecución oficial, el factor de descuento se fija en $\gamma = 0$ (valor registrado en el artefacto de configuración de la ejecución). La elección es deliberada y coherente con la formulación conjunto-de-datos-como-entorno: dado que el entorno es un conjunto de datos estático y las acciones del agente no afectan a los estados futuros, no existe dependencia temporal entre flujos que un factor de descuento positivo pudiera explotar.

Con $\gamma = 0$, el error de diferencia temporal definido más arriba se reduce a

$$\delta_{ij} = r - \theta_i(s, a),$$

es decir, el término de bootstrapping $\gamma \theta_j^-(s', a^*)$ se anula y la red objetivo deja de contribuir. Cada cuantil aprendido $\theta_i(s, a)$ regresa por tanto hacia la recompensa inmediata de la decisión, y QRDQN actúa como un estimador distribucional de un solo paso: formalmente, un *bandit contextual sensible al coste*, y no un proceso de decisión de Markov secuencial.

Esta reducción no vacía de contenido la elección de QRDQN. La cabeza distribucional sigue modelando la distribución completa de la recompensa asimétrica en torno a la frontera PERMIT/BLOCK —y no únicamente su media—, lo que aporta información sobre la incertidumbre del valor bajo coste asimétrico. La diferencia frente a un clasificador supervisado no reside en la presencia de dinámica secuencial, inexistente aquí por diseño, sino en que el aprendizaje se realiza mediante valores sobre una matriz de coste explícita (con la penalización de falso negativo muy superior a la de falso positivo) en lugar de mediante entropía cruzada sobre etiquetas. La tesis no reivindica autonomía secuencial ni respuesta adversarial en múltiples pasos: tal y como se recoge en las limitaciones de la formulación, esas capacidades quedan fuera del alcance implementado.

4.6.4 Estrategia de exploración

Durante el entrenamiento, el agente emplea una política ϵ -greedy. Con probabilidad ϵ selecciona una acción uniformemente aleatoria de {PERMIT, BLOCK}; con probabilidad $1 - \epsilon$ selecciona la acción cuya estimación media de cuantiles es más alta (Mnih et al. 2015). ϵ decae linealmente desde un valor inicial cercano a uno hasta un valor final pequeño a lo largo de una fracción fija del total de pasos de entrenamiento, y luego permanece constante. En el entorno de acción binaria, el presupuesto de exploración es relativamente barato porque

solo existen dos alternativas. La fracción de exploración y el valor final de ε se almacenan en el artefacto de configuración de la ejecución para que las ejecuciones con diferentes programas no se mezclen o comparen en silencio sin tener en cuenta sus diferencias de exploración.

4.6.5 Configuración del entrenamiento

La implementación utiliza el módulo QRDQN de `sb3-contrib` (Stable-Baselines3 Contributors 2023). El modelo emplea una política de perceptrón multicapa sobre las observaciones de 152 dimensiones. El entrenamiento utiliza un buffer de repetición, actualizaciones por minibatch, una red objetivo y predicción determinista durante la evaluación. El script de entrenamiento mantenido admite preajustes rápidos y completos, modos de partición aleatoria y por día, semillas explícitas, límites de filas configurables y pasos temporales configurables.

Los valores por defecto del preajuste completo actual utilizan lotes más grandes y más pasos de gradiente que el preajuste rápido. Esta separación permite al proyecto ejecutar comprobaciones rápidas de humo sin confundirlas con las ejecuciones finales del benchmark. Cualquier resultado reportado debe incluir el identificador exacto de la ejecución y la configuración.

El Algoritmo 3 formaliza este pipeline de entrenamiento de extremo a extremo, desde la carga de los datos brutos de CICIDS2017 hasta la persistencia de los artefactos finales, con independencia del preajuste concreto o de los valores de hiperparámetros seleccionados en cada ejecución. La secuencia de pasos que ejecuta el script mantenido es única; solo los valores de las variables de configuración, documentados aparte, distinguen una ejecución de otra.

Algoritmo 3 Entrenamiento del agente defensor QRDQN sobre CICIDS2017: carga y limpieza de datos, mapeo canónico, partición train/test, ajuste del escalador exclusivamente sobre train, bucle de entrenamiento QRDQN con $\gamma = 0$ y persistencia de artefactos. Fuente: elaboración propia a partir de `src/train_rl_defender.py`.

Require: CSV de CICIDS2017; semilla; preajuste (rápido/completo); modo de partición (aleatorio/día); hiperparámetros QRDQN $\eta, B, G, \tau, \varepsilon_0, \varepsilon_T, f_\varepsilon$

Ensure: Modelo QRDQN, escalador, percentiles de entrenamiento, configuración y métricas de test persistentes

- 1: Fijar la semilla en `numpy/random/torch`; cargar y limpiar los CSV según el preajuste $\triangleright$ sin escalar
 - 2: Mapear a esquema canónico (76 características + máscara de 76) $\rightarrow$ observaciones de 152 dimensiones; particionar en $(X_{\text{train}}, y_{\text{train}}), (X_{\text{test}}, y_{\text{test}})$
 - 3: Calcular percentiles $p_{0.5}, p_{99.5}$ de X_{train} sin escalar; ajustar $\Sigma = \text{StandardScaler}$ solo sobre X_{train} $\triangleright X_{\text{test}}$ no interviene en el ajuste
 - 4: $X_{\text{train}} \leftarrow \Sigma(X_{\text{train}})$; $X_{\text{test}} \leftarrow \Sigma(X_{\text{test}})$; persistir Σ , percentiles, nombres de características y metadatos del entorno
 - 5: Construir el entorno Gymnasium sobre $(X_{\text{train}}, y_{\text{train}})$ con la matriz de recompensa fijada
 - 6: Inicializar red de cuantiles θ , red objetivo $\theta^- \leftarrow \theta$, buffer $\mathcal{D} \leftarrow \emptyset$, $\varepsilon \leftarrow \varepsilon_0$, $t \leftarrow 0$
 - 7: **while** $t < \text{total de pasos temporales}$ **do**
 - 8: Observar s ; con probabilidad ε , $a \leftarrow$ acción aleatoria en $\{\text{PERMIT}, \text{BLOCK}\}$; en caso contrario $a \leftarrow \arg \max_{a'} \frac{1}{N} \sum_i \theta_i(s, a')$
 - 9: Ejecutar a ; observar r, s' , terminado; almacenar $(s, a, r, s', \text{terminado})$ en $\mathcal{D}$; $t \leftarrow t + 1$
 - 10: Actualizar ε por decaimiento lineal de ε_0 a ε_T sobre la fracción f_ε del total de pasos
 - 11: **if** $t \bmod \text{intervalo de red objetivo} = 0$ **then**
 - 12: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ $\triangleright \tau = 1$: copia dura
 - 13: **end if**
 - 14: **if** $t \bmod \text{train_freq} = 0$ y $t \geq \text{learning_starts}$ **then**
 - 15: **for** $g = 1, \dots, G$ **do**
 - 16: Muestrear de $\mathcal{D}$ un minibatch $\{(s_k, a_k, r_k, s'_k, \text{terminado}_k)\}_{k=1}^B$
 - 17: Para cada k y cada cuantil objetivo $j = 1, \dots, N$: $y_j \leftarrow r_k$ $\triangleright \gamma = 0$: sin bootstrap ni θ^- ; regresión hacia r_k
 - 18: Pérdida $\mathcal{L} \leftarrow$ Huber cuantílica entre $\theta_i(s_k, a_k)$ y $\{y_j\}$, sumada sobre todos los pares (i, j)
 - 19: Actualizar θ por descenso de gradiente sobre $\mathcal{L}$ (Adam, tasa η); recortar norma del gradiente si corresponde
 - 20: **end for**
 - 21: **end if**
 - 22: **end while**
 - 23: Guardar el modelo; evaluar de forma determinista sobre $(X_{\text{test}}, y_{\text{test}})$; calcular matriz de confusión y métricas
 - 24: Persistir configuración final, métricas de test y manifiesto de artefactos
-

4.6.6 Procedencia de los hiperparámetros

Los hiperparámetros de la ejecución oficial ($\gamma = 0$, arquitectura [1024, 1024, 512], 200 cuantiles, tasa de aprendizaje 5×10^{-5} , 20 pasos de gradiente y 3 000 000 de pasos de entrenamiento) constituyen un *perfil fijo establecido manualmente*, no el resultado de una búsqueda automática. El repositorio incluye una sonda de optimización con Optuna (`src/tune_hparams.py`) con fines exploratorios, pero su espacio de búsqueda — $\gamma \in [0,95, 0,999]$, arquitecturas en $\{[256, 128], [512, 256], [256, 256]\}$ y pasos de gradiente en $\{10, 50, 100\}$ — *no contiene* la configuración oficial. Por tanto, dicha configuración no pudo originarse en esa búsqueda y no debe interpretarse como un óptimo ajustado por Optuna; responde a un diseño manual coherente con la formulación de un solo paso, y queda congelada y verificada por las pruebas del proyecto.

El listado completo de valores empleados en la ejecución oficial MAIN se recoge en la Tabla 4.4.

4.6.7 Persistencia del modelo y el preprocesamiento

El entrenamiento produce un directorio de ejecución y un artefacto de modelo. El directorio de ejecución almacena la configuración, las métricas, el escalador ajustado y los límites de percentiles de entrenamiento. Persistir los artefactos de preprocesamiento es esencial porque la inferencia en la Fase 2 debe utilizar el mismo estado de normalización que el entrenamiento. Recalcular un escalador sobre el tráfico de laboratorio filtraría información de la distribución de inferencia hacia la interfaz del modelo y haría la ejecución irreproducible.

Hiperparámetro	Valor	Observación
Arquitectura de la red (<code>net_arch</code>)	[1024, 1024, 512]	MLP de tres capas ocultas
Número de cuantiles (<code>n_quantiles</code>)	200	distribución de retorno QRDQN
Tasa de aprendizaje	5×10^{-5}	
Tamaño del buffer de repetición	1 000 000	transiciones
Pasos de calentamiento (<code>learning_starts</code>)	50 000	
Tamaño de lote (<code>batch_size</code>)	2048	
Factor de descuento (γ)	0.0	formulación de un solo paso
Tasa de actualización objetivo (τ)	1.0	con $\gamma = 0$ la red objetivo no contribuye al error de diferencia temporal
Intervalo de actualización objetivo	10 000	pasos
Frecuencia de entrenamiento (<code>train_freq</code>)	100	pasos
Pasos de gradiente por actualización	20	
ε inicial de exploración	1.0	decaimiento lineal
ε final de exploración	0.02	decaimiento lineal
Fracción de exploración	0.1	decaimiento lineal
Norma máxima de gradiente	10.0	
Pasos totales de entrenamiento	3 000 000	
Semilla	42	

Tabla 4.4: Hiperparámetros de la ejecución oficial MAIN. Perfil fijado a mano, no procedente de la sonda Optuna. Fuente: elaboración propia a partir de `runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655/config.json`.

4.7 Marco de implementación

El pipeline experimental está implementado en Python y se apoya en un conjunto de bibliotecas de código abierto bien establecidas. El componente de RL profundo principal utiliza la extensión `sb3-contrib` de Stable Baselines3, que proporciona una implementación contrastada de QRDQN con arquitectura de red configurable, buffer de repetición, programa de exploración y actualizaciones de la red objetivo (Stable-Baselines3 Contributors 2023). El envoltorio del entorno sigue la API de Gymnasium (Farama Foundation 2023), haciendo que el conjunto de datos de flujos sea compatible con cualquier bucle de entrenamiento que cumpla con Gymnasium sin cambios en la implementación del algoritmo.

4.7.1 Pila de procesamiento de datos y aprendizaje supervisado

La carga y el preprocesamiento de datos se apoyan en `pandas` para la ingesta de CSV, la estandarización de columnas y la lectura fragmentada de ficheros grandes. Las matrices de características se convierten a arrays de `numpy` y se castean a `float32` antes de entrar en el entorno de Gymnasium o en la línea base supervisada. La estandarización utiliza `StandardScaler` de `scikit-learn`, elegido por su compatibilidad con el protocolo de serialización de `sklearn`, que permite persistir y recargar los escaladores ajustados sin reimplementación. La línea base de Random Forest también utiliza `scikit-learn`, consumiendo los mismos arrays de entrada que entran en el entorno de RL. Esta ruta de datos compartida garantiza que cualquier diferencia de rendimiento entre los dos modelos refleje el comportamiento algorítmico y no una discrepancia en el manejo de los datos.

4.7.2 Gestión de artefactos

Cada ejecución de entrenamiento produce un `RUN_ID` único compuesto por una marca temporal y un sufijo aleatorio corto. El directorio de ejecución reúne la política QRDQN serializada, el escalador ajustado, los límites de percentiles, un JSON de configuración y un JSON de métricas. Las ejecuciones de validación y de inferencia de la Fase 2 producen subdirectorios independientes con sus propios registros de configuración y métricas. Este esquema de directorios plano evita dependencias de bases de datos al tiempo que hace las ejecuciones recuperables de forma independiente y comparables mediante inspección del sistema de ficheros.

4.7.3 Controles de reproducibilidad

Las semillas se fijan para los generadores de números aleatorios de `numpy`, `random` y `torch` al comienzo de cada ejecución. Los reinicios del entorno de `Gymnasium` también reciben una semilla explícita. El pipeline está preparado para que, cuando se utilicen múltiples semillas para la estimación de la varianza, cada semilla genere un directorio de ejecución independiente de modo que los resultados por semilla se preserven individualmente en lugar de promediarse en silencio; no obstante, los resultados reportados en esta tesis corresponden a una única semilla y no incluyen una estimación de varianza entre ejecuciones. Estos controles buscan reducir la varianza interna a la ejecución lo suficiente como para distinguir diferencias de rendimiento significativas del ruido, aunque no pueden eliminar todo el indeterminismo derivado del ordenamiento de los kernels de la GPU.

Protocolo experimental

5.1 Fases experimentales

La metodología experimental se divide en dos fases. La primera fase comprueba el aprendizaje y la generalización interna sobre un benchmark público. La segunda fase comprueba si el pipeline de preprocesamiento y modelo resultante es capaz de procesar tráfico capturado por el operador en un laboratorio doméstico cerrado bajo cambio de dominio.

5.1.1 Fase 1: benchmark interno de CICIDS2017

La Fase 1 utiliza CICIDS2017 como benchmark interno principal (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017). CICIDS2017 es adecuado para esta tesis porque proporciona tráfico benigno y de ataque etiquetado, capturas de paquetes en bruto y CSVs de flujos derivados de una extracción al estilo CICFlowMeter (Habibi Lashkari et al. 2017). También contiene múltiples escenarios de ataque en diferentes días de captura, lo que permite diseños de partición más estrictos que una única partición aleatoria entrenamiento-prueba.

La fase no se trata como un sustituto limpio de un entorno de producción. CICIDS2017 tiene problemas conocidos que incluyen flujos duplicados o inconsistentes, errores de etiquetado y de extracción, y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022). Por esa razón, la Fase 1 se enmarca como un benchmark interno con controles explícitos. El pipeline elimina los campos de tipo identificador, ajusta la normalización únicamente sobre los datos de entrenamiento, registra las configuraciones de ejecución y evalúa el modelo mediante una escalera de validación. Una puntuación elevada en la partición aleatoria se interpreta como evidencia de que el modelo es capaz de aprender la tarea del benchmark; no se interpreta como preparación para el despliegue.

La estructura por día de captura de CICIDS2017 (Tabla 5.1) muestra una proporción de ataque muy desigual entre ficheros, desde el 0 % del lunes hasta más del 55 % en las capturas del viernes por la tarde. Esta heterogeneidad motiva los protocolos de partición por día, más estrictos que una partición aleatoria, que se describen más adelante en este capítulo.

Día	Filas	Benigno	Ataque	Ataque (%)
Lunes	529 918	529 918	0	0.0 %
Martes	445 909	432 074	13 835	3.1027 %
Miércoles	692 703	440 031	252 672	36.4762 %
Jueves (mañana, ataques web)	170 366	168 186	2 180	1.2796 %
Jueves (tarde, infiltración)	288 602	288 566	36	0.0125 %
Viernes (mañana)	191 033	189 067	1 966	1.0291 %
Viernes (tarde, PortScan)	286 467	127 537	158 930	55.4793 %
Viernes (tarde, DDoS)	225 745	97 718	128 027	56.7131 %

Tabla 5.1: Composición por día de CICIDS2017: número de filas y proporción de tráfico benigno frente a ataque por fichero oficial de captura. Fuente: elaboración propia a partir de `memoria/figuras/data_composicion_dia.json`.

La Figura 5.1 ofrece la misma información como comparación visual entre días. La lectura relevante para el protocolo no es solo el volumen total, sino la variación extrema de prevalencia de ataque entre capturas, que justifica no

depender únicamente de una partición aleatoria por filas.

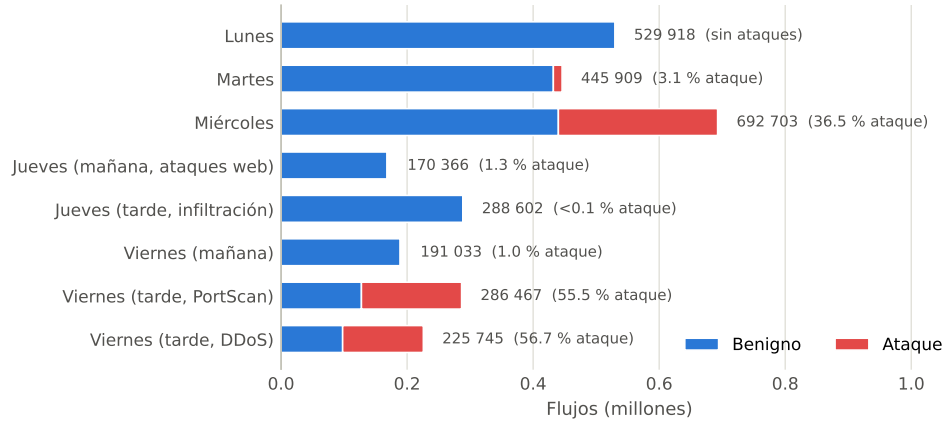


Figura 5.1: Composición visual por día de CICIDS2017. Fuente: elaboración propia a partir de `memoria/figuras/data_composicion_dia.json`.

5.1.2 Fase 2: inferencia offline sobre tráfico de laboratorio

La Fase 2 evalúa la transferencia más allá de la distribución del benchmark. El tráfico se captura en un entorno de laboratorio privado, se convierte offline en CSVs de flujos, se mapea al mismo esquema canónico, se normaliza con los artefactos de entrenamiento persistidos y se procesa con el modelo entrenado. Esta fase es una prueba de inferencia sobre una distribución externa, no un despliegue en vivo.

El propósito metodológico de la Fase 2 es exponer el cambio de dominio. Los estudios públicos sobre NIDS suelen reportar resultados sólidos dentro del mismo conjunto de datos que se degradan cuando el modelo se evalúa sobre tráfico producido por redes, herramientas de captura, periodos temporales o definiciones de características diferentes (Apruzzese et al. 2022; Layeghy et al. 2023). La Fase 2 se centra por tanto en la inferencia offline reproducible, los diagnósticos y el seguimiento de artefactos. No garantiza un rendimiento estable en redes empresariales arbitrarias y no implementa bloqueo activo en tiempo real.

5.2 Separación entrenamiento-prueba y escalado

La metodología separa el ajuste y la evaluación en cada etapa de preprocesamiento que pueda aprender de los datos. Esto es necesario porque la fuga de información puede producirse no solo a través de columnas, sino también a través de estadísticas como medias globales, varianzas, percentiles y distribuciones de clases.

5.2.1 Partición aleatoria estratificada

La partición aleatoria estratificada es el primer peldaño de validación. Preserva las proporciones de clases entre entrenamiento y prueba y es útil como comprobación de la capacidad de aprendizaje. Si el modelo no puede aprender bajo esta partición favorable, las evaluaciones más estrictas posteriores tienen pocas probabilidades de tener éxito.

En la ejecución MAIN, la estratificación mantiene prácticamente idéntica la prevalencia de ataque entre entrenamiento y prueba, como muestra la Figura 5.2. Esto permite atribuir los cambios de rendimiento entre entrenamiento y evaluación al modelo y al protocolo, no a una variación accidental de la proporción de clases en la partición aleatoria.

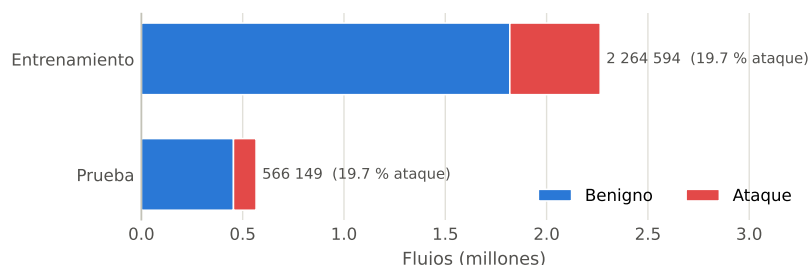


Figura 5.2: Balance de clases en la partición aleatoria fija de la ejecución MAIN. Fuente: elaboración propia a partir de `config.json` de la ejecución MAIN.

Sin embargo, la partición aleatoria por filas es la evidencia más débil de este proyecto. Los conjuntos de datos de flujos pueden contener registros correlacionados, patrones repetidos y artefactos del contexto de captura. Situar flujos adyacentes o muy similares a ambos lados de la partición puede sobreestimar la generalización (Lanvin et al. 2023; Arp et al. 2020). Los resultados de la partición aleatoria se reportan por tanto únicamente como evidencia del benchmark interno.

5.2.2 Partición por CSV/día

La partición por CSV/día entrena sobre ficheros seleccionados de CICSIDS2017 y evalúa sobre ficheros diferentes. Esta partición comprueba si el modelo se transfiere entre días de captura y contextos de ataque. En la implementación mantenida, la partición por día por defecto entrena sobre los patrones del lunes, martes y miércoles, y prueba sobre los patrones del jueves y viernes. Se trata de una configuración más estricta que la partición aleatoria porque ficheros enteros quedan fuera del entrenamiento.

La partición por día también se alinea con la estructura conocida de CICSIDS2017: diferentes días contienen diferentes cargas de trabajo benignas y escenarios de ataque. Por ello es más adecuada para revelar si el modelo ha aprendido un comportamiento de flujo general o artefactos específicos de un día.

5.2.3 Partición leave-one-CSV-out

El protocolo interno más estricto es la validación leave-one-CSV-out. Cada fold retiene un CSV oficial de CICSIDS2017 como datos de prueba y entrena con los CSV restantes. El script de validación implementado agrega métricas por fold, incluyendo exactitud, exactitud balanceada, precisión de ataque, recall de ataque, F1 de ataque, precisión de benigno, recall de benigno, tasa de falsos

positivos, tasa de falsos negativos, tasa de bloqueo, retorno total y costes temporales.

Este protocolo es metodológicamente importante porque trata cada fichero de captura como una unidad de tipo dominio. Es más sólido que una partición por día basada en subcadenas porque utiliza los nombres de fichero CSV oficiales exactos y evita solapamientos accidentales. En el momento de redacción de este capítulo, la implementación existe, mientras que las métricas agregadas finales solo deben reportarse cuando se disponga de un artefacto de ejecución comprometido.

5.2.4 Estandarización ajustada sobre entrenamiento

El entrenamiento de redes neuronales es sensible a la escala de las características. El pipeline utiliza por tanto normalización mediante `StandardScaler`, ajustando el escalador únicamente sobre la partición de entrenamiento y aplicando la transformación ajustada a los datos de prueba. El mismo principio se aplica a los artefactos de inferencia persistidos: el escalador guardado durante el entrenamiento se reutiliza en la Fase 2, en lugar de recalcularse a partir de los flujos de laboratorio.

Para la inferencia robusta en la Fase 2, el pipeline de entrenamiento también persiste los límites de percentiles sobre las características canónicas en bruto. El script de inferencia de la Fase 2 puede aplicar recorte por percentiles sobre las características en bruto y recorte por z-score después del escalado, y luego exportar diagnósticos. Estos controles no se presentan como una solución al cambio de dominio; son salvaguardas que hacen visibles y acotados los valores extremos fuera de distribución durante la inferencia offline.

5.3 Línea base supervisada

Un método de RL para NIDS a nivel de flujo debe compararse con líneas base supervisadas sólidas. Los datos de flujo tabulares son un entorno donde los modelos basados en árboles, especialmente los Random Forests y los ensamblados con gradient boosting, suelen ser competitivos (H. Liu et al. 2019; Apruzzese et al. 2022).

5.3.1 Línea base con Random Forest

La línea base supervisada principal es un clasificador Random Forest. La línea base es metodológicamente útil porque obtiene buenos resultados en datos tabulares, gestiona las interacciones no lineales entre características y requiere menos complejidad de entrenamiento que el RL profundo. También proporciona una comprobación de cordura frente a la sobreestimación del valor de la formulación de RL en una tarea que puede comportarse como una clasificación sensible al coste.

5.3.2 Comparación bajo el mismo protocolo

La línea base debe consumir la misma representación canónica de observaciones, utilizar los mismos protocolos de partición y reportar las mismas métricas que QRDQN. Esta regla de mismo protocolo evita comparaciones sesgadas. Si la línea base utiliza un preprocesamiento diferente, controles de fuga distintos o particiones más sencillas, la comparación no aísla la contribución algorítmica de QRDQN.

La metodología trata por tanto la comparación con la línea base como un requisito del protocolo y no como un apéndice opcional. También evita afirmar la superioridad de QRDQN hasta que existan artefactos de línea base con la misma partición.

5.3.3 Desbalanceo de clases y ponderación sensible al coste

CICIDS2017 presenta desbalanceo de clases: los flujos benignos superan sustancialmente en número a los flujos de ataque, y dentro de los flujos de ataque las familias individuales varían en prevalencia. Un Random Forest entrenado con pesos de clase uniformes por defecto tenderá por tanto a sesgar sus predicciones hacia la clase benigna mayoritaria. Para que la comparación sea significativa bajo los supuestos de coste asimétrico de la tesis, la línea base de Random Forest utiliza ponderación de clases balanceada, donde cada clase recibe un peso inversamente proporcional a su frecuencia en la partición de entrenamiento. Esto garantiza que el modelo supervisado no sea penalizado en la misma dirección que la función de recompensa de RL por razones diferentes.

La configuración de los pesos de clase se registra en el artefacto de configuración de la línea base junto al modo de partición, el esquema de características y la semilla de evaluación, de modo que el supuesto de ponderación es trazable. Si la tesis encuentra que el Random Forest con ponderación balanceada alcanza un rendimiento en falsos negativos similar o superior al de QRDQN bajo la recompensa asimétrica, esa comparación es informativa precisamente porque ambos modelos han tenido acceso a mecanismos de sensibilidad al coste comparables. Comparar un QRDQN ajustado al coste frente a un Random Forest sin corrección no aportaría esta evidencia.

5.4 Protocolo de escala de entrenamiento y eficiencia de datos

La metodología contempla un componente de escala de entrenamiento, planificado pero aún no ejecutado en esta tesis, motivado por una pregunta

práctica: ¿cuántos datos de flujo etiquetados necesita cada enfoque antes de alcanzar un rendimiento estable? Esto es relevante porque el tráfico de ataque etiquetado es costoso de obtener, la abundancia de benchmarks no refleja la disponibilidad en el despliegue, y los métodos de RL profundo pueden ser sustancialmente más exigentes en cuanto a muestras que los clasificadores supervisados basados en árboles (Sutton et al. 2018; Ring et al. 2019).

5.4.1 Protocolo de curva de aprendizaje

El protocolo de curva de aprendizaje, planificado pero todavía no ejecutado en esta tesis (sin artefactos comprometidos), evaluaría tanto el agente QRDQN como la línea base de Random Forest con presupuestos de entrenamiento progresivamente mayores, todos extraídos de la misma partición estratificada. El conjunto de evaluación se mantendría constante en todos los presupuestos como la partición de prueba completa, de modo que los cambios en las métricas reflejarían únicamente el tamaño de los datos de entrenamiento y no las condiciones de evaluación. Las métricas se registrarían en cada nivel de presupuesto utilizando el mismo código de evaluación que las ejecuciones principales del benchmark.

En este protocolo, cada presupuesto utilizaría un escalador ajustado de nuevo calculado únicamente a partir de las filas incluidas en ese subconjunto de entrenamiento. Los presupuestos más grandes se beneficiarían por tanto de una normalización más representativa sin contaminar la evaluación fija de prueba. Esto significa que los artefactos de preprocesamiento producidos en cada nivel de presupuesto serían distintos e independientemente reproducibles.

5.4.2 Niveles de presupuesto y comparación

Los niveles de presupuesto candidatos incluyen 100 k, 250 k, 500 k, 1 M y 2 M registros de flujo. No todos los niveles son alcanzables bajo todos los modos

de partición; cuando una partición no produce conjuntos de entrenamiento del tamaño objetivo, se registra el tamaño disponible más cercano. El presupuesto final es la partición de entrenamiento completa bajo la partición seleccionada.

Comparar QRDQN y Random Forest en cada nivel de presupuesto revela si la formulación de RL tiene un requisito de muestras o una trayectoria de convergencia diferente. Si el Random Forest alcanza una meseta de rendimiento estable con sustancialmente menos filas, esto constituye evidencia interna de que la tarea del benchmark no requiere la complejidad de muestras adicional del RL profundo sobre el esquema de características actual. Por el contrario, si el comportamiento de RL sensible al coste mejora desproporcionadamente a medida que crece el presupuesto, los resultados de escala de entrenamiento pueden identificar a qué escala la estructura de recompensa distribucional comienza a aportar una ventaja medible. Cuando este protocolo se ejecute, todos los resultados de escala de entrenamiento se almacenarían bajo el mismo esquema de artefactos de ejecución que los experimentos principales, etiquetados por nivel de presupuesto y semilla, de modo que permanecerían recuperables de forma independiente y comparables en experimentos futuros. En el momento de redacción de este capítulo no se ha comprometido ningún artefacto de curva de aprendizaje.

5.5 Salidas de evaluación y reproducibilidad

La reproducibilidad es una restricción de diseño fundamental, especialmente porque la investigación en seguridad con ML suele adolecer de código faltante, detalles de procesamiento de datos incompletos y registros de evaluación incompletos (Olszewski et al. [2023](#)).

5.5.1 Métricas

Las métricas binarias principales son exactitud, precisión, recall y F1 tanto para la clase benigna como para la de ataque. El análisis también rastrea los conteos de la matriz de confusión: verdaderos positivos, falsos positivos, falsos negativos y verdaderos negativos. Estos conteos son necesarios porque dos modelos con exactitud agregada similar pueden tener implicaciones de seguridad muy diferentes.

La tasa de falsos positivos y la tasa de falsos negativos son especialmente importantes. La tasa de falsos positivos mide la proporción de tráfico benigno bloqueado incorrectamente, lo que se relaciona con la disponibilidad y la carga de trabajo del analista. La tasa de falsos negativos mide la proporción de ataques permitidos incorrectamente, lo que se relaciona directamente con el riesgo de intrusiones no detectadas. El análisis de precisión y recall es particularmente relevante bajo desbalanceo de clases, donde la exactitud sola puede ser engañosa (Fawcett 2006; Saito et al. 2015).

5.5.2 Escalera de validación

La escalera de evaluación está ordenada de la evidencia más débil a la más sólida:

1. partición aleatoria estratificada, utilizada como comprobación de la capacidad de aprendizaje;
2. comprobación de predicción directa, que verifica las salidas del modelo frente a las etiquetas de prueba sin depender de los metadatos del entorno;
3. comprobación de etiquetas barajadas, que ayuda a detectar fugas de información confirmando que el rendimiento se degrada cuando las etiquetas de entrenamiento se aleatorizan;
4. partición por CSV/día, que retiene grupos de captura completos;

5. validación leave-one-CSV-out, que retiene cada CSV oficial por turnos;
6. inferencia sobre tráfico de laboratorio en la Fase 2, que prueba el pipeline entrenado sobre tráfico capturado por el operador en un laboratorio doméstico cerrado.

En la implementación, los peldaños segundo a cuarto se corresponden con las comprobaciones *Check A* (predicción directa), *Check B* (etiquetas barajadas) y *Check C* (partición por CSV/día) del script `src/validate_checks.py`; el quinto peldaño se implementa en `src/validate_leave_one_csv_out.py`.

La Figura 5.3 sintetiza esta escalera de validación y sitúa cada peldaño junto al artefacto o resultado que lo respalda. La figura no añade métricas nuevas: reordena visualmente las cifras ya documentadas para hacer explícito el aumento progresivo de exigencia.

Cada peldaño apunta a un modo de fallo diferente. La escalera no hace que ningún resultado individual sea definitivo, pero hace la evaluación más informativa que una única puntuación aleatoria de entrenamiento-prueba.

La partición aleatoria estratificada comprueba si el modelo y la implementación pueden resolver el benchmark en absoluto. Un modelo que falla aquí tiene un problema de entrenamiento o de arquitectura, no un problema de generalización. La comprobación de etiquetas barajadas es un detector de fugas: un modelo que alcanza una exactitud muy por encima de la de la clase mayoritaria en una ejecución con etiquetas barajadas ha aprendido a partir de artefactos correlacionados con las etiquetas en las características y no a partir de estadísticas de flujo reales (Arp et al. 2020). La partición por CSV/día comprueba la generalización temporal y por escenario reteniendo grupos de captura completos; la degradación del rendimiento respecto a la partición aleatoria revela si el modelo depende del contexto específico del día. El protocolo leave-one-CSV-out va más lejos: cada fichero oficial de CICIDS2017 se retiene por turnos, y la distribución de métricas agregada muestra si algún fichero domina el benchmark o si el rendimiento es estable entre escenarios de captura.

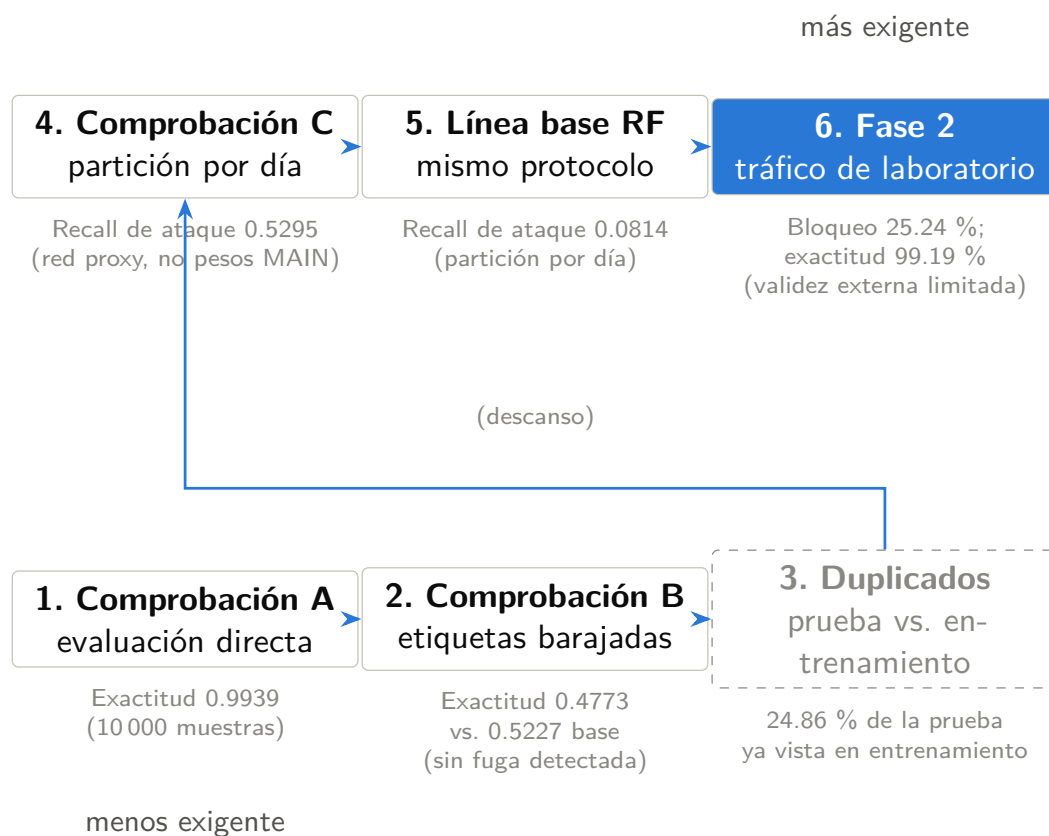


Figura 5.3: Escalera de validación del proyecto, desde la evaluación directa hasta la inferencia de Fase 2. Fuente: elaboración propia a partir de los artefactos de validación y Fase 2 bajo runs/.

La inferencia en la Fase 2 es el peldaño final y más estricto, que comprueba si todo el pipeline puede operar sobre tráfico producido fuera del entorno del benchmark.

5.5.3 Artefactos de ejecución

Toda ejecución de entrenamiento o evaluación significativa debe persistir un RUN_ID único, un fichero de configuración, métricas y cualquier estado de preprocesamiento necesario para su reutilización exacta. Las ejecuciones de entrenamiento persisten los pesos del modelo y los artefactos del escalador. Las ejecuciones de validación persisten los resultados de validación. Las ejecuciones de inferencia de la Fase 2 persisten las predicciones, las métricas, la configuración y los diagnósticos opcionales.

Esta disciplina de artefactos evita afirmaciones ambiguas. Un resultado solo se trata como medido cuando puede vincularse a una carpeta de ejecución y una configuración específicas. Las ejecuciones históricas no se fusionan en silencio con los valores por defecto actuales; los valores de recompensa, la lógica de partición, el preprocesamiento, los límites de filas y las semillas deben leerse del artefacto.

La Tabla 5.2 recoge el inventario completo de las ejecuciones oficiales – organizadas por el peldaño de la escalera de validación o el análisis que respaldan – junto con los artefactos concretos que permiten reproducir o auditar cada cifra citada en esta memoria. Las ejecuciones no listadas en la tabla (variantes históricas, sondeos previos al diseño o repeticiones posteriores con configuración distinta) se consideran no oficiales y no deben citarse como respaldo de ninguna cifra reportada.

Ejecución (RUN_ID)	Propósito	Artefactos clave
MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655	Entrenamiento principal QRDQN y evaluación en la partición aleatoria estratificada fija (semilla 42).	config.json; metrics.json; modelo entrenado en models/ (mismo RUN_ID); scaler.joblib; train_percentiles.npz; feature_names.json; artifact_manifest.json
VAL_checks_A_20260212_235443	Comprobación A: predicción directa sin depender de los metadatos del entorno.	config.json; validation_results.json
VAL_checks_B_20260212_235736	Comprobación B: etiquetas barajadas (detección de fugas de información).	config.json; validation_results.json
VAL_checks_C_20260213_004847	Comprobación C: partición por día.	config.json; validation_results.json
bootstrap_ci_seed42.json	Intervalos de confianza bootstrap (10 000 remuestreos estratificados) sobre las métricas de MAIN en la partición de prueba fija, sin reentrenamiento.	bootstrap_ci_seed42.json (incluye los conteos de la matriz de confusión y la verificación del hash SHA-256 del modelo)
duplicate_analysis_seed42.json	Análisis de duplicados y de fuga cruzada entre entrenamiento y prueba sobre la partición aleatoria fija.	duplicate_analysis_seed42.json
rf_cicids2017_canonical_20260628_024735	Línea base supervisada Random Forest (balanceada y escalada) sobre tres particiones: aleatoria, por día y leave-one-out (miércoles).	config.json + metrics.json por subejecución: __random_split, __day_split, __leave_one_out_wednesday
P2v2_pred_20260610_161231_MAIN	Inferencia de la Fase 2 sobre tráfico de laboratorio etiquetado, con el modelo MAIN.	config.json; metrics.json; diagnostics.json; predictions.csv.gz

Tabla 5.2: Inventario de las ejecuciones oficiales cuyos artefactos respaldan las cifras presentadas en esta memoria: entrenamiento y evaluación principal (MAIN), escalera de validación (comprobaciones A, B y C), análisis estadísticos sobre la partición fija (intervalos de confianza bootstrap y análisis de duplicados), el barrido de la línea base Random Forest sobre las tres particiones consideradas, y la inferencia oficial de la Fase 2 sobre tráfico de laboratorio etiquetado. Fuente: elaboración propia a partir de runs/.

5.6 Pipeline de inferencia offline en la Fase 2

El punto de entrada principal de la Fase 2 mantenido es el script de inferencia offline robusta. Acepta un CSV de flujos extraído, un modelo entrenado, un escalador persistido, límites de percentiles opcionales y recorte por z-score opcional. A continuación mapea las columnas de entrada al esquema canónico, aplica la misma representación de características y máscara, escala con el escalador guardado, calcula diagnósticos, predice acciones en lotes y escribe las salidas en un nuevo directorio de ejecución de la Fase 2.

La Figura 5.4 resume el recorrido operativo de esa inferencia. La clave metodológica es que el escalador de MAIN se reutiliza sin reajuste y que las salidas son ficheros de predicción, métricas y diagnósticos: no existe actuación activa sobre la red.

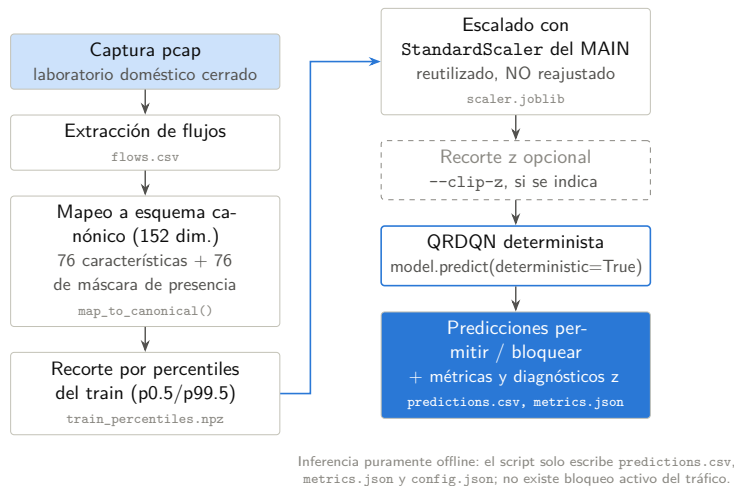


Figura 5.4: Pipeline de inferencia offline de la Fase 2. Fuente: elaboración propia a partir de `scripts/predict_real_traffic_v2.py`.

El Algoritmo 4 formaliza el orden exacto de estas etapas, indicando cuáles son opcionales y bajo qué indicador de línea de comandos se activan.

Algoritmo 4 Inferencia offline robusta de la Fase 2. Fuente: elaboración propia a partir de `scripts/predict_real_traffic_v2.py`.

Require: CSV de flujos D (`-flows`); modelo entrenado M ; escalador persistido S (omitido si `-no-scale`); límites de percentiles opcionales (p_{low}, p_{high}) (`-percentiles`); umbral de recorte z opcional τ_z (`-clip-z`)

Ensure: Directorio de ejecución `runs/phase2/RUN_ID` con predicciones, configuración, métricas y diagnósticos opcionales

```

1: RUN_ID  $\leftarrow$  "P2v2_pred_" + marca de tiempo actual
2: Resolver rutas confiables de  $M$ ,  $S$  y de los percentiles frente al manifiesto de -run-dir, salvo que se indique -allow-unsafe-artifacts
3:  $D \leftarrow$  cargar el CSV de flujos
4: Separar de  $D$  las columnas de metadatos (IP, puertos, protocolo, marca temporal) y las columnas de verdad (truth_y, truth_label)
5: if la mediana de flow_duration en  $D$  es menor que 1 then
6:   Convertir las columnas temporales de segundos a microsegundos ( $\times 10^6$ )
7: end if
8:  $(X, \text{mask}) \leftarrow$  mapear las columnas de  $D$  al esquema canónico de 76 características, imputando con 0 las ausentes
9:  $X \leftarrow [X \mid \text{mask}]$   $\triangleright$  vector combinado de 152 dimensiones
10: if se proporcionaron límites de percentiles (-percentiles) then
11:   Recortar las 76 características crudas de  $X$  a  $[p_{low}, p_{high}]$  (percentiles de entrenamiento p0.5 / p99.5)
12: end if
13: if no se indicó -no-scale then
14:    $X \leftarrow S.\text{transform}(X)$   $\triangleright$  escalador persistido de entrenamiento
15: end if
16: if se proporcionó  $\tau_z$  (-clip-z) then
17:    $X \leftarrow \text{clip}(X, -\tau_z, \tau_z)$   $\triangleright$  recorte tras el escalado
18: end if
19: Calcular diagnósticos de cambio de distribución sobre  $X$ :  $z$  absoluto máximo,  $z$  absoluto medio, proporción con  $|z| > 10$  y características más desviadas
20: if -export-diagnostics then
21:   Guardar los diagnósticos en diagnostics.json
22: end if
23: Cargar  $M$  (QRDQN, con reserva a DQN solo si sb3_contrib no está instalado)
24:  $y_{pred} \leftarrow$  predecir acciones sobre  $X$  en lotes de 4096 filas
25: if  $|y_{pred}| \neq |D|$  then
26:   abortar  $\triangleright$  las predicciones deben alinearse fila a fila con  $D$ 
27: end if
28: Escribir predictions.csv con  $y_{pred}$ ; si -include-sensitive-metadata, escribir también predictions_sensitive_local.csv con metadatos
29: Calcular estadísticas de predicción:  $n_{flows}$ , tasa de bloqueo, tasa de permiso y diagnósticos de  $z$ -score
30: if  $D$  contiene truth_y o truth_label con valores válidos then
31:   Añadir métricas binarias (matriz de confusión y métricas derivadas) comparando  $y_{pred}$  con la verdad
32: end if
33: Escribir config.json y metrics.json en runs/phase2/RUN_ID return Directorio de ejecución runs/phase2/RUN_ID

```

5.6.1 Mapeo canónico para flujos de laboratorio

Los extractores de flujos de laboratorio pueden utilizar nombres de columna diferentes a los de CICIDS2017. El script de la Fase 2 define por tanto un mapeo explícito de los nombres del extractor a los nombres canónicos. Las columnas de metadatos como la IP de origen, la IP de destino, los puertos, el protocolo y la marca temporal pueden preservarse en los ficheros de salida para su inspección, pero no forman parte del vector de características del modelo. Esto mantiene separadas la interpretabilidad y la entrada al modelo.

5.6.2 Diagnósticos de cambio de distribución

La inferencia de la Fase 2 calcula diagnósticos sobre las características canónicas escaladas. Los z-scores absolutos elevados indican que los valores de flujo de laboratorio están lejos de la distribución de entrenamiento de CICIDS2017. El script también puede exportar las características con las mayores desviaciones. Estos diagnósticos no son etiquetas y no prueban la corrección, pero ayudan a explicar los cambios en la tasa de bloqueo e identificar discrepancias entre el tráfico del benchmark y el del laboratorio.

Más allá de los z-scores por característica, la salida de diagnóstico registra la proporción de características canónicas que fueron imputadas a partir de la máscara de presencia/ausencia, la media y la varianza de los valores de características escaladas relativas a las estadísticas de entrenamiento, y cualquier característica que caiga consistentemente fuera de los límites de percentiles observados durante el entrenamiento con CICIDS2017. En conjunto, estos indicadores proporcionan un resumen estructurado de la distancia distribucional entre el tráfico de laboratorio y el benchmark, permitiendo una investigación dirigida en lugar de requerir la inspección manual de los registros brutos de predicción.

5.6.3 Métricas de verdad cuando están disponibles

Si un CSV de flujos de la Fase 2 incluye columnas de verdad como `truth_y` o `truth_label`, el script puede calcular las mismas métricas binarias utilizadas en la evaluación interna. Si no se dispone de la verdad fundamental, el script reporta estadísticas de predicción como la tasa de bloqueo, la tasa de permiso y los diagnósticos de z-score. Esta distinción es importante: una ejecución de laboratorio no etiquetada, solo benigna o mixta, no puede respaldar las mismas afirmaciones que un conjunto de validación externo etiquetado.

5.7 Limitaciones metodológicas

La metodología realiza compromisos deliberados que definen la interpretación correcta de los resultados de la tesis. Esta sección organiza esos compromisos en cuatro categorías.

5.7.1 Entorno estático y toma de decisiones secuencial

La limitación más fundamental es que el entorno de RL es un conjunto de datos estático. Las acciones del agente no alteran las observaciones de flujo futuras, la topología de la red, el comportamiento del atacante ni el estado del sistema. La formulación se asemeja por tanto más a un problema de decisión contextual sensible al coste que a un proceso de decisión de Markov completo (Sutton et al. 2018; Levine et al. 2020). Las dependencias secuenciales, las cadenas de ataque de múltiples etapas, la adaptación del atacante y las contramedidas activas del defensor quedan todas fuera del alcance implementado. Las afirmaciones sobre autonomía secuencial, defensa cibernética adaptativa o aprendizaje por refuerzo en línea no están respaldadas por el pipeline actual.

5.7.2 Mapeo de etiquetas binarias y detalle por familia de ataque

El mapeo de etiquetas binarias colapsa todos los tipos de ataque en una única clase BLOCK. La información sobre la familia de ataque se descarta en la ruta de aprendizaje principal de RL. Este diseño está justificado por el modelo de decisión binario estudiado aquí, pero significa que el agente no puede especializar su calidad de detección para familias de ataque individuales. El análisis de errores por familia requiere etiquetas de ataque preservadas y código de evaluación independiente, y no puede inferirse a partir de las métricas de exactitud binaria reportadas por el pipeline principal.

5.7.3 Fragilidad del benchmark y generalización

CICIDS2017 sigue siendo un benchmark público controlado con problemas documentados: artefactos de extracción, inconsistencias en las etiquetas, flujos duplicados y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022). Incluso con preprocesamiento anti-fuga y protocolos de partición más estrictos, el rendimiento sobre CICIDS2017 no equivale al rendimiento sobre una red de producción. La inferencia de laboratorio en la Fase 2 comprueba el cambio de dominio, pero es inferencia offline sobre una captura privada y no una validación sobre un flujo de producción etiquetado de forma independiente. Una evidencia de generalización más sólida requeriría conjuntos de datos etiquetados externos, múltiples entornos de captura y, en última instancia, un análisis de seguridad para cualquier despliegue activo.

5.7.4 Algoritmo único y alcance de la reproducibilidad

El algoritmo de RL principal es una única configuración de QRDQN. Los resultados de una sola ejecución no pueden cuantificar la varianza entre ejecu-

ciones a menos que se evalúen múltiples semillas y sus resultados se agreguen (Sutton et al. 2018). La línea base supervisada se limita a Random Forest en la comparación principal; otros modelos tabulares se discuten en la revisión de la literatura pero no forman parte del protocolo evaluado. Estas restricciones de alcance son prácticas: completar el pipeline central y la escalera de validación es metodológicamente más informativo que implementar parcialmente muchas comparaciones. Afirmaciones más sólidas en el futuro requerirían artefactos completos de leave-one-CSV-out, métricas de línea base supervisada con el mismo protocolo, múltiples semillas, tráfico de laboratorio etiquetado más rico y, en última instancia, un análisis de seguridad independiente para cualquier despliegue activo.

Resultados

Este capítulo reporta únicamente cifras respaldadas por artefactos de ejecución comprometidos en el repositorio. Cada resultado se identifica mediante su `RUN_ID`, de modo que puede recuperarse e inspeccionarse de forma independiente. Siguiendo la escalera de validación descrita en la metodología, los resultados se presentan desde el peldaño más permisivo (partición aleatoria) hasta los más exigentes (partición por día y leave-one-CSV-out), seguidos de la comparación con la línea base supervisada y de la inferencia de la Fase 2.

6.1 Configuración del modelo principal

El experimento principal y oficial de la tesis es la ejecución `MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655`. Se entrenó un agente QRDQN sobre el conjunto de datos completo de CICIDS2017 con la representación canónica de 152 dimensiones, una partición aleatoria estratificada con semilla 42 (entrenamiento de 2 264 594 filas y prueba de 566 149 filas) y la configuración de recompensa `tp=1.5`, `fp=-2.0`, `fn=-5.0`, `omission=0.0`. Los hiperparámetros (factor de descuento $\gamma = 0$, arquitectura `[1024,1024,512]`, 200 cuantiles, tasa de aprendizaje `5e-5`, 20 pasos de gradiente, 3 000 000 de

pasos de entrenamiento) constituyen un perfil fijado a mano, no el resultado de una búsqueda de Optuna, como se documenta en la metodología.

La Figura 6.1 muestra las curvas registradas durante la ejecución MAIN. Sirven como evidencia descriptiva del proceso de entrenamiento, no como sustituto de la evaluación final sobre la partición de prueba fija.

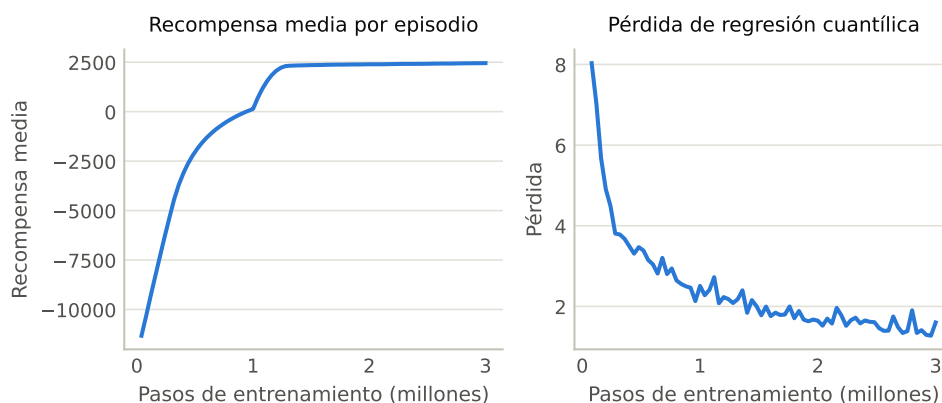


Figura 6.1: Curvas de entrenamiento de la ejecución MAIN. Fuente: elaboración propia a partir de los escalares TensorBoard exportados de MAIN.

6.2 Rendimiento en distribución (partición aleatoria fija)

Sobre la partición aleatoria fija, el modelo MAIN alcanza una exactitud de 0.99381, con un recall de ataque de 0.99536 y una puntuación F1 de ataque de 0.98445 (Tabla 6.1). La matriz de confusión sobre las 566 149 filas de prueba es $TN = 451\,631$, $FP = 2\,989$, $FN = 518$, $TP = 111\,011$. Estas cifras deben interpretarse como evidencia de *capacidad de aprendizaje* en el peldaño más permisivo de la escalera de validación: una partición aleatoria por filas es la evidencia más débil del proyecto, y la Sección 6.5 cuantifica por qué.

La Figura 6.2 desagrega ese resultado en la matriz de confusión completa. La cifra crítica para la interpretación de seguridad es el bajo número de falsos

Métrica	Valor
Exactitud	0.99381
Precisión (ataque)	0.97378
Recall (ataque)	0.99536
F1 (ataque)	0.98445
Precisión (benigno)	0.99885
Recall (benigno)	0.99343
F1 (benigno)	0.99613

Tabla 6.1: Métricas del modelo MAIN sobre la partición aleatoria fija (566 149 filas de prueba). Fuente: `metrics.json` de la ejecución MAIN.

negativos en esta partición aleatoria fija, aunque esta partición sigue siendo el peldaño más permisivo de la evaluación.

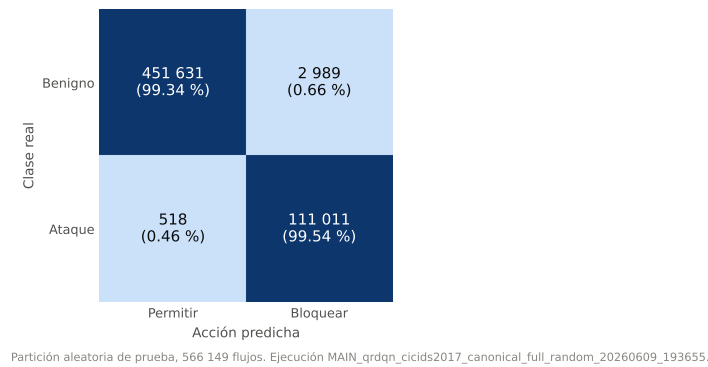


Figura 6.2: Matriz de confusión del modelo MAIN sobre la partición aleatoria fija. Fuente: elaboración propia a partir de `runs/validation/bootstrap_ci_seed42.json`.

6.3 Intervalos de confianza bootstrap

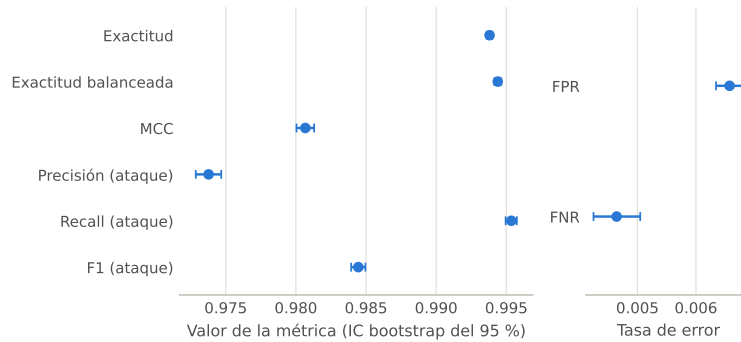
Para cuantificar la precisión de muestreo de estas métricas sobre el conjunto de prueba fijo, se aplicó un bootstrap estratificado de 10 000 remuestros sobre las 566 149 filas de prueba, sin reentrenar el modelo (artefacto `bootstrap_ci_seed42.json`). Los intervalos de confianza al 95 % son estrechos (Tabla 6.2). Es importante delimitar su alcance: estos intervalos describen

la precisión de muestreo del conjunto de prueba para este *único* modelo entrenado; no cuantifican la varianza asociada a la semilla de entrenamiento, que requeriría múltiples ejecuciones y queda fuera del alcance de esta tesis.

Métrica	Valor	IC 95 %
Exactitud	0.99381	[0.99360, 0.99401]
Exactitud balanceada	0.99439	[0.99416, 0.99462]
MCC	0.98068	[0.98004, 0.98130]
Precisión (ataque)	0.97378	[0.97286, 0.97468]
Recall (ataque)	0.99536	[0.99495, 0.99575]
F1 (ataque)	0.98445	[0.98394, 0.98496]
FPR	0.00658	[0.00634, 0.00681]
FNR	0.00465	[0.00425, 0.00505]

Tabla 6.2: Intervalos de confianza bootstrap al 95 % (10 000 re-muestreos estratificados) de las métricas del modelo MAIN. Fuente: `bootstrap_ci_seed42.json`.

La Figura 6.3 representa los mismos intervalos como barras de error. La lectura debe mantenerse acotada: son intervalos de precisión de muestreo sobre un único modelo entrenado, no intervalos de variabilidad entre semillas.



Bootstrap estratificado, 10 000 remuestreos del conjunto de prueba fijo (566 149 flujos): precisión de muestreo, no varianza entre semillas.

Figura 6.3: Intervalos bootstrap al 95 % de las métricas de MAIN. Fuente: elaboración propia a partir de `runs/validation/bootstrap_ci_seed42.json`.

6.4 Escalera de validación: comprobaciones A, B y C

Las tres comprobaciones de `validate_checks.py` cumplen propósitos distintos. La **Comprobación A** (`VAL_checks_A_20260212_235443`) evalúa el modelo mediante predicción directa sobre `X_test`, sin depender de la etiqueta que informa el entorno, y obtiene una exactitud de 0.9939 (TP = 4772, FP = 60, FN = 1, TN = 5167 sobre 10 000 muestras); confirma que las buenas métricas no dependen de un artefacto del bucle del entorno. La **Comprobación B** (`VAL_checks_B_20260212_235736`) reentrena con etiquetas barajadas y obtiene una exactitud de 0.4773, por debajo de la línea base de clase mayoritaria (0.5227), con `leakage_detected = false`: cuando se destruye la relación entre características y etiquetas, el rendimiento no se mantiene artificialmente alto, como es deseable en ausencia de fuga.

La **Comprobación C** (`VAL_checks_C_20260213_004847`) es la evidencia de generalización dura del proyecto. Entrenando en lunes/martes/miércoles y evaluando en jueves/viernes (entrenamiento de 1 668 530 filas, prueba de 1 162 213 filas), la exactitud desciende a 0.84135 y el recall de ataque a 0.52954 (precisión de ataque 0.76479, F1 de ataque 0.62578). La caída frente al rendimiento in-distribution es exactamente el tipo de evidencia que distingue un benchmark cómodo de una evaluación honesta: el pipeline aprende muy bien dentro de la distribución, pero generalizar a días y patrones de ataque distintos es sustancialmente más difícil.

La matriz de confusión de la Comprobación C se muestra en la Figura 6.4. Esta comprobación usó una red proxy [512,256] entrenada durante 30 000 pasos, no los pesos del modelo MAIN; por tanto, se interpreta como peldaño de validación por día y no como evaluación directa del modelo oficial sobre la partición MAIN.

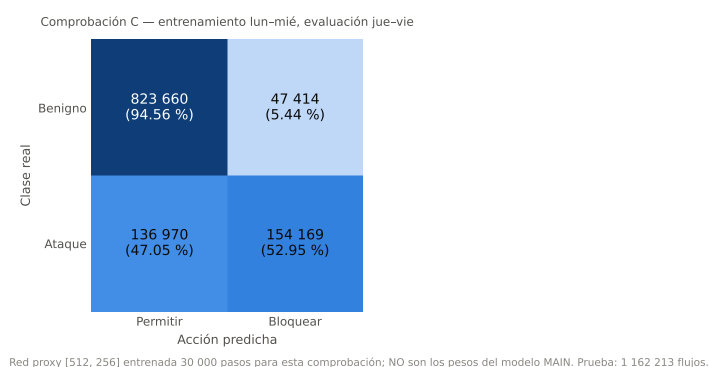


Figura 6.4: Matriz de confusión de la Comprobación C sobre la partición por día. Fuente: elaboración propia a partir de VAL_checks_C_20260213_004847/validation_results.json.

6.5 Análisis de duplicados y fuga en la partición aleatoria

Para explicar por qué la partición aleatoria es el peldaño más débil, se cuantificó la duplicación de filas y la fuga entre particiones sobre la misma partición con semilla 42, sin entrenar ningún modelo (artefacto `duplicate_analysis_seed42.json`). El 22.30 % de las filas del conjunto completo son duplicados exactos a nivel de características. Más relevante para la evaluación: el 24.86 % de las filas de prueba son duplicados exactos de una fila de entrenamiento, proporción que asciende al 40.12 % entre las filas de ataque de prueba (frente al 21.12 % entre las benignas). Esto significa que una fracción considerable del conjunto de prueba aleatorio es, en efecto, memorizable a partir del entrenamiento, lo que infla las métricas in-distribution. Es la razón metodológica por la que la Comprobación C —y no la partición aleatoria— se interpreta como la señal creíble de generalización. Estas cifras se reportan como contexto honesto del peldaño aleatorio, no como una objeción al modelo MAIN, cuyo rendimiento sobre tráfico de laboratorio se examina en la Sección 6.7.

La Figura 6.5 resume visualmente estas cuatro tasas de duplicación y fuga.

La barra de ataques de prueba ya presentes en entrenamiento es la más relevante para la interpretación de recall de ataque en la partición aleatoria.

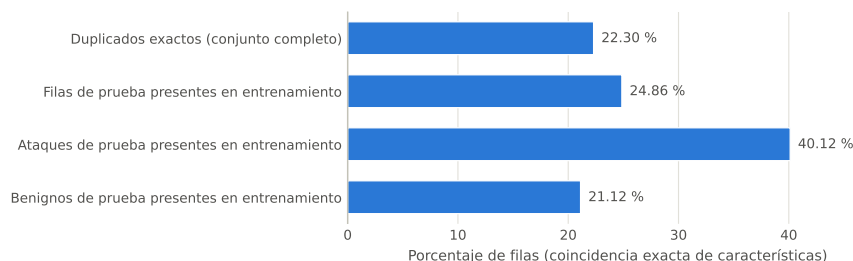


Figura 6.5: Duplicados exactos y fuga entre entrenamiento y prueba en la partición aleatoria fija. Fuente: elaboración propia a partir de `runs/validation/duplicate_analysis_seed42.json`.

6.6 Línea base de Random Forest bajo el mismo protocolo

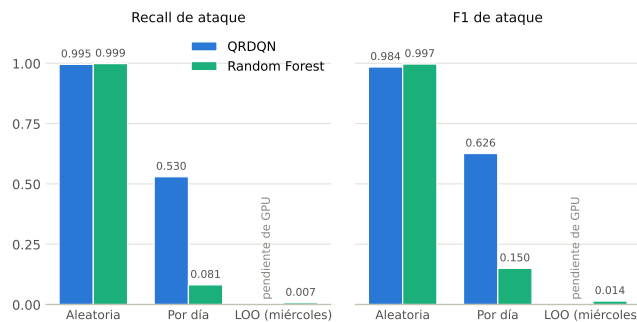
La línea base de Random Forest se ejecutó bajo el mismo protocolo que QRDQN (representación canónica, escalado, particiones equivalentes y ponderación de clases balanceada), persistiendo `config.json` y `metrics.json` por barrido (RUN_ID `rf_cicids2017_canonical_20260628_024735`). Sobre la partición aleatoria fija —el mismo conjunto de prueba de 566 149 filas que MAIN— el Random Forest alcanza una exactitud de 0.99872 y una F1 de ataque de 0.99676, ligeramente por encima de QRDQN. Bajo la partición por día equivalente a la Comprobación C (entrenamiento lunes/martes/miércoles, prueba jueves/viernes), su recall de ataque colapsa a 0.08135 (F1 de ataque 0.15005, exactitud 0.76913), y bajo el leave-one-CSV-out con miércoles reservado el recall de ataque cae a 0.00719. La Tabla 6.3 resume la comparación con el mismo protocolo.

La Figura 6.6 hace visible la diferencia principal de esta comparación: Random Forest supera a QRDQN en la partición aleatoria, pero su recall de

Modelo	Partición	Exactitud	Prec. ataque	Recall ataque	F1 ataque
QRDQN (MAIN)	Aleatoria fija	0.99381	0.97378	0.99536	0.98445
Random rest	Fo- Aleatoria fija	0.99872	0.99501	0.99853	0.99676
QRDQN (Check C)	Día (Lun–Mié → Jue–Vie)	0.84135	0.76479	0.52954	0.62578
Random rest	Fo- Día (Lun–Mié → Jue–Vie)	0.76913	0.96473	0.08135	0.15005

Tabla 6.3: Comparación con el mismo protocolo entre QRDQN y la línea base de Random Forest balanceada. Las filas aleatorias comparten el conjunto de prueba de 566 149 filas con semilla 42; las filas por día comparten la partición lunes/martes/miércoles → jueves/viernes (1 162 213 filas de prueba). Fuentes: ejecuciones MAIN, VAL_checks_C_20260213_004847 y rf_cicids2017_canonical_20260628_024735.

ataque se desploma en la partición por día. El hueco leave-one-CSV-out de QRDQN se mantiene como pendiente de GPU, sin barra fabricada.



QRDQN «Por día» proviene de la Comprobación C (red proxy [512, 256], 30 000 pasos), no de los pesos MAIN. QRDQN en LOO: diseñado, ejecución pendiente de GPU.

Figura 6.6: Comparación visual de QRDQN y Random Forest en recall y F1 de ataque bajo particiones equivalentes. Fuente: elaboración propia a partir de MAIN, VAL_checks_C_20260213_004847 y rf_cicids2017_canonical_20260628_024735.

La matriz de confusión de Random Forest en la partición por día (Figura 6.7) explica el bajo recall de ataque: el modelo conserva muy pocos falsos positivos, pero omite una gran mayoría de los ataques en jueves/viernes.

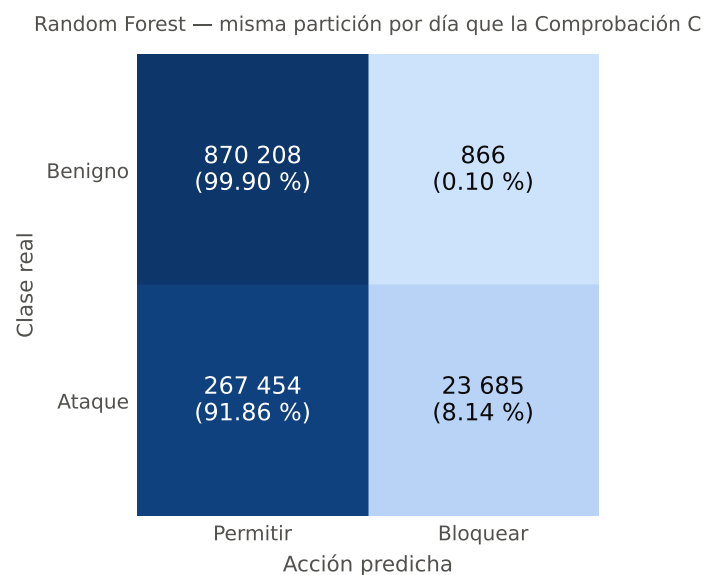


Figura 6.7: Matriz de confusión de Random Forest en la partición por día. Fuente: elaboración propia a partir de `rf_cicids2017_canonical_20260628_024735__day_split/metrics.json`.

6.7 Inferencia de la Fase 2 sobre tráfico de laboratorio

La inferencia offline de la Fase 2 aplicó el modelo MAIN, junto con su escalador y percentiles persistidos, sobre tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial (archivo `lab_capture_traffic.csv`, 2 000 000 de flujos etiquetados; artefacto `P2v2_pred_20260610_161231_MAIN`). El modelo alcanza una exactitud de 0.991862, con una precisión de ataque de 0.97919, un recall de ataque de 0.988452 y una F1 de ataque de 0.98380 ($TP = 494\,226$, $TN = 1\,489\,498$, $FP = 10\,502$, $FN = 5\,774$), con una tasa de bloqueo de 0.252364. Los diagnósticos de distribución son benignos: el valor absoluto máximo de z observado es 9.547 y el medio 0.3417, sin ninguna característica que supere el recorte en $|z| = 10$. El modelo se comporta, por tanto, de forma razonable sobre esta captura de laboratorio concreta. No obstante, dado que se trata de tráfico generado por el operador en un único entorno

cerrado y no adversarial, este resultado es una comprobación controlada de validez externa limitada y no constituye evidencia de rendimiento de detección en el mundo real.

La Figura 6.8 resume la matriz de confusión y la tasa de bloqueo de esta ejecución. Su función es documentar el comportamiento del pipeline sobre esta captura concreta, manteniendo explícita su validez externa limitada.

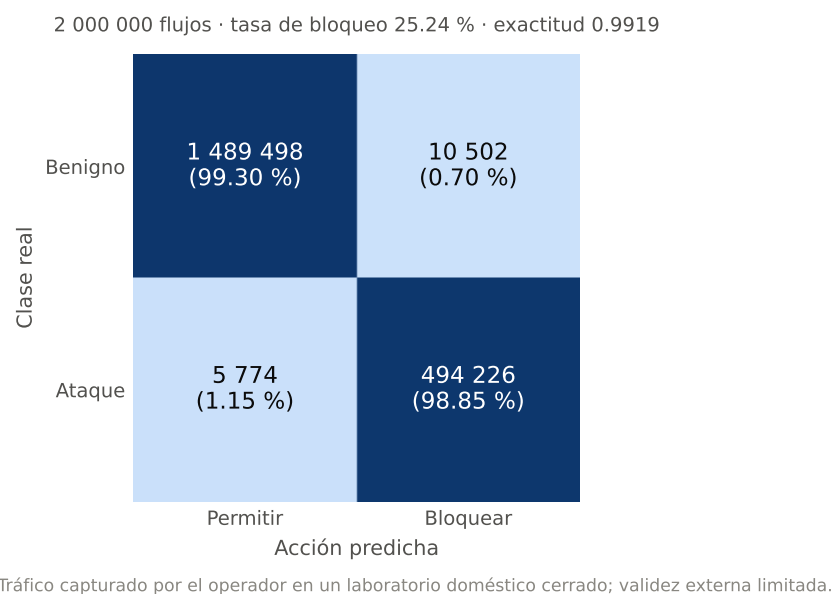


Figura 6.8: Resultados de la inferencia de la Fase 2 sobre tráfico de laboratorio etiquetado. Fuente: elaboración propia a partir de `runs/phase2/P2v2_pred_20260610_161231_MAIN/metrics.json`.

Discusión

Este capítulo interpreta los resultados del Capítulo anterior sin introducir ninguna cifra nueva. El objetivo es leer la evidencia de forma honesta: qué demuestra el pipeline, qué no, y qué aporta realmente la formulación de aprendizaje por refuerzo frente a una línea base supervisada sólida.

7.1 In-distribution frente a generalización dura

El contraste más informativo del trabajo no es la cifra in-distribution, sino su distancia respecto a la evaluación por días. Sobre la partición aleatoria fija, el modelo MAIN obtiene una exactitud de 0.99381; bajo la partición por día (Comprobación C) desciende a 0.84135 y el recall de ataque cae de 0.99536 a 0.52954. Esta brecha no debe leerse como un fallo, sino como la medida honesta del problema: la partición aleatoria comparte distribución entre entrenamiento y prueba y, como cuantifica el análisis de duplicados (24.86 % de las filas de prueba duplican una fila de entrenamiento, y hasta el 40.12 % entre los ataques de prueba), una parte considerable de su rendimiento es memorización. La partición por día rompe esa comodidad y revela la verdadera dificultad de generalizar a patrones temporales y familias de ataque distintos.

Reportar ambos peldaños, en lugar de solo el más favorable, es la contribución metodológica central de esta tesis.

7.2 QRDQN frente a la línea base supervisada

La comparación con el mismo protocolo permite una conclusión matizada y deliberadamente poco triunfalista. *En distribución*, la línea base de Random Forest balanceada iguala —e incluso supera ligeramente— a QRDQN (F1 de ataque 0.99676 frente a 0.98445). Por tanto, esta tesis no afirma que el aprendizaje por refuerzo sea superior sobre el peldaño aleatorio; sobre datos tabulares de flujo en distribución, un Random Forest bien configurado es un competidor muy fuerte, coherente con la literatura (H. Liu et al. 2019; Apruzzese et al. 2022).

La diferencia aparece bajo el cambio de distribución. Sobre la misma partición por día (lunes/martes/miércoles $\rightarrow$ jueves/viernes), ambos modelos se degradan, pero de forma muy distinta: el Random Forest conserva una precisión de ataque alta (0.96473) a costa de un recall de ataque que se desploma a 0.08135 —es decir, deja pasar más del 91 % de los ataques—, mientras que QRDQN mantiene un recall de ataque de 0.52954. En términos de seguridad operativa, donde un falso negativo es el error caro, el agente sensible al coste retiene una fracción mucho mayor de la capacidad de detección bajo condiciones adversas. Esta divergencia es atribuible a la asimetría de la recompensa ($f_n = -5.0$ frente a $f_p = -2.0$), que empuja a la política hacia el bloqueo de ataques inciertos en lugar de hacia la precisión agregada. La lectura honesta no es «RL gana», sino que *la formulación sensible al coste degrada de forma más favorable en la métrica crítica de seguridad bajo cambio de dominio*, aun cuando ambos enfoques quedan muy por debajo de su rendimiento in-distribution y ninguno está listo para el despliegue.

7.3 Comportamiento en la Fase 2 y cambio de dominio

La inferencia de la Fase 2 ofrece una tercera perspectiva. Sobre el tráfico de laboratorio generado por el operador, el modelo MAIN alcanza una exactitud de 0.991862 con diagnósticos de distribución benignos (z absoluto máximo 9.547, medio 0.3417, sin recortes activados). Esto indica que la captura de laboratorio se sitúa cerca de la distribución de entrenamiento y que el pipeline de preprocesamiento robusto —recorte por percentiles y por z-score sobre el escalador persistido— transfiere correctamente. Sin embargo, este resultado debe interpretarse con cautela: a diferencia de la partición por día, que somete al modelo a un cambio de distribución *adverso* dentro de CICIDS2017, la captura de laboratorio es tráfico propio, no adversarial y de un único entorno cerrado. Un buen resultado en la Fase 2 es, por tanto, una comprobación de validez externa limitada y no una prueba de robustez frente al cambio de dominio en el mundo real; las dos señales de generalización —Comprobación C y Fase 2— son complementarias y no deben confundirse.

Limitaciones

El capítulo del protocolo experimental ya expuso las limitaciones *conceptuales* del enfoque: el entorno estático y la formulación de bandido contextual sensible al coste, el mapeo de etiquetas binarias, la fragilidad del benchmark y el alcance de un único algoritmo. Este capítulo las complementa a la luz de los resultados ya medidos, de modo que cada limitación quede anclada en evidencia concreta y no en una declaración general.

8.1 Modelo y semilla únicos

Los resultados corresponden a un único modelo entrenado con una sola semilla. Los intervalos de confianza bootstrap reportados (± 0.0002 a ± 0.001 al 95 %) cuantifican la precisión de muestreo del conjunto de prueba fijo, no la varianza entre ejecuciones de entrenamiento: caracterizan cuán estable es la métrica sobre remuestreos del mismo conjunto de prueba para *este* modelo, pero no cuánto variaría el rendimiento al reentrenar con semillas distintas. Una estimación de la varianza entre semillas requeriría múltiples ejecuciones de entrenamiento completas, que no se han realizado. Las cifras deben leerse, por tanto, como el comportamiento de un punto de configuración bien caracterizado,

no como una media sobre la aleatoriedad del entrenamiento.

8.2 Fuga en la partición aleatoria

El análisis de duplicados muestra que el 22.30 % de las filas del conjunto completo son duplicados exactos y que el 24.86 % de las filas de prueba de la partición aleatoria duplican una fila de entrenamiento (40.12 % entre los ataques de prueba). En consecuencia, las métricas in-distribution sobreestiman la capacidad de generalización y no deben presentarse como el resultado principal en sentido operativo. Esta limitación se mitiga metodológicamente reportando la Comprobación C (partición por día) como la señal de generalización creíble, pero no se elimina: la cifra aleatoria sigue siendo el peldaño más débil del proyecto y se conserva únicamente como evidencia de capacidad de aprendizaje.

8.3 Comparación con la línea base

La comparación con Random Forest se ejecutó bajo el mismo protocolo (representación canónica, escalado, ponderación de clases balanceada y particiones equivalentes), con artefactos de configuración y métricas comprometidos. Persisten, no obstante, dos limitaciones. En primer lugar, la comparación abarca un único modelo supervisado; otros clasificadores tabulares competitivos, como los ensamblados con gradient boosting, se discuten en la revisión de la literatura pero no forman parte del protocolo evaluado. En segundo lugar, el barrido leave-one-CSV-out de la línea base (miércoles reservado, recall de ataque 0.00719) no tiene todavía un artefacto QRDQN comprometido equivalente, por lo que en ese peldaño la comparación es solo de la línea base consigo misma y no entre modelos. La comparación entre QRDQN y Random Forest es directa y del mismo protocolo en la partición aleatoria fija y en la partición por día,

pero no en el leave-one-CSV-out.

8.4 Validez externa de la Fase 2

El alto rendimiento de la Fase 2 (0.991862 de exactitud) se obtiene sobre tráfico generado por el operador en un laboratorio doméstico cerrado y no adversarial, de un único entorno y con etiquetas de intención del generador. No es tráfico de producción, no contiene un adversario real y no procede de una fuente independiente. Su validez externa es, por tanto, limitada: un buen resultado en la Fase 2 es una comprobación controlada de que el contrato de características y el preprocesamiento robusto transfieren a una captura cercana a la distribución, no una prueba de detección en el mundo real (Sommer et al. 2010; Layeghy et al. 2023).

8.5 Experimentos planificados pero no ejecutados

Por honestidad metodológica, se enumeran explícitamente los análisis previstos en el diseño cuyos artefactos no se han comprometido. El protocolo de curva de aprendizaje (escala de entrenamiento a distintos presupuestos) está planificado pero no ejecutado: no existe ningún artefacto de curva de aprendizaje. La validación leave-one-CSV-out está implementada en el código para QRDQN, pero no se ha comprometido un artefacto agregado de sus métricas. La evaluación con múltiples semillas para estimar la varianza entre ejecuciones no se ha llevado a cabo. Ninguna de estas tres capacidades se reporta con cifras en esta tesis, y su ausencia se declara aquí en lugar de insinuarse como completada.

8.6 Trabajo futuro

Las limitaciones anteriores definen una agenda de trabajo futuro coherente: ejecutar y comprometer la curva de aprendizaje y el leave-one-CSV-out de QRDQN; reentrenar con múltiples semillas para cuantificar la varianza; ampliar la comparación a otros modelos supervisados; y, sobre todo, validar sobre tráfico etiquetado independiente de múltiples entornos antes de considerar cualquier paso hacia un despliegue activo, que a su vez exigiría un análisis de seguridad específico. Estas extensiones refuerzan la evidencia sin alterar la arquitectura experimental ya establecida.

Consideraciones éticas y legales

Las herramientas de detección de intrusiones operan sobre tráfico de red que puede contener información sensible, y un sistema que decide entre permitir y bloquear conlleva consecuencias operativas directas. Aunque esta tesis se limita a un prototipo experimental de inferencia offline, las decisiones de diseño que la hacen reproducible y segura tienen también una dimensión ética y legal que conviene hacer explícita.

9.1 Uso dual y alcance estrictamente defensivo

La investigación en ciberseguridad tiene una naturaleza intrínsecamente dual: las mismas técnicas que permiten detectar tráfico malicioso pueden, en principio, informar su evasión. El trabajo aquí descrito es defensivo por construcción y, además, deliberadamente inerte respecto a cualquier red real. Las salidas **PERMIT** y **BLOCK** son clasificaciones experimentales sobre registros de flujo estáticos: el sistema no realiza bloqueo en vivo, descarte de paquetes ni manipulación de **iptables**, y no se integra con cortafuegos, controladores de redes definidas por software (SDN) ni sistemas de prevención de intrusiones (IPS), tal como se delimita en los no objetivos del Capítulo 2. Esta ausencia de

actuación en línea elimina el riesgo de que el artefacto, por sí mismo, provoque un bloqueo indebido de tráfico legítimo en una red en producción. El sistema es un instrumento de medición experimental, no un mecanismo de defensa desplegable.

9.2 Privacidad y tratamiento de los datos

El sistema opera exclusivamente sobre características estadísticas a nivel de flujo y no sobre el contenido (*payload*) de los paquetes, evitando la inspección profunda de paquetes y buena parte de sus implicaciones legales y de privacidad. El contrato canónico de características refuerza esta posición al excluir a nivel de esquema los identificadores potencialmente sensibles o reidentificadores —direcciones IP de origen y destino, marcas temporales absolutas, Flow ID y números de puerto—, que se descartan antes de cualquier entrenamiento. El benchmark CICIDS2017 es un conjunto de datos público publicado por el Canadian Institute for Cybersecurity (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017). El tráfico empleado en la Fase 2 fue generado por el propio operador en un laboratorio doméstico cerrado y no adversarial: se trata de tráfico propio, no contiene datos personales de terceros y no fue capturado de redes ajenas, por lo que su uso no plantea problemas de consentimiento ni de exposición de información personal.

9.3 Riesgo operativo y asimetría de errores

Las consecuencias de los dos tipos de error son profundamente asimétricas. Un falso negativo —un ataque clasificado como benigno y permitido— puede habilitar exfiltración de datos o interrupción del servicio; un falso positivo —tráfico benigno bloqueado— interrumpe comunicaciones legítimas y consume

tiempo de análisis. El propio diseño de la función de recompensa codifica esta asimetría penalizando los falsos negativos con mayor severidad. Esta tesis reporta estos riesgos de forma honesta en lugar de ocultarlos tras la exactitud agregada: sobre la partición aleatoria fija *in-distribution*, el modelo oficial MAIN presenta una tasa de falsos positivos de 0.00658 y una tasa de falsos negativos de 0.00465; sin embargo, bajo la partición por días, sustancialmente más exigente (Check C), el recall de ataque desciende hasta 0.52954. Esta caída del rendimiento ante un cambio de condiciones es precisamente la razón por la que el sistema se presenta como un prototipo de investigación y no como una herramienta lista para el despliegue: un uso operativo responsable exigiría validación sobre tráfico de producción etiquetado, múltiples entornos de captura y un análisis de seguridad independiente antes de delegar en el modelo cualquier decisión con efectos reales (Apruzzese et al. 2022; Sommer et al. 2010).

9.4 Reproducibilidad como práctica responsable

Por último, la disciplina de reproducibilidad descrita en la metodología es también una práctica ética: la investigación en seguridad con aprendizaje automático adolece con frecuencia de código ausente y evaluaciones irreproducibles, lo que dificulta verificar o refutar las afirmaciones publicadas (Olszewski et al. 2023; Arp et al. 2020). La persistencia de artefactos por ejecución, la partición de prueba fija con semilla 42 y su manifiesto de referencia permiten que cualquier tercero audite exactamente bajo qué condiciones se obtuvo cada cifra reportada en esta tesis. Hacer las limitaciones verificables, en lugar de presentarlas como una declaración de buenas intenciones, es la forma de honestidad metodológica que este trabajo persigue.

Bibliografía

- [1] Amrin Maria Khan Adawadkar y Nilima Kulkarni. «Cyber-Security and Reinforcement Learning: A Brief Survey». En: *Engineering Applications of Artificial Intelligence* 114 (2022), pág. 105116. DOI: [10.1016/j.engappai.2022.105116](https://doi.org/10.1016/j.engappai.2022.105116) (vid. pág. 38).
- [2] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah y Farhan Ahmad. «Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches». En: *Transactions on Emerging Telecommunications Technologies* 32.1 (2021), e4150. DOI: [10.1002/ett.4150](https://doi.org/10.1002/ett.4150) (vid. pág. 31).
- [3] Hooman Alavizadeh, Julian Jang-Jaccard y Hootan Alavizadeh. «Deep Q-Learning Based Reinforcement Learning Approach for Network Intrusion Detection». En: (2021). arXiv: [2111.13978 \[cs.CR\]](https://arxiv.org/abs/2111.13978) (vid. pág. 39).
- [4] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood y Adnan Anwar. «TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems». En: *IEEE Access* 8 (2020), págs. 165130-165150. DOI: [10.1109/ACCESS.2020.3022862](https://doi.org/10.1109/ACCESS.2020.3022862) (vid. pág. 28).
- [5] Giovanni Apruzzese, Luca Pajola y Mauro Conti. «The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems». En:

- IEEE Transactions on Network and Service Management* 19.4 (2022), págs. 5152-5169. DOI: [10.1109/TNSM.2022.3157344](https://doi.org/10.1109/TNSM.2022.3157344) (vid. págs. 4, 13, 16, 27, 32, 46, 79, 83, 109, 117).
- [6] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro y Konrad Rieck. «Dos and Don'ts of Machine Learning in Computer Security». En: (2020). arXiv: [2010.09470 \[cs.CR\]](https://arxiv.org/abs/2010.09470) (vid. págs. 3, 14, 26, 33, 42, 50, 55, 81, 88, 117).
- [7] Momand Asadullah, Jan Sana Ullah y Naeem Ramzan. «A Systematic and Comprehensive Survey of Recent Advances in Intrusion Detection Systems Using Machine Learning: Deep Learning, Datasets, and Attack Taxonomy». En: *Expert Systems* (2023). DOI: [10.1155/2023/6048087](https://doi.org/10.1155/2023/6048087) (vid. pág. 33).
- [8] Stefan Axelsson. «The Base-Rate Fallacy and the Difficulty of Intrusion Detection». En: *ACM Transactions on Information and System Security* 3.3 (2000), págs. 186-205. DOI: [10.1145/357802.357804](https://doi.org/10.1145/357802.357804) (vid. págs. 2, 43, 45).
- [9] Marc G. Bellemare, Will Dabney y Rémi Munos. «A Distributional Perspective on Reinforcement Learning». En: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. 2017, págs. 449-458. URL: <https://proceedings.mlr.press/v70/bellemare17a.html> (vid. págs. 37, 67, 69).
- [10] Guillermo Caminero, Manuel Lopez-Martin y Belen Carro. «Adversarial Environment Reinforcement Learning Algorithm for Intrusion Detection». En: *2019 International Joint Conference on Neural Networks (IJCNN)*. 2019, págs. 1-8. DOI: [10.1109/IJCNN.2019.8852380](https://doi.org/10.1109/IJCNN.2019.8852380) (vid. pág. 39).
- [11] Canadian Institute for Cybersecurity. *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. Dataset documentation page. 2017. URL: [https:](https://www.cicids.org/)

- [//www.unb.ca/cic/datasets/ids-2017.html](http://www.unb.ca/cic/datasets/ids-2017.html) (vid. págs. 5, 25, 29, 77, 116).
- [12] Alvaro A. Cárdenas, John S. Baras y Karl Seamon. «A Framework for the Evaluation of Intrusion Detection Systems». En: *2006 IEEE Symposium on Security and Privacy (S&P)*. 2006, págs. 63-77. DOI: [10.1109/SP.2006.2](https://doi.org/10.1109/SP.2006.2) (vid. págs. 5, 12, 40, 45, 62).
- [13] Center for Security and Emerging Technology y The Alan Turing Institute. *Autonomous Cyber Defense*. <https://cset.georgetown.edu/publication/autonomous-cyber-defense/>. 2023 (vid. pág. 39).
- [14] Communications Security Establishment y Canadian Institute for Cybersecurity. *A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)*. 2018. URL: <https://registry.opendata.aws/cse-cic-ids2018/> (vid. pág. 28).
- [15] Will Dabney, Mark Rowland, Marc G. Bellemare y Rémi Munos. «Distributional Reinforcement Learning with Quantile Regression». En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, págs. 2892-2901. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11791> (vid. págs. 2, 5, 12, 37, 67, 68).
- [16] Davide Di Monda et al. «Few-Shot Class-Incremental Learning for Network Intrusion Detection Systems». En: (2024). Exact venue and DOI require verification; see arXiv preprint search for Di Monda 2024 (vid. pág. 46).
- [17] Rohit Dube. «Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research». En: *Journal of Computer Virology and Hacking Techniques* 20.1 (2024), págs. 203-211. DOI: [10.1007/s11416-023-00509-7](https://doi.org/10.1007/s11416-023-00509-7) (vid. págs. 30, 42).
- [18] Gints Engelen, Vera Rimmer y Wouter Joosen. «Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study». En: *2021*

- IEEE Security and Privacy Workshops (SPW)*. 2021, págs. 7-12. DOI: [10.1109/SPW53761.2021.00009](https://doi.org/10.1109/SPW53761.2021.00009) (vid. págs. 3, 5, 28, 30, 42, 50, 78, 96).
- [19] Farama Foundation. *Gymnasium Documentation*. Software documentation; access date 2026. 2023. URL: <https://gymnasium.farama.org/> (vid. págs. 5, 12, 37, 59, 75).
- [20] Tom Fawcett. «An Introduction to ROC Analysis». En: *Pattern Recognition Letters* 27.8 (2006), págs. 861-874. DOI: [10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010) (vid. págs. 45, 87).
- [21] Scott Fujimoto, David Meger y Doina Precup. «Off-Policy Deep Reinforcement Learning without Exploration». En: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. 2019, págs. 2052-2062. URL: <https://proceedings.mlr.press/v97/fujimoto19a.html> (vid. pág. 36).
- [22] Afrah Gueriani, Hamza Kheddar y Ahmed Cherif Mazari. «Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey». En: (2024). Preprint; published version in IC2EM proceedings 2023. arXiv: [2405.20038](https://arxiv.org/abs/2405.20038) [cs.CR] (vid. págs. 21, 38).
- [23] Arash Habibi Lashkari, Gerard Draper Gil, Mohammad Saiful Islam Mamun y Ali A. Ghorbani. «Characterization of Tor Traffic Using Time Based Features». En: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, págs. 253-262. DOI: [10.5220/0006105602530262](https://doi.org/10.5220/0006105602530262) (vid. págs. 25, 29, 77).
- [24] Dongkun Han, Zhiping Wang, Yihua Zhong, Wenhan Chen, Jiaze Yang, Shanqing Lu, Xinyan Shi y Xia Yin. «Evaluating Network Intrusion Detection Systems for Different Vulnerability Conditions Using an Adversarial Environment DQN». En: *IEEE Access* 9 (2020). IEEE Xplore document ID 9289884; full author list requires verification, págs. 4345-4359. DOI: [10.1109/ACCESS.2020.3047430](https://doi.org/10.1109/ACCESS.2020.3047430) (vid. pág. 39).

- [25] Hado van Hasselt, Arthur Guez y David Silver. «Deep Reinforcement Learning with Double Q-Learning». En: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. 2016, págs. 2094-2100. arXiv: [1509.06461](https://arxiv.org/abs/1509.06461) (vid. pág. 36).
- [26] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar y David Silver. «Rainbow: Combining Improvements in Deep Reinforcement Learning». En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, págs. 3215-3222. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11796> (vid. págs. 37, 46).
- [27] Md. Alamgir Hossain et al. «Deep Q-Learning Intrusion Detection System (DQ-IDS): A Novel Reinforcement Learning Approach for Adaptive and Self-Learning Cybersecurity». En: *ICT Express* (2025). Full author list requires verification; accessed via ScienceDirect. DOI: [10.1016/j.icte.2025.02.006](https://doi.org/10.1016/j.icte.2025.02.006) (vid. pág. 39).
- [28] Chia-Wei Hsu, Chia-Hsuan Lin et al. «A Soft Actor-Critic Reinforcement Learning Algorithm for Network Intrusion Detection». En: *Computers & Security* 136 (2023). Full author list requires verification against published paper, pág. 103580. DOI: [10.1016/j.cose.2023.103580](https://doi.org/10.1016/j.cose.2023.103580) (vid. pág. 39).
- [29] Zhisheng Hu, Minghui Zhu y Peng Liu. «Adaptive Cyber Defense Against Multi-Stage Attacks Using Learning-Based POMDP». En: *ACM Transactions on Privacy and Security* 24.1 (2020), págs. 1-31. DOI: [10.1145/3418897](https://doi.org/10.1145/3418897) (vid. pág. 39).
- [30] Shachar Kaufman, Saharon Rosset y Claudia Perlich. «Leakage in Data Mining: Formulation, Detection, and Avoidance». En: *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, págs. 556-563. DOI: [10.1145/2020408.2020496](https://doi.org/10.1145/2020408.2020496) (vid. págs. 3, 14, 26, 42, 55).

- [31] KDD Cup. *KDD Cup 1999 Data*. Dataset page. 1999. URL: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html> (vid. págs. 27, 28).
- [32] Hamza Kheddar, Diana W. Dawoud, Ali Ismail Awad, Yassine Himeur y Muhammad Khurram Khan. «Reinforcement-Learning-Based Intrusion Detection in Communication Networks: A Review». En: *IEEE Communications Surveys & Tutorials* 27.4 (2025), págs. 2420-2469. DOI: [10.1109/COMST.2024.3484491](https://doi.org/10.1109/COMST.2024.3484491) (vid. págs. 21, 38, 46, 48).
- [33] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova y Benjamin Turnbull. «Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset». En: *Future Generation Computer Systems* 100 (2019), págs. 779-796. DOI: [10.1016/j.future.2019.05.041](https://doi.org/10.1016/j.future.2019.05.041) (vid. pág. 28).
- [34] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé y Éric Totel. «Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes». En: *Risks and Security of Internet and Systems*. Vol. 13857. Lecture Notes in Computer Science. Springer, 2023, págs. 19-34. DOI: [10.1007/978-3-031-31108-6_2](https://doi.org/10.1007/978-3-031-31108-6_2) (vid. págs. 3, 6, 28, 30, 42, 50, 78, 81, 96).
- [35] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé y Éric Totel. «Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research». En: *Risks and Security of Internet and Systems*. Vol. 13561. Lecture Notes in Computer Science. Earlier presentation of dataset-critique work; see also Lanvin2023Errors. Springer, 2022 (vid. pág. 30).
- [36] Siamak Layeghy y Marius Portmann. «Explainable Cross-Domain Evaluation of ML-Based Network Intrusion Detection Systems». En: *Computers and Electrical Engineering* 108 (2023), pág. 108692. DOI: [10.1016/j.compeleceng.2023.108692](https://doi.org/10.1016/j.compeleceng.2023.108692) (vid. págs. 16, 27, 79, 113).

- [37] Wenke Lee, Wei Fan, Matthew Miller, Salvatore J. Stolfo y Erez Zadok. «Toward Cost-Sensitive Modeling for Intrusion Detection and Response». En: *Journal of Computer Security* 10.1–2 (2002), págs. 5-22. DOI: [10.3233/JCS-2002-101-202](https://doi.org/10.3233/JCS-2002-101-202) (vid. págs. 2, 12, 40, 62).
- [38] Sergey Levine, Aviral Kumar, George Tucker y Justin Fu. «Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems». En: *arXiv preprint arXiv:2005.01643* (2020). Preprint. arXiv: [2005.01643 \[cs.LG\]](https://arxiv.org/abs/2005.01643) (vid. págs. 4, 36, 67, 95).
- [39] Long-Ji Lin. «Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching». En: *Machine Learning* 8.3–4 (1992), págs. 293-321. DOI: [10.1007/BF00992699](https://doi.org/10.1007/BF00992699) (vid. pág. 35).
- [40] Hongyu Liu y Bo Lang. «Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey». En: *Applied Sciences* 9.20 (2019), pág. 4396. DOI: [10.3390/app9204396](https://doi.org/10.3390/app9204396) (vid. págs. 4, 13, 31-33, 48, 83, 109).
- [41] Lisa Liu, Gints Engelen, Timothy M. Lynar, Daryl Essam y Wouter Joosen. «Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018». En: *2022 IEEE Conference on Communications and Network Security (CNS)*. 2022, págs. 254-262. DOI: [10.1109/CNS56114.2022.9947235](https://doi.org/10.1109/CNS56114.2022.9947235) (vid. págs. 28, 30, 42, 78, 96).
- [42] Manuel Lopez-Martin, Belén Carro y Antonio Sanchez-Esguevillas. «Application of Deep Reinforcement Learning to Intrusion Detection for Supervised Problems». En: *Expert Systems with Applications* 141 (2020), pág. 112963. DOI: [10.1016/j.eswa.2019.112963](https://doi.org/10.1016/j.eswa.2019.112963) (vid. pág. 38).
- [43] Manuel Lopez-Martin, Antonio Sanchez-Esguevillas, Juan I. Arribas y Belén Carro. «Network Intrusion Detection Based on Extended RBF Neural Network With Offline Reinforcement Learning». En: *IEEE Access* 9 (2021), págs. 153153-153170. DOI: [10.1109/ACCESS.2021.3127689](https://doi.org/10.1109/ACCESS.2021.3127689) (vid. pág. 38).

- [44] Aleksandar Milenkoski, Marco Vieira, Samuel Kounev, Alberto Avritzer y Bryan D. Payne. «Evaluating Computer Intrusion Detection Systems: A Survey of Common Practices». En: *ACM Computing Surveys* 48.1 (2015), 12:1-12:41. DOI: [10.1145/2808691](https://doi.org/10.1145/2808691) (vid. pág. 45).
- [45] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg y Demis Hassabis. «Human-Level Control through Deep Reinforcement Learning». En: *Nature* 518.7540 (2015), págs. 529-533. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236) (vid. págs. 36, 67, 70).
- [46] Nour Moustafa, Mohiuddin Ahmed y Sherif Ahmed. «Data Analytics-Enabled Intrusion Detection: Evaluations of ToN_IoT Linux Datasets». En: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 2020, págs. 727-735. DOI: [10.1109/TrustCom50675.2020.00080](https://doi.org/10.1109/TrustCom50675.2020.00080) (vid. pág. 28).
- [47] Nour Moustafa y Jill Slay. «The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set». En: *Information Security Journal: A Global Perspective* 25.1-3 (2016), págs. 18-31. DOI: [10.1080/19393555.2015.1125974](https://doi.org/10.1080/19393555.2015.1125974) (vid. pág. 27).
- [48] Nour Moustafa y Jill Slay. «UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)». En: *2015 Military Communications and Information Systems Conference (MilCIS)*. 2015, págs. 1-6. DOI: [10.1109/MilCIS.2015.7348942](https://doi.org/10.1109/MilCIS.2015.7348942) (vid. págs. 27, 28).
- [49] Loc Gia Nguyen y Kohei Watabe. «Flow-Based Network Intrusion Detection Based on BERT Masked Language Model». En: *CoNEXT 2022*

- Student Workshop*. 2022. DOI: [10.1145/3565477.3569152](https://doi.org/10.1145/3565477.3569152). arXiv: [2306.04920](https://arxiv.org/abs/2306.04920) (vid. pág. 33).
- [50] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler y Patrick Traynor. «“Get in Researchers; We’re Measuring Reproducibility”: A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences». En: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, págs. 3433-3459. DOI: [10.1145/3576915.3623130](https://doi.org/10.1145/3576915.3623130) (vid. págs. 45, 86, 117).
- [51] Gregory Palmer, Chris Parry, Daniel J. B. Harrold y Chris Willis. «Deep Reinforcement Learning for Autonomous Cyber Defence: A Survey». En: (2023). arXiv: [2310.07745](https://arxiv.org/abs/2310.07745) [cs.CR] (vid. pág. 39).
- [52] Kezhou Ren, Jinshu Zhu, Tao Yuan, Sheng Wen y Frank Jiang. «ID-RDRL: A Deep Reinforcement Learning-Based Feature Selection Intrusion Detection Model». En: *Scientific Reports* 12 (2022), pág. 15370. DOI: [10.1038/s41598-022-19366-3](https://doi.org/10.1038/s41598-022-19366-3) (vid. pág. 38).
- [53] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes y Andreas Hotho. «A Survey of Network-Based Intrusion Detection Data Sets». En: *Computers & Security* 86 (2019), págs. 147-167. DOI: [10.1016/j.cose.2019.06.005](https://doi.org/10.1016/j.cose.2019.06.005). arXiv: [1903.02460](https://arxiv.org/abs/1903.02460) (vid. págs. 3, 15, 23, 27, 28, 33, 46, 48, 85).
- [54] Arnaud Rosay, Eloïse Cheval, Florent Carlier y Pascal Leroux. «Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017». En: *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP)*. 2022, págs. 25-36. DOI: [10.5220/0010774000003120](https://doi.org/10.5220/0010774000003120) (vid. págs. 30, 42).
- [55] Takaya Saito y Marc Rehmsmeier. «The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers

- on Imbalanced Datasets». En: *PLOS ONE* 10.3 (2015), e0118432. DOI: [10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432) (vid. págs. 45, 87).
- [56] Mohanad Sarhan, Siamak Layeghy y Marius Portmann. «Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection». En: (2021). Proposes standardised NetFlow-aligned feature mapping across CIC and UNSW-NB15 datasets. arXiv: [2104.07183](https://arxiv.org/abs/2104.07183) [cs.CR] (vid. págs. 26, 34, 56).
- [57] Habeeb Mohammed Sayeeduddin y T. Ranga Babu. «Network Intrusion Detection System: A Survey on Artificial Intelligence-Based Techniques». En: *Expert Systems* (2022). DOI: [10.1111/exsy.13066](https://doi.org/10.1111/exsy.13066) (vid. pág. 33).
- [58] Karen A. Scarfone y Peter M. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Inf. téc. Special Publication 800-94. National Institute of Standards y Technology, 2007. URL: <https://csrc.nist.gov/pubs/sp/800/94/final> (vid. págs. 1, 21, 22, 45).
- [59] Tom Schaul, John Quan, Ioannis Antonoglou y David Silver. «Prioritized Experience Replay». En: *International Conference on Learning Representations*. 2016. arXiv: [1511.05952](https://arxiv.org/abs/1511.05952) (vid. pág. 37).
- [60] Iman Sharafaldin, Arash Habibi Lashkari y Ali A. Ghorbani. «A Detailed Analysis of the CICIDS2017 Data Set». En: *Information Systems Security and Privacy*. Vol. 977. Communications in Computer and Information Science. Springer, 2019, págs. 172-188. DOI: [10.1007/978-3-030-25109-3_9](https://doi.org/10.1007/978-3-030-25109-3_9) (vid. pág. 29).
- [61] Iman Sharafaldin, Arash Habibi Lashkari y Ali A. Ghorbani. «Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization». En: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, págs. 108-116. DOI: [10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116). URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (vid. págs. 5, 25, 28, 29, 77, 116).

- [62] Robin Sommer y Vern Paxson. «Outside the Closed World: On Using Machine Learning for Network Intrusion Detection». En: *2010 IEEE Symposium on Security and Privacy (S&P)*. 2010, págs. 305-316. DOI: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25) (vid. págs. 23, 41, 113, 117).
- [63] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras y Burkhard Stiller. «An Overview of IP Flow-Based Intrusion Detection». En: *IEEE Communications Surveys & Tutorials* 12.3 (2010), págs. 343-356. DOI: [10.1109/SURV.2010.032210.00054](https://doi.org/10.1109/SURV.2010.032210.00054) (vid. págs. 2, 10, 24, 25, 53, 54).
- [64] Stable-Baselines3 Contributors. *QR-DQN: Stable Baselines3 Contrib Documentation*. Software documentation; access date 2026. 2023. URL: <https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html> (vid. págs. 5, 13, 37, 71, 75).
- [65] Maxwell Standen, Martin Lucas, David Bowman, Toby J. Richer, Junae Kim y Damian Marriott. «CybORG: A Gym for the Development of Autonomous Cyber Agents». En: *arXiv preprint arXiv:2108.09118* (2021). Preprint; also cited as an IJCAI-21 Adaptive Cyber Defense workshop paper. arXiv: [2108.09118](https://arxiv.org/abs/2108.09118) [cs.CR] (vid. pág. 39).
- [66] Richard S. Sutton y Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2.^a ed. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html> (vid. págs. 2, 34, 46, 67, 85, 95, 97).
- [67] Mahbod Tavallaei, Ebrahim Bagheri, Wei Lu y Ali A. Ghorbani. «A Detailed Analysis of the KDD CUP 99 Data Set». En: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. 2009, págs. 1-6. DOI: [10.1109/CISDA.2009.5356528](https://doi.org/10.1109/CISDA.2009.5356528) (vid. págs. 27, 28).
- [68] Ashish Thakkar y Rachna Lohiya. «A Review of the Advancement in Intrusion Detection Datasets». En: *Procedia Computer Science* 167 (2020), págs. 636-645. DOI: [10.1016/j.procs.2020.03.270](https://doi.org/10.1016/j.procs.2020.03.270) (vid. pág. 27).

- [69] Muhammad Fahad Umer, Muhammad Sher y Yaxin Bi. «Flow-Based Intrusion Detection: Techniques and Challenges». En: *Computers & Security* 70 (2017), págs. 238-254. DOI: [10.1016/j.cose.2017.05.009](https://doi.org/10.1016/j.cose.2017.05.009) (vid. págs. 2, 10, 24, 25, 53, 54).
- [70] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot y Nando de Freitas. «Dueling Network Architectures for Deep Reinforcement Learning». En: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. Vol. 48. 2016, págs. 1995-2003. URL: <https://proceedings.mlr.press/v48/wangf16.html> (vid. pág. 36).
- [71] Christopher J. C. H. Watkins y Peter Dayan. «Q-Learning». En: *Machine Learning* 8.3-4 (1992), págs. 279-292. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698) (vid. pág. 35).
- [72] Jianping Xiao, Chun Long, Jing Zhao, Jinxia Wei, Anlei Hu y Guanyao Du. «A Survey on Network Intrusion Detection Based on Deep Learning». En: *Frontiers of Data and Computing* 3.3 (2021), págs. 59-74. DOI: [10.11871/jfdc.issn.2096-742X.2021.03.006](https://doi.org/10.11871/jfdc.issn.2096-742X.2021.03.006) (vid. pág. 33).
- [73] Wanrong Yang, Alberto Acuto, Yihang Zhou y Dominik Wojtczak. «A Survey for Deep Reinforcement Learning Based Network Intrusion Detection». En: *Applied AI Letters* 7.2 (2026), e70026. DOI: [10.1002/ai12.70026](https://doi.org/10.1002/ai12.70026). arXiv: [2410.07612](https://arxiv.org/abs/2410.07612) [cs.CR] (vid. págs. 21, 38, 46, 48).